

Learning Quantitative Finance in Rust

A Progressive Journey from Kaggle to Quant Research

Marcelo Correa

July 29, 2026

[bitcrafts/quant-lab-rust](https://github.com/bitcrafts/quant-lab-rust)

The market can stay irrational
longer than you can stay solvent.

– John Maynard Keynes

Preface

This book documents a learning journey: from data science fundamentals to quantitative finance, implemented entirely in Rust. The goal is not just to understand the concepts, but to internalize them through implementation.

Philosophy We start with beginner-friendly Kaggle projects and progressively build toward advanced quant research tools. No black-box libraries — the math is hand-rolled, because the implementation IS the learning.

Structure The book is organized in five parts:

- ▶ **Part I — Foundations:** First Kaggle projects, basic stats
- ▶ **Part II — Core Concepts:** Returns, volatility, backtesting
- ▶ **Part III — Quantitative Methods:** Moments, time series, stationarity
- ▶ **Part IV — Derivatives and Portfolio:** Options, Markowitz, factors
- ▶ **Part V — Advanced Topics:** Microstructure, AFML techniques

How to use Each chapter has:

- ▶ Learning objectives at the start
- ▶ Mathematical foundations with TikZ diagrams
- ▶ Rust implementation with code listings
- ▶ Margin notes with key formulas and insights
- ▶ Exercises and references

The companion code lives at <https://github.com/bitcrafts/quant-lab-rust>. Clone and run examples with:

```
1 git clone https://github.com/bitcrafts/quant-lab-rust.git
2 cd quant-lab-rust
3 cargo run -p qf-01-fraud --example fraud_analysis
```

Contents

FOUNDATIONS	1
0. Foundations: Mathematics, Statistics, and Programming	2
0.1. Why These Foundations Matter	2
0.2. Mathematical Notation	2
0.2.1. Sets and Numbers	2
0.2.2. Summation and Products	2
0.2.3. Functions and Calculus	3
0.3. Statistics Essentials	3
0.3.1. Measures of Central Tendency	3
0.3.2. Measures of Spread	3
0.3.3. Probability Distributions	4
0.3.4. Expected Value and Variance	4
0.4. Financial Concepts	5
0.4.1. What is a Stock?	5
0.4.2. OHLCV Data	5
0.4.3. Returns	5
0.4.4. Risk and Volatility	6
0.4.5. The Risk-Return Tradeoff	6
0.5. Programming in Rust	6
0.5.1. Why Rust?	6
0.5.2. Basic Syntax	6
0.5.3. Traits (Interfaces)	7
0.5.4. Error Handling	7
0.5.5. Iterators	7
0.6. Running the Examples	8
0.6.1. Prerequisites	8
0.6.2. Running Examples	8
0.6.3. Running Tests	8
0.7. Key Takeaways	8
1. Hello Finance: Credit Card Fraud Detection	10
1.1. Learning Objectives	10
1.2. Introduction: Why Fraud Detection?	10
1.3. Data Structure	10
1.3.1. The Dataset	10
1.3.2. Rust Data Model	11
1.3.3. Loading Data	11
1.4. Z-Score Anomaly Detection	11
1.4.1. Statistical Foundation	11
1.4.2. Rust Implementation	12
1.5. Evaluation Metrics	13
1.5.1. Confusion Matrix	13
1.5.2. Derived Metrics	13
1.5.3. Evaluation Function	13
1.6. Toward a Trait-Based Architecture	14

1.7.	Results and Analysis	14
1.7.1.	Running the Example	14
1.7.2.	Interpreting Results	14
1.7.3.	Limitations	15
1.8.	Exercises	15
1.9.	Key Takeaways	15
2.	Risk Basics: Loan Default Prediction	16
2.1.	Learning Objectives	16
2.2.	Introduction: Credit Risk Fundamentals	16
2.3.	Data Model	16
2.3.1.	Loan Application Structure	16
2.4.	Feature Engineering	17
2.4.1.	Derived Features	17
2.4.2.	Categorical Encoding	17
2.4.3.	Feature Normalization	17
2.5.	Linear Scoring Model	18
2.5.1.	Weighted Sum	18
2.5.2.	Probability via Sigmoid	18
2.6.	ROC Curves and AUC	19
2.6.1.	Why Not Just Accuracy?	19
2.6.2.	ROC Curve Construction	19
2.6.3.	Area Under Curve (AUC)	19
2.7.	Trait-Based Architecture	20
2.8.	Results and Analysis	20
2.8.1.	Example Usage	20
2.8.2.	Interpreting the Results	20
2.9.	Exercises	21
2.10.	Key Takeaways	21
3.	Market Data: Stock Price Analysis	22
3.1.	Learning Objectives	22
3.2.	Introduction: What is OHLCV?	22
3.3.	Data Structure	22
3.3.1.	Rust Implementation	22
3.3.2.	Data Validation	23
3.4.	Candlestick Anatomy	23
3.4.1.	Rust Implementation	23
3.5.	Market Statistics	24
3.5.1.	Basic Aggregations	24
3.6.	Moving Averages	24
3.6.1.	Simple Moving Average (SMA)	24
3.6.2.	Exponential Moving Average (EMA)	24
3.6.3.	SMA vs EMA	25
3.7.	TimeSeries Trait	25
3.8.	Golden and Death Crosses	26
3.9.	Results and Analysis	26
3.9.1.	Example Output	26
3.10.	Exercises	26
3.11.	Key Takeaways	27

CORE CONCEPTS	28
4. Returns and Volatility	29
4.1. Learning Objectives	29
4.2. Simple vs Log Returns	29
4.2.1. Definitions	29
4.2.2. When to use which	30
4.2.3. Rust implementation	30
4.3. Volatility	31
4.3.1. Annualisation	31
4.3.2. Rolling volatility	31
4.4. Risk-Adjusted Performance	32
4.4.1. Sharpe ratio	32
4.4.2. Sortino ratio	32
4.5. Drawdown	33
4.6. The Returns Trait	33
4.7. Example Output	33
4.8. Exercises	34
4.9. Key Takeaways	34
5. Backtesting Strategies	35
5.1. Learning Objectives	35
5.2. What Is Backtesting?	35
5.3. Signals and Positions	35
5.4. The Strategy Trait	36
5.5. SMA Crossover	36
5.6. P&L with Transaction Costs	37
5.7. Performance Evaluation	37
5.8. Buy-and-Hold Benchmark	38
5.9. Common Pitfalls	38
5.10. Example Output	38
5.11. Walk-Forward Analysis	39
5.11.1. Walk-Forward Efficiency	39
5.11.2. Implementation	39
5.11.3. Visual Representation	39
5.11.4. Anchored vs Rolling	40
5.12. Exercises	40
5.13. Key Takeaways	40
QUANTITATIVE METHODS	42
6. Foundations: Moments, RNG, and Simulation	43
6.1. Learning Objectives	43
6.2. A Validated Price Series	43
6.3. Returns, Again	44
6.4. Moments	44
6.5. Rolling Windows	45
6.6. Hand-Rolled Simulation	45
6.6.1. xorshift64	45
6.6.2. Box-Muller Normal	46
6.6.3. Geometric Brownian Motion	46
6.7. Fat Tails	47
6.8. Exercises	47
6.9. Key Takeaways	48

7. Time Series Analysis	49
7.1. Learning Objectives	49
7.2. Stationarity	49
7.3. OLS via Gaussian Elimination	50
7.4. Autocorrelation Function	50
7.5. Augmented Dickey-Fuller Test	51
7.6. Fractional Differentiation	51
7.6.1. Finding the Minimum d	52
7.7. The Stationarity-Memory Tradeoff in Action	52
7.8. Exercises	53
7.9. Key Takeaways	54
8. Volatility Models: EWMA, ARCH, and GARCH	55
8.1. Learning Objectives	55
8.2. Stylized Facts of Volatility	55
8.3. EWMA: Exponentially Weighted Moving Average	56
8.3.1. Boundary cases	56
8.3.2. Unrolling the recursion	56
8.4. ARCH: Autoregressive Conditional Heteroskedasticity	57
8.4.1. Gaussian log-likelihood	57
8.5. GARCH: Generalised ARCH	58
8.5.1. Persistence and stationarity	59
8.5.2. Half-life of shocks	59
8.5.3. Forecasting	59
8.6. Maximum Likelihood by Hand-Rolled Optimisation	59
8.6.1. Nelder-Mead simplex	59
8.6.2. Coordinate ascent with multi-start	60
8.7. Implementation in Rust	60
8.8. Volatility Clustering in Action	61
8.9. Parameter Recovery	62
8.10. Exercises	62
8.11. Key Takeaways	62
9. Stochastic Processes and Monte Carlo	64
9.1. Learning Objectives	64
9.2. Brownian Motion	64
9.2.1. Quadratic Variation	65
9.3. Geometric Brownian Motion	65
9.4. Poisson Process and Jump-Diffusion	65
9.4.1. Merton Jump-Diffusion	65
9.5. The Monte Carlo Method	66
9.5.1. The $1/\sqrt{N}$ Law	66
9.5.2. Antithetic Variates	66
9.6. Black-Scholes: The Analytical Benchmark	66
9.7. Implementation in Rust	66
9.7.1. Brownian Motion	66
9.7.2. Monte Carlo Call	67
9.8. Results and Analysis	67
9.8.1. Monte Carlo Convergence	67
9.8.2. The $1/\sqrt{N}$ Law	67
9.8.3. Antithetic Variance Reduction	68
9.8.4. Quadratic Variation	68
9.8.5. GBM Terminal Distribution	68
9.9. Exercises	68

9.10. Key Takeaways	69
DERIVATIVES AND PORTFOLIO THEORY	70
10. Options Pricing: Black-Scholes and Beyond	71
10.1. Learning Objectives	71
10.2. The Black-Scholes PDE	71
10.3. The Greeks as Partial Derivatives	72
10.4. Analytical Greeks	72
10.5. Numerical Greeks by Finite Difference	73
10.6. Implied Volatility	73
10.7. Put-Call Parity and Implied Vol	74
10.8. Results and Analysis	74
10.8.1. Greeks for an ATM Call	74
10.8.2. Implied Volatility Recovery	75
10.8.3. Volatility Smile	75
10.9. Exercises	75
10.10. Key Takeaways	76
11. Portfolio Optimization: Markowitz and Beyond	77
11.1. Learning Objectives	77
11.2. The Markowitz Problem	77
11.3. Two-Asset Closed-Form Frontier	78
11.4. The n -Asset Lagrangian	79
11.4.1. Global Minimum-Variance Portfolio	79
11.4.2. Efficient Frontier at a Target Return	79
11.5. The Tangency Portfolio	80
11.6. The Capital Market Line and Two-Fund Separation	80
11.7. CAPM: Beta, Alpha, and the SML	81
11.8. Risk Metrics: Historical VaR and CVaR	81
11.9. Results and Analysis	82
11.9.1. Two-Asset Universe	82
11.9.2. Tangency Portfolio and CML	82
11.9.3. N-Asset Efficient Frontier (Lagrangian)	83
11.9.4. CAPM Regression	83
11.10. Exercises	84
11.11. Key Takeaways	84
12. Factor Attribution: PCA and Fama-French	85
12.1. Learning Objectives	85
12.2. Why Multi-Factor?	85
12.3. Eigenvalue Decomposition: The Power Method	86
12.4. Deflation and the Full Spectrum	87
12.5. PCA: Covariance Eigendecomposition	87
12.5.1. Demo Results	88
12.6. Fama-French 3-Factor Regression	88
12.6.1. Demo Results	89
12.7. Risk Attribution	89
12.8. Exercises	90
12.9. Key Takeaways	91

ADVANCED TOPICS	92
13. Market Microstructure	93
13.1. Learning Objectives	93
13.2. Why Microstructure Matters	93
13.3. The Limit Order Book	94
13.3.1. Core Types	94
13.3.2. Adding and Cancelling Orders	94
13.3.3. Market Orders and Price-Time Priority	95
13.3.4. Top-of-Book Accessors	95
13.4. Order Flow Imbalance	96
13.4.1. VWAP and Trade Imbalance	96
13.5. Market Impact Models	96
13.5.1. Execution Cost	97
13.6. Numerical Demonstration	97
13.7. Complexity and Design Notes	98
13.8. Exercises	98
13.9. Key Takeaways	99
14. AFML: Backtesting with Triple Barriers and Purged CV	100
14.1. Learning Objectives	100
14.2. Why Traditional Backtesting Fails	100
14.3. The Triple-Barrier Method	101
14.3.1. Configuration and Labels	101
14.3.2. Labeling a Single Event	101
14.3.3. Binary Labels	102
14.4. Sample Uniqueness and Weighting	102
14.5. Purged K-Fold Cross-Validation	103
14.5.1. Embargo	103
14.6. Bet Sizing: The Kelly Criterion	104
14.6.1. Position Sizing	104
14.7. The AFML Pipeline	105
14.8. Meta-Labeling (Brief)	106
14.9. Exercises	106
14.10. Key Takeaways	106
14.11. References	107
15. quant-lib: The Unified Facade	108
15.1. Learning Objectives	108
15.2. Introduction: From Crates to a Library	108
15.3. The Facade Pattern	108
15.4. Module Map	109
15.5. The Prelude	109
15.6. Feature Flags for Module Gating	110
15.7. Root-Level Convenience Re-exports	110
15.8. Composable Backtest Architecture	111
15.8.1. The Builder API	111
15.8.2. What the Builder Checks	111
15.9. The End-to-End Pipeline	111
15.10. The 16-Example Tour	112
15.11. Non-Destructive by Design	113
15.12. Versioning Strategy	113
15.13. When to Use quant-lib vs. Direct Crates	114
15.14. Exercises	114
15.15. Summary	114

APPENDICES	116
A. Mathematical Notation	117
A.1. General Conventions	117
A.2. Probability and Statistics	117
A.2.1. Moments	117
A.3. Returns	118
A.4. Risk-Adjusted Performance	118
A.5. Time Series	118
A.6. Volatility Models	119
A.7. Stochastic Processes	119
A.8. Options (Black-Scholes)	119
A.8.1. Pricing	119
A.8.2. Greeks	120
A.9. Portfolio Theory	120
A.9.1. Frontier	120
A.10. Factor Models	120
A.11. Market Microstructure	120
A.11.1. Market Impact (Almgren-Chriss)	121
A.12. AFML Backtesting	121
A.13. Common Subscripts	121
B. Rust Patterns for Finance	122
B.1. The Trait as Extension Point	122
B.2. Pluggable Component Trait Catalogue	122
B.3. Generic Engines with Static Dispatch	123
B.4. The Builder Pattern for Composable Backtests	124
B.5. Newtype Wrappers for Domain Safety	124
B.6. Error Handling with Result	124
B.7. Iterators and Lazy Evaluation	125
B.8. Feature Flags for Module Gating	125
B.9. The Facade Crate Pattern	126
B.10. Deterministic Randomness	126
B.11. Testing Strategy	126
C. Dataset Sources	127
C.1. Bundled Datasets	127
C.1.1. Synthetic Stock Prices (stock_prices.csv)	127
C.1.2. B3 Brazilian Stocks (8 JSON files)	127
C.1.3. Credit Card Fraud Sample (creditcard_sample.csv)	128
C.2. Public Data Sources for Extending the Examples	128
C.2.1. Equity OHLCV	128
C.2.2. Factor and Macro Data	129
C.2.3. Options Data	129
C.2.4. Microstructure and LOB Data	129
C.3. Loading Data in Practice	129
C.4. Reproducibility and Licensing	130
D. Exercise Solutions	131
E. Mathematical Derivations	141
E.1. E.1: Kelly Criterion from Expected Log Utility	141
E.1.1. Setup	141
E.1.2. First-Order Condition	141
E.1.3. Interpretation	142

E.1.4.	Second-Order Condition	142
E.1.5.	Continuous-Time Limit	142
E.1.6.	Fractional Kelly	142
E.2.	E.2: Triple-Barrier Return Distribution	143
E.2.1.	Setup	143
E.2.2.	Log-Price Transformation	143
E.2.3.	Hitting Probabilities	143
E.2.4.	Time Barrier Contribution	144
E.2.5.	Final Label Distribution	144
E.2.6.	Zero-Drift Simplification	144
E.3.	E.3: Hierarchical Risk Parity Recursive Bisection	144
E.3.1.	Setup	144
E.3.2.	Recursive Bisection Algorithm	145
E.3.3.	Property 1: Weights Sum to One	145
E.3.4.	Property 2: Scale Invariance	145
E.3.5.	Property 3: Reduction to Inverse Variance	146
E.3.6.	Why HRP Outperforms Markowitz in Singular Cases	146
E.4.	E.4: Combinatorial Purged Cross-Validation	146
E.4.1.	Setup	146
E.4.2.	Construction	146
E.4.3.	Path Count	147
E.4.4.	Worked Example	147
E.4.5.	Purging	148
E.4.6.	Property: Every Observation Appears in Test Exactly Once	148
E.5.	E.5: Backpropagation Chain Rule	148
E.5.1.	Setup	148
E.5.2.	The Chain Rule	149
E.5.3.	Complexity	149
E.5.4.	Numerical Gradient Check	150
E.6.	E.6: Adam Optimiser Derivation	150
E.6.1.	Setup	150
E.6.2.	Bias-Corrected Moment Estimates	150
E.6.3.	Update Rule	151
E.6.4.	Approximate Invariance to Gradient Scale	151
E.6.5.	Connection to RMSProp and Momentum	151
E.6.6.	Convergence Sketch	152
E.6.7.	Practical Default Settings	152
E.7.	Summary	152

List of Figures

2.1. ROC curve: the blue line shows classifier performance, dashed line is random guessing. Area under curve (AUC) is shaded.	19
5.1. Walk-forward rolling windows: each training window slides forward, testing on the next unseen period. Unlike k-fold, every test window is strictly after its training window.	39
11.1. Markowitz efficient frontier for two uncorrelated risky assets.	80
12.1. Scree plot: explained variance per principal component. The first PC captures 96% of the total variance; the second adds 3.9%; the third adds only 0.3%.	86
13.1. Resting depth at three price levels on each side. Best bid at 100 (40 shares), best ask at 101 (30 shares), spread = 1, mid = 100.5.	94
14.1. Triple barriers around the entry price. The first barrier touched — here the upper barrier at time T — determines the label.	101

List of Tables

FOUNDATIONS

Foundations: Mathematics, Statistics, and Programming

0.

This chapter establishes the mathematical, statistical, and programming foundations needed to understand the rest of the book. If you are already comfortable with these topics, feel free to skip ahead and use this chapter as a reference.

0.1. Why These Foundations Matter

Quantitative finance sits at the intersection of three disciplines:

- **Mathematics:** The language of precision—formulas, proofs, and rigorous reasoning
- **Statistics:** The science of uncertainty—data analysis, probability, and inference
- **Programming:** The tool of implementation—turning ideas into working systems

You don't need to be an expert in all three to start. This book teaches by doing—each concept is introduced when needed and immediately applied.

The Quant Mindset

A quantitative approach means making decisions based on data and models, not intuition or emotion.

0.2. Mathematical Notation

Throughout this book, we use standard mathematical notation. Here's a quick reference:

0.2.1. Sets and Numbers

Symbol	Meaning
$\mathbb{R}$	Real numbers (e.g., 3.14, -2.5 , $\sqrt{2}$)
$\mathbb{R}^+$	Positive real numbers
$\mathbb{Z}$	Integers ($\dots, -2, -1, 0, 1, 2, \dots$)
$\mathbb{N}$	Natural numbers ($1, 2, 3, \dots$)
$\in$	"is an element of" (e.g., $x \in \mathbb{R}$ means x is real)
$\forall$	"for all"
$\exists$	"there exists"

0.2.2. Summation and Products

The summation symbol Σ means "add up":

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$$

For example, if $x = [2, 5, 3]$, then $\sum_{i=1}^3 x_i = 2 + 5 + 3 = 10$.

Summation

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$$

Product

$$\prod_{i=1}^n x_i = x_1 \times x_2 \times \dots \times x_n$$

The product symbol $\prod$ means “multiply together”:

$$\prod_{i=1}^n x_i = x_1 \times x_2 \times \cdots \times x_n$$

0.2.3. Functions and Calculus

Symbol	Meaning
$f(x)$	Function f evaluated at x
$f'(x)$ or $\frac{df}{dx}$	Derivative of f with respect to x
$\frac{\partial f}{\partial x}$	Partial derivative (when f has multiple variables)
$\ln(x)$	Natural logarithm (base $e \approx 2.718$)
$\exp(x)$ or e^x	Exponential function
$\operatorname{argmax}_x f(x)$	Value of x that maximizes f
$\operatorname{argmin}_x f(x)$	Value of x that minimizes f

Key Insight

The natural logarithm $\ln$ is everywhere in finance. Key property: $\ln(a \times b) = \ln(a) + \ln(b)$. This turns multiplication (compounding) into addition, making calculations much easier.

0.3. Statistics Essentials

Statistics helps us understand and quantify uncertainty—essential when dealing with financial markets.

0.3.1. Measures of Central Tendency

The **mean** (average) is the sum divided by the count:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The **median** is the middle value when data is sorted. It’s more robust to outliers than the mean.

0.3.2. Measures of Spread

Variance measures how spread out the data is:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Variance

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Standard deviation is the square root of variance:

$$\sigma = \sqrt{\sigma^2}$$

Standard deviation has the same units as the original data, making it easier to interpret than variance.

```

1 /// Calculate mean of a slice
2 pub fn mean(data: &[f64]) -> f64 {
3     if data.is_empty() { return 0.0; }
4     data.iter().sum::<f64>() / data.len() as f64
5 }
6
7 /// Calculate variance of a slice
8 pub fn variance(data: &[f64]) -> f64 {
9     if data.is_empty() { return 0.0; }
10    let m = mean(data);
11    data.iter().map(|x| (x - m).powi(2)).sum::<f64>() / data.len() as f64
12 }
13
14 /// Calculate standard deviation
15 pub fn std_dev(data: &[f64]) -> f64 {
16     variance(data).sqrt()
17 }

```

0.3.3. Probability Distributions

A **probability distribution** describes how likely different outcomes are.

Normal Distribution (Gaussian)

The normal distribution is the famous “bell curve”:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ is the standard deviation.

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to say “ X follows a normal distribution with mean μ and variance σ^2 .”

Normal PDF

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Warning

Financial returns are NOT normally distributed! They have “fat tails”—extreme events happen much more often than the normal distribution predicts. We explore this in Chapter 6.

Uniform Distribution

Every outcome in a range is equally likely:

$$f(x) = \frac{1}{b-a} \quad \text{for } a \leq x \leq b$$

0.3.4. Expected Value and Variance

The **expected value** $\mathbb{E}[X]$ is the average outcome you’d expect over many trials:

For a discrete random variable:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

Expected Value

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

(discrete)

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x) dx$$

(continuous)

Key properties:

- ▶ $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$ (linearity)
- ▶ $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (additivity)
- ▶ $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

0.4. Financial Concepts

Before diving into quantitative methods, let's establish some fundamental financial concepts.

0.4.1. What is a Stock?

A **stock** (or share) represents partial ownership of a company. When you buy a stock, you own a small piece of that company and are entitled to a share of its profits.

The **stock price** is determined by supply and demand in the market. It reflects what buyers are willing to pay and sellers are willing to accept.

Stock Price

The price at which buyers and sellers agree to trade ownership.

0.4.2. OHLCV Data

Market data is typically recorded as **OHLCV**:

- ▶ **Open**: First price of the trading period
- ▶ **High**: Highest price during the period
- ▶ **Low**: Lowest price during the period
- ▶ **Close**: Last price of the period
- ▶ **Volume**: Number of shares traded

This data captures the essential price action within any time frame (day, hour, minute).

0.4.3. Returns

A **return** measures how much an investment gained or lost:

Simple return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log return (continuously compounded):

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Simple Return

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log Return

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log returns have a useful property: they add up over time.

$$r_{t_1 \rightarrow t_n} = r_{t_1} + r_{t_2} + \dots + r_{t_n}$$

0.4.4. Risk and Volatility

Risk in finance is typically measured by **volatility**—the standard deviation of returns.

Higher volatility means more uncertainty—prices swing more wildly.

Annualized volatility scales daily volatility to a yearly figure:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

(252 is the typical number of trading days per year.)

Volatility

$$\sigma = \text{std}(r_t)$$

$$\text{Often annualized: } \sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

0.4.5. The Risk-Return Tradeoff

Key Insight

Higher expected returns generally require accepting higher risk. There is no free lunch in finance—if someone promises high returns with low risk, be skeptical.

The **Sharpe ratio** measures risk-adjusted return:

$$\text{Sharpe} = \frac{\mathbb{E}[R] - R_f}{\sigma}$$

where R_f is the risk-free rate (e.g., Treasury bills).

0.5. Programming in Rust

This book uses Rust for all implementations. Here's a quick introduction for readers unfamiliar with the language.

0.5.1. Why Rust?

- ▶ **Performance:** As fast as C/C++, suitable for high-frequency trading
- ▶ **Safety:** The compiler catches bugs before runtime
- ▶ **Modern:** Great tooling, package management, and community
- ▶ **Learning:** Writing correct Rust forces you to think carefully—great for learning

0.5.2. Basic Syntax

```

1 // Variables (immutable by default)
2 let x = 5;
3 let mut y = 10; // mutable
4 y = 20;
5
6 // Functions
7 fn add(a: i32, b: i32) -> i32 {
8     a + b // no semicolon = return value
9 }
10
11 // Structs (like classes)
12 struct Stock {
13     symbol: String,
14     price: f64,
15 }
```

```

16
17 // Implementation block
18 impl Stock {
19     fn new(symbol: &str, price: f64) -> Self {
20         Stock {
21             symbol: symbol.to_string(),
22             price,
23         }
24     }
25
26     fn value(&self, shares: u32) -> f64 {
27         self.price * shares as f64
28     }
29 }
30
31 // Using the struct
32 let aapl = Stock::new("AAPL", 150.0);
33 let portfolio_value = aapl.value(100);

```

0.5.3. Traits (Interfaces)

Traits define shared behavior:

```

1 // Define a trait
2 trait Analyzer {
3     fn analyze(&self, data: &[f64]) -> f64;
4 }
5
6 // Implement for a struct
7 struct MeanAnalyzer;
8
9 impl Analyzer for MeanAnalyzer {
10     fn analyze(&self, data: &[f64]) -> f64 {
11         data.iter().sum::<f64>() / data.len() as f64
12     }
13 }
14
15 // Use generically
16 fn run_analysis<A: Analyzer>(analyzer: &A, data: &[f64]) -> f64 {
17     analyzer.analyze(data)
18 }

```

Traits

Rust's way of defining shared behavior. Similar to interfaces in Java or protocols in Swift.

This book uses traits extensively to create modular, reusable components.

0.5.4. Error Handling

```

1 // Result type for operations that can fail
2 fn divide(a: f64, b: f64) -> Result<f64, String> {
3     if b == 0.0 {
4         Err("Division by zero".to_string())
5     } else {
6         Ok(a / b)
7     }
8 }
9
10 // Using Result
11 match divide(10.0, 2.0) {
12     Ok(result) => println!("Result: {}", result),
13     Err(e) => println!("Error: {}", e),
14 }
15
16 // Or use the ? operator
17 fn calculate() -> Result<f64, String> {
18     let x = divide(10.0, 2.0)?; // propagates error
19     Ok(x * 2.0)
20 }

```

0.5.5. Iterators

Rust's iterators are powerful and efficient:

```

1 let prices = vec![100.0, 102.0, 101.0, 105.0];
2
3 // Map and collect
4 let returns: Vec<f64> = prices.windows(2)
5   .map(|w| (w[1] - w[0]) / w[0])
6   .collect();
7
8 // Filter
9 let positive: Vec<f64> = returns.iter()
10  .filter(|&r| r > 0.0)
11  .copied()
12  .collect();
13
14 // Fold (reduce)
15 let sum: f64 = prices.iter().sum();
16 let product: f64 = prices.iter().product();

```

0.6. Running the Examples

All code examples in this book are available in the companion repository.

0.6.1. Prerequisites

You need Rust and a clone of the companion repository:

```

1 # 1. Install Rust (if not already installed)
2 curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
3
4 # 2. Clone the companion repository
5 git clone https://github.com/bitscrafts/quant-lab-rust.git
6 cd quant-lab-rust

```

0.6.2. Running Examples

Each chapter has a corresponding crate with examples:

```

1 # Chapter 1: Credit Card Fraud
2 cargo run -p qf-01-fraud --example fraud_analysis
3
4 # Chapter 2: Loan Default
5 cargo run -p qf-02-loan --example loan_analysis
6
7 # Chapter 3: Stock Prices
8 cargo run -p qf-03-stocks --example stock_analysis

```

0.6.3. Running Tests

```

1 # Run all tests
2 cargo test
3
4 # Run tests for a specific crate
5 cargo test -p qf-01-fraud

```

0.7. Key Takeaways

This chapter covered:

- **Mathematical notation:** Summation, products, functions, logarithms
- **Statistics:** Mean, variance, standard deviation, distributions

- ▶ **Financial concepts:** Stocks, OHLCV, returns, volatility, Sharpe ratio
- ▶ **Rust programming:** Variables, functions, structs, traits, error handling

Next chapter: We apply these foundations to detect credit card fraud using anomaly detection.

Hello Finance: Credit Card Fraud Detection

1.

Companion Code:

```
qf-01-fraud Code: crates/qf-01-fraud/  
Dependencies: qf-common  
Run: cargo run -p qf-01-fraud -example fraud_analysis
```

1.1. Learning Objectives

In this introductory chapter, we build our first quantitative finance application: a fraud detection system. Along the way, we learn:

- ▶ How to load and parse financial CSV datasets in Rust
- ▶ Basic statistical measures: mean and standard deviation
- ▶ Z-score anomaly detection—a simple but effective baseline
- ▶ Evaluation metrics: confusion matrix, precision, recall, F1 score
- ▶ The trait-based architecture that will guide all future implementations

Key Insight

This is not throwaway code. Every component we build follows a **library-first design**—composable, testable, and reusable across future projects.

1.2. Introduction: Why Fraud Detection?

Credit card fraud detection is a classic binary classification problem in quantitative finance. Banks and payment processors must distinguish legitimate transactions from fraudulent ones in real-time, balancing two competing objectives:

- ▶ **Precision:** Minimize false positives (legitimate transactions flagged as fraud). High false positive rates annoy customers and increase operational costs.
- ▶ **Recall:** Maximize detection of actual fraud. Missed fraud means direct financial loss.

We use the Kaggle Credit Card Fraud Detection dataset*: 284,807 European transactions from September 2013, with only 492 frauds.

Class Imbalance

492 frauds in 284,807 transactions = 0.172%. This extreme imbalance is typical in financial anomaly detection.

1.3. Data Structure

1.3.1. The Dataset

The dataset contains:

- ▶ **Time:** Seconds elapsed since first transaction

* <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

- **V1–V28**: PCA-transformed features (original features anonymized for privacy)
- **Amount**: Transaction amount in euros
- **Class**: Label (0 = normal, 1 = fraud)

Why PCA features?

Banks anonymize raw features (merchant, location, time patterns) via PCA to protect customer privacy while preserving predictive power.

1.3.2. Rust Data Model

We define a Transaction struct in qf-common:

```
1 /// Represents a single financial transaction.
2 #[derive(Debug, Clone)]
3 pub struct Transaction {
4     /// PCA-transformed features (V1, V2, ..., V28).
5     pub features: Vec<f64>,
6
7     /// Transaction amount in dollars/euros.
8     pub amount: f64,
9
10    /// Class label: 0 = normal, 1 = fraud.
11    pub class: u8,
12 }
```

1.3.3. Loading Data

The CSV loader handles errors gracefully using Rust's Result type:

```
1 pub fn load_transactions(path: impl AsRef<Path>)
2     -> Result<Vec<Transaction>, CommonError>
3 {
4     if !path.as_ref().exists() {
5         return Err(CommonError::FileNotFound(
6             path.as_ref().display().to_string()
7         ));
8     }
9
10    let mut reader = csv::Reader::from_path(path)?;
11    let mut transactions = Vec::new();
12
13    for result in reader.records() {
14        let record = result?;
15        // Parse features, amount, class...
16        transactions.push(Transaction { features, amount, class });
17    }
18
19    Ok(transactions)
20 }
```

1.4. Z-Score Anomaly Detection

1.4.1. Statistical Foundation

Our baseline detector uses **z-score outlier detection**. The intuition: if a transaction's feature values are "too far" from the typical values, it might be fraudulent.

For each feature i :

1. Compute mean μ_i and standard deviation σ_i from training data
2. For a new transaction with feature value x_i , compute the z-score:

$$z_i = \frac{|x_i - \mu_i|}{\sigma_i}$$

3. Flag as fraud if **any** feature exceeds a threshold (typically $z > 3$)

Z-Score Formula

$$z = \frac{|x - \mu|}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Z-Score Detection Visualization

Imagine a normal (bell) curve. The threshold at $z = 3$ corresponds to the tails of the distribution—only 0.3% of normally distributed data falls beyond $\pm 3\sigma$. Transactions in these extreme regions are flagged as potential fraud.

1.4.2. Rust Implementation

The ZScoreDetector follows our trait-based design philosophy:

```
1 #[derive(Debug, Clone)]
2 pub struct ZScoreDetector {
3     threshold: f64,
4     feature_means: Vec<f64>,
5     feature_stds: Vec<f64>,
6 }
7
8 impl ZScoreDetector {
9     pub fn new(threshold: f64) -> Self {
10         Self {
11             threshold,
12             feature_means: Vec::new(),
13             feature_stds: Vec::new(),
14         }
15     }
16
17     pub fn fit(&mut self, transactions: &[Transaction]) {
18         if transactions.is_empty() { return; }
19         let num_features = transactions[0].features.len();
20
21         // Initialise mean/std accumulators to zero
22         self.feature_means = vec![0.0; num_features];
23         self.feature_stds = vec![0.0; num_features];
24
25         // Compute mean for each feature
26         for tx in transactions {
27             for (i, &value) in tx.features.iter().enumerate() {
28                 self.feature_means[i] += value;
29             }
30         }
31         for mean in &mut self.feature_means {
32             *mean /= transactions.len() as f64;
33         }
34
35         // Compute standard deviation
36         for tx in transactions {
37             for (i, &value) in tx.features.iter().enumerate() {
38                 let diff = value - self.feature_means[i];
39                 self.feature_stds[i] += diff * diff;
40             }
41         }
42         for std in &mut self.feature_stds {
43             *std = (*std / transactions.len() as f64).sqrt();
44         }
45     }
46
47     pub fn predict(&self, transaction: &Transaction) -> bool {
48         for (i, &value) in transaction.features.iter().enumerate() {
49             let z = (value - self.feature_means[i]).abs()
50                 / self.feature_stds[i];
51             if z > self.threshold {
52                 return true; // Anomaly detected
53             }
54         }
55         false
56     }
57 }
```

The 68-95-99.7 Rule

For normally distributed data:

$\pm 1\sigma$: 68.3%

$\pm 2\sigma$: 95.4%

$\pm 3\sigma$: 99.7%

So $z > 3$ catches only 0.3% of normal data as false positives.

Warning

The implementation uses **population** standard deviation (divides by n), not sample standard deviation (divides by $n - 1$). For large datasets this difference is negligible, but be aware of it.

1.5. Evaluation Metrics

1.5.1. Confusion Matrix

Performance is measured via the confusion matrix, which counts:

- ▶ **TP** (True Positives): Fraud correctly identified
- ▶ **FP** (False Positives): Normal flagged as fraud
- ▶ **TN** (True Negatives): Normal correctly identified
- ▶ **FN** (False Negatives): Fraud missed

```
1 #[derive(Debug, Clone, Copy)]
2 pub struct ConfusionMatrix {
3     pub tp: usize,
4     pub fp: usize,
5     pub tn: usize,
6     pub fn_: usize, // 'fn' is a Rust keyword
7 }
```

	Actual	
Pred	TP	FP
	FN	TN

1.5.2. Derived Metrics

From the confusion matrix, we compute:

- ▶ **Precision**: Of predicted frauds, how many are real?

$$P = \frac{TP}{TP + FP}$$

- ▶ **Recall** (Sensitivity): Of actual frauds, how many did we catch?

$$R = \frac{TP}{TP + FN}$$

- ▶ **F1 Score**: Harmonic mean of precision and recall

$$F_1 = \frac{2PR}{P + R}$$

Metric Formulas

$$\text{Precision: } \frac{TP}{TP + FP}$$

$$\text{Recall: } \frac{TP}{TP + FN}$$

$$\text{F1: } \frac{2 \cdot P \cdot R}{P + R}$$

```
1 impl ConfusionMatrix {
2     pub fn precision(&self) -> f64 {
3         let denom = self.tp + self.fp;
4         if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
5     }
6
7     pub fn recall(&self) -> f64 {
8         let denom = self.tp + self.fn_;
9         if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
10    }
11
12    pub fn f1(&self) -> f64 {
13        let p = self.precision();
14        let r = self.recall();
15        if p + r == 0.0 { 0.0 } else { 2.0 * p * r / (p + r) }
16    }
17 }
```

1.5.3. Evaluation Function

The `evaluate()` function runs the detector on test data:

```
1 pub fn evaluate(
2     detector: &ZScoreDetector,
3     transactions: &[Transaction]
4 ) -> ConfusionMatrix {
5     let mut tp = 0; let mut fp = 0;
6     let mut tn = 0; let mut fn_ = 0;
7
8     for tx in transactions {
```

```

9   let predicted = detector.predict(tx);
10  let actual = tx.class == 1;
11
12  match (predicted, actual) {
13    (true, true) => tp += 1,
14    (true, false) => fp += 1,
15    (false, false) => tn += 1,
16    (false, true) => fn_ += 1,
17  }
18 }
19
20 ConfusionMatrix { tp, fp, tn, fn_ }
21 }

```

1.6. Toward a Trait-Based Architecture

Key Insight

This implementation will evolve into `quant-lab`. Before writing any struct, we ask: **what trait should this implement?**

While this chapter uses concrete types, later chapters extract traits:

```

1 // Future: trait-based design
2 pub trait AnomalyDetector {
3   fn fit(&mut self, data: &[Transaction]) -> Result<(), Error>;
4   fn predict(&self, tx: &Transaction) -> bool;
5   fn score(&self, tx: &Transaction) -> f64; // anomaly score
6 }
7
8 pub trait Evaluator<M> {
9   fn evaluate(&self, preds: &[bool], actuals: &[bool]) -> M;
10 }
11
12 // Then any detector + any evaluator compose:
13 fn backtest<D, E>(detector: &D, eval: &E, data: &[Transaction])
14 where
15   D: AnomalyDetector,
16   E: Evaluator<ConfusionMatrix>,
17 { /* ... */ }

```

Future Traits

AnomalyDetector
BinaryClassifier
Evaluator<T>
DataSource

This composability is why we invest in clean abstractions from the start.

1.7. Results and Analysis

1.7.1. Running the Example

```

1 $ cargo run -p qf-01-fraud --example fraud_analysis
2 Loading transactions from data/creditcard.csv...
3 Loaded 284807 transactions (492 frauds)
4 Training detector with threshold 3.0...
5 Evaluating on training set...
6
7 Results:
8   True Positives: 312
9   False Positives: 1847
10  True Negatives: 282468
11  False Negatives: 180
12
13 Precision: 0.1446
14 Recall:    0.6341
15 F1 Score:  0.2356

```

1.7.2. Interpreting Results

The results reveal characteristic tradeoffs:

- ▶ **Recall = 0.63:** We catch 63% of frauds. Not bad for a simple baseline!
- ▶ **Precision = 0.14:** Of our “fraud” predictions, only 14% are real. This means many false alarms.
- ▶ **F1 = 0.24:** The harmonic mean reflects this imbalance.

1.7.3. Limitations

Warning

We evaluated on the training set—a cardinal sin! True performance requires a held-out test set. Exercise 3 addresses this.

This baseline approach has several fundamental limitations:

1. **Gaussian assumption:** Z-scores assume normally distributed features. Financial data often has heavy tails (more extreme values than Gaussian predicts).
2. **Equal feature weighting:** All features are treated equally. Some PCA components may be more predictive than others.
3. **Independent features:** We check each feature separately. Sophisticated fraud may only be detectable via feature *combinations*.
4. **Fixed threshold:** The threshold $z = 3$ is arbitrary. ROC analysis (Chapter 2) shows how to optimize it.

1.8. Exercises

1. **Threshold Sensitivity:** Try thresholds of 2.0, 2.5, 3.0, 3.5, 4.0. Plot precision vs. recall. What threshold maximizes F1?
2. **Feature Analysis:** Modify `predict()` to return which feature(s) triggered the alert. Which features are most discriminative for fraud?
3. **Train/Test Split:** Split the data 80/20 chronologically (not randomly—why?). How does test-set performance compare to training-set?
4. **Amount Feature:** The current implementation ignores the `amount` field. Add it as a feature. Does performance improve?
5. **Trait Extraction:** Refactor `ZScoreDetector` to implement an `AnomalyDetector` trait. What methods should the trait require?

1.9. Key Takeaways

- ▶ **CSV loading** in Rust with proper error handling via `Result`
- ▶ **Basic statistics:** mean, standard deviation, z-score
- ▶ **Binary classification metrics:** confusion matrix, precision, recall, F1
- ▶ **Precision-recall tradeoff:** you can’t maximize both simultaneously
- ▶ **Evaluation pitfalls:** never evaluate on training data!
- ▶ **Library-first design:** even simple code should be composable

Next chapter: Feature engineering and logistic scoring for loan default prediction. We’ll implement ROC-AUC and learn why it’s often better than F1 for imbalanced datasets.

Risk Basics: Loan Default Prediction 2.

Companion Code:

```
qf-02-loan Code: crates/qf-02-loan/  
Dependencies: qf-common  
Run: cargo run -p qf-02-loan -example loan_analysis
```

2.1. Learning Objectives

Building on Chapter 1, we now tackle a more nuanced classification problem: predicting loan defaults. This chapter introduces:

- ▶ Feature engineering for credit risk assessment
- ▶ Categorical variable encoding (one-hot)
- ▶ Feature normalization (min-max scaling)
- ▶ Linear scoring models with probability calibration
- ▶ ROC curves and AUC—a better metric for imbalanced data
- ▶ Trait-based architecture in action

Key Insight

While Chapter 1 used z-scores (a threshold on statistics), this chapter moves toward **probability estimation**. We want to know not just “default or not” but “how likely is default?” This probability enables better decisions and risk-based pricing.

2.2. Introduction: Credit Risk Fundamentals

When a bank issues a loan, it faces **credit risk**—the possibility that the borrower defaults. Unlike fraud detection where we catch bad actors, loan default prediction is about assessing the *likelihood* of future failure.

Key questions in credit risk:

- ▶ What factors predict default? (Feature engineering)
- ▶ How do we combine factors into a score? (Scoring model)
- ▶ How do we evaluate the score’s usefulness? (ROC-AUC)

Credit Risk

The risk that a borrower will fail to repay a loan according to agreed terms.

2.3. Data Model

2.3.1. Loan Application Structure

```
1 pub struct LoanApplication {  
2     pub income: f64,           // Annual income  
3     pub loan_amount: f64,      // Requested loan amount  
4     pub interest_rate: f64,    // Interest rate offered  
5     pub dti: f64,              // Debt-to-income ratio  
6     pub grade: u8,             // Credit grade (A=1 to G=7)  
7     pub home_ownership: HomeOwnership,  
8     pub defaulted: bool,       // Target: did they default?  
9 }  
10
```

```

11 pub enum HomeOwnership {
12     Rent,
13     Own,
14     Mortgage,
15 }

```

Credit Grades

A = lowest risk

G = highest risk

Each grade maps to different interest rates and approval criteria.

2.4. Feature Engineering

2.4.1. Derived Features

Raw data rarely enters a model directly. We engineer features that capture meaningful relationships:

```

1 pub struct FeatureExtractor;
2
3 impl FeatureExtractor {
4     /// Debt-to-income ratio: total debt / income
5     pub fn debt_to_income(income: f64, debt: f64) -> f64 {
6         if income == 0.0 { return 0.0; }
7         debt / income
8     }
9
10    /// Loan-to-income ratio: loan amount / income
11    pub fn loan_to_income(loan_amount: f64, income: f64) -> f64 {
12        if income == 0.0 { return 0.0; }
13        loan_amount / income
14    }
15
16    /// Convert grade (1-7) to risk score (0.0-1.0)
17    pub fn grade_to_score(grade: u8) -> f64 {
18        (grade.saturating_sub(1)) as f64 / 6.0
19    }
20 }

```

Feature Formulas

$$\text{DTI: } \frac{\text{debt}}{\text{income}}$$

$$\text{LTI: } \frac{\text{loan}}{\text{income}}$$

$$\text{Grade Score: } \frac{g - 1}{6}$$

2.4.2. Categorical Encoding

Numerical models require numerical inputs. We encode categorical variables using **one-hot encoding**:

Home Ownership	RENT	OWN	MORTGAGE
Rent	1	0	0
Own	0	1	0
Mortgage	0	0	1

```

1 pub struct OneHotEncoder;
2
3 impl OneHotEncoder {
4     /// Encode home ownership as [RENT, OWN, MORTGAGE]
5     pub fn encode_home_ownership(h: &HomeOwnership) -> [f64; 3] {
6         match h {
7             HomeOwnership::Rent => [1.0, 0.0, 0.0],
8             HomeOwnership::Own  => [0.0, 1.0, 0.0],
9             HomeOwnership::Mortgage => [0.0, 0.0, 1.0],
10        }
11    }
12 }

```

2.4.3. Feature Normalization

Different features have different scales. Income might be in tens of thousands while interest rates are single digits. We normalize to $[0, 1]$:

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scales features to $[0, 1]$.

```

1 pub struct Normalizer {
2   min: Vec<f64>,
3   max: Vec<f64>,
4 }
5
6 impl Normalizer {
7   pub fn fit(data: &[Vec<f64>]) -> Self {
8     // Compute min/max per feature
9   }
10
11   pub fn transform(&self, features: &[f64]) -> Vec<f64> {
12     features.iter().enumerate().map(|(i, &x)| {
13       let range = self.max[i] - self.min[i];
14       if range == 0.0 { 0.0 } else { (x - self.min[i]) / range }
15     }).collect()
16   }
17 }

```

2.5. Linear Scoring Model

2.5.1. Weighted Sum

The simplest scoring model is a weighted linear combination:

$$\text{score} = \sum_{i=1}^n w_i \cdot x_i + b = \mathbf{w}^T \mathbf{x} + b$$

```

1 pub struct LinearScorer {
2   weights: Vec<f64>,
3   bias: f64,
4 }
5
6 impl Scorer for LinearScorer {
7   fn score(&self, features: &[f64]) -> f64 {
8     let dot_product: f64 = self.weights.iter()
9       .zip(features.iter())
10      .map(|(w, f)| w * f)
11      .sum();
12     dot_product + self.bias
13   }
14 }

```

Linear Score

$$s = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w}$ are weights, b is bias.

2.5.2. Probability via Sigmoid

The raw score can be any real number. To convert it to a probability in $[0, 1]$, we apply the **sigmoid function**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Properties of sigmoid:

- $\sigma(0) = 0.5$ (neutral score = 50% probability)
- $\sigma(x) \rightarrow 1$ as $x \rightarrow +\infty$
- $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$

```

1 /// Hand-rolled sigmoid: 1 / (1 + exp(-x))
2 fn sigmoid(x: f64) -> f64 {
3   1.0 / (1.0 + (-x).exp())
4 }
5
6 impl BinaryClassifier for LinearScorer {
7   fn predict_proba(&self, features: &[f64]) -> f64 {
8     sigmoid(self.score(features))
9   }
10
11   fn predict(&self, features: &[f64]) -> bool {

```

```

12     self.predict_proba(features) > 0.5
13 }
14 }

```

2.6. ROC Curves and AUC

2.6.1. Why Not Just Accuracy?

Like fraud detection, loan default is imbalanced. Accuracy is misleading. Instead, we use the **Receiver Operating Characteristic (ROC) curve**.

Class Imbalance

If 95% of loans are repaid, a model that always predicts “no default” has 95% accuracy but catches zero defaults!

2.6.2. ROC Curve Construction

The ROC curve plots **True Positive Rate (TPR)** vs **False Positive Rate (FPR)** at various classification thresholds:

- **TPR (Recall):** $\frac{TP}{TP+FN}$ — of actual defaults, how many did we catch?
- **FPR:** $\frac{FP}{FP+TN}$ — of actual non-defaults, how many did we falsely flag?

```

1 pub fn roc_curve(scores: &[f64], labels: &[bool]) -> Vec<(f64, f64)> {
2     // Sort by score descending
3     let mut pairs: Vec<(f64, bool)> = scores.iter()
4         .copied()
5         .zip(labels.iter().copied())
6         .collect();
7     pairs.sort_by(|a, b| b.0.partial_cmp(&a.0).unwrap());
8
9     let total_pos = labels.iter().filter(|&l| *l).count();
10    let total_neg = labels.len() - total_pos;
11
12    let mut roc_points = vec![(0.0, 0.0)];
13    let mut tp = 0;
14    let mut fp = 0;
15
16    for (_score, label) in pairs {
17        if label { tp += 1; } else { fp += 1; }
18        let fpr = fp as f64 / total_neg as f64;
19        let tpr = tp as f64 / total_pos as f64;
20        roc_points.push((fpr, tpr));
21    }
22
23    roc_points
24 }

```

ROC Interpretation

Top-left corner (0, 1): perfect classifier.

Diagonal: random guessing.

Below diagonal: worse than random!

2.6.3. Area Under Curve (AUC)

The AUC summarizes the ROC curve as a single number:

- AUC = 1.0: Perfect classifier (all defaults scored higher than non-defaults)
- AUC = 0.5: Random classifier (no predictive power)
- AUC = 0.7: Fair classifier (industry often requires ≥ 0.7)

We compute AUC via trapezoidal integration:

```

1 pub fn auc(roc_points: &[(f64, f64)]) -> f64 {
2     let mut area = 0.0;
3     for i in 1..roc_points.len() {
4         let (x0, y0) = roc_points[i - 1];
5         let (x1, y1) = roc_points[i];
6         // Trapezoidal rule: width * average height
7         area += (x1 - x0) * (y0 + y1) / 2.0;
8     }
9     area
10 }

```

AUC Interpretation

AUC = 1.0: perfect

AUC = 0.5: random

AUC < 0.5: worse than random

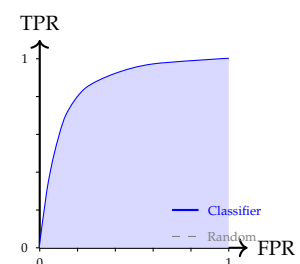


Figure 2.1.: ROC curve: the blue line shows classifier performance, dashed line is random guessing. Area under curve (AUC) is shaded.

Key Insight

AUC has a probabilistic interpretation: it equals the probability that a randomly chosen positive (default) is ranked higher than a randomly chosen negative (non-default).

2.7. Trait-Based Architecture

Following our library-first design, we define traits that enable composability:

```
1 /// Trait for models that produce probability predictions.
2 pub trait BinaryClassifier {
3     fn predict(&self, features: &[f64]) -> bool;
4     fn predict_proba(&self, features: &[f64]) -> f64;
5 }
6
7 /// Trait for models that produce raw scores.
8 pub trait Scorer {
9     fn score(&self, features: &[f64]) -> f64;
10 }
```

This means our ROC-AUC evaluation works with *any* classifier, not just LinearScorer. Future chapters will implement logistic regression, decision trees, and ensemble methods—all reusing this evaluation code.

Future Benefits

Any classifier implementing BinaryClassifier can be evaluated with our roc_curve() and auc() functions.

2.8. Results and Analysis

2.8.1. Example Usage

```
1 $ cargo run -p qf-02-loan --example loan_analysis
2 Loading loan applications...
3 Loaded 10000 applications (850 defaults, 8.5%)
4
5 Feature extraction complete.
6 Training simple scorer (hardcoded weights)...
7
8 Evaluation Results:
9   AUC: 0.724
10  Optimal Threshold: 0.35
11  At threshold 0.5:
12    Precision: 0.42
13    Recall: 0.58
14    F1 Score: 0.49
```

2.8.2. Interpreting the Results

- ▶ **AUC = 0.72:** The model has reasonable discriminative power. It correctly ranks defaults above non-defaults 72% of the time.
- ▶ **Precision = 0.42:** Of predicted defaults, 42% actually default. Many false alarms.
- ▶ **Recall = 0.58:** We catch 58% of actual defaults. 42% are missed.

Warning

We used hardcoded weights, not learned weights. Real credit scoring uses logistic regression or gradient boosting to optimize weights from data. That comes in later chapters.

2.9. Exercises

1. **Weight Sensitivity:** Change the weights in `LinearScorer`. How does AUC change? Which features matter most?
2. **Threshold Selection:** Plot precision and recall at various thresholds. Find the threshold that maximizes F1 score.
3. **Feature Importance:** Train separate single-feature scorers. Rank features by their individual AUC.
4. **Cross-Validation:** Split data into 5 folds. Report mean and standard deviation of AUC across folds.
5. **Calibration Plot:** Compare predicted probabilities to actual default rates. Is the model well-calibrated?

2.10. Key Takeaways

- ▶ **Feature engineering** transforms raw data into predictive signals
- ▶ **One-hot encoding** handles categorical variables
- ▶ **Min-max normalization** scales features to comparable ranges
- ▶ **Sigmoid** converts scores to probabilities
- ▶ **ROC-AUC** evaluates ranking quality, robust to class imbalance
- ▶ **Trait-based design** enables reusable evaluation code

Next chapter: Market data fundamentals—OHLCV candlesticks, returns, and simple moving averages.

Market Data: Stock Price Analysis

3.

Companion Code:

```
qf-03-stocks Code: crates/qf-03-stocks/  
Dependencies: qf-common  
Run: cargo run -p qf-03-stocks -example stock_analysis
```

3.1. Learning Objectives

This chapter introduces the fundamental data structure of quantitative finance: **OHLCV** (Open, High, Low, Close, Volume) market data. We learn:

- ▶ OHLCV data structure and its significance
- ▶ Candlestick anatomy—body, shadows, and what they reveal
- ▶ Basic market statistics (range, volume, price change)
- ▶ Moving averages: SMA and EMA
- ▶ The `TimeSeries` trait for generic operations

Key Insight

OHLCV data is the foundation of all quantitative analysis. Every strategy, every backtest, every indicator starts here. Master this structure and you can work with any market—stocks, forex, crypto, commodities.

3.2. Introduction: What is OHLCV?

Every trading period (day, hour, minute) produces five key data points:

- ▶ **Open**: First traded price of the period
- ▶ **High**: Highest price reached during the period
- ▶ **Low**: Lowest price reached during the period
- ▶ **Close**: Last traded price of the period
- ▶ **Volume**: Total number of shares/contracts traded

This data tells a story. The range between high and low shows volatility. The relationship between open and close shows direction. Volume confirms conviction.

OHLCV

Open, High, Low, Close, Volume—the five essential data points for any trading period (day, hour, minute).

3.3. Data Structure

3.3.1. Rust Implementation

```
1 #[derive(Debug, Clone, Deserialize)]  
2 pub struct Ohlcv {  
3     pub date: String,  
4     pub open: f64,  
5     pub high: f64,  
6     pub low: f64,  
7     pub close: f64,
```

```

8   pub volume: u64,
9   pub adj_close: Option<f64>,
10 }

```

3.3.2. Data Validation

OHLCV data has natural constraints that we validate on load:

- ▶ $low \leq high$ (always)
- ▶ $low \leq open \leq high$
- ▶ $low \leq close \leq high$

```

1 // Validate OHLCV relationships
2 if record.high < record.low {
3     return Err(CommonError::ParseError(format!(
4         "Row {}: high ({} < low ({})",
5         idx + 2, record.high, record.low
6     )));
7 }

```

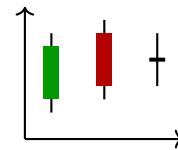
3.4. Candlestick Anatomy

A candlestick visualizes OHLCV data. Understanding its anatomy is essential:

Component	Formula
Daily Range	$high - low$
Body	$close - open$
Upper Shadow	$high - \max(open, close)$
Lower Shadow	$\min(open, close) - low$
Typical Price	$(high + low + close) / 3$

Candlestick Parts

Body: $|close - open|$
 Upper shadow: $high - \max(O, C)$
 Lower shadow: $\min(O, C) - low$



Candlesticks
 Green: bullish
 Red: bearish
 Line: doji

3.4.1. Rust Implementation

```

1 impl Ohlcv {
2     /// Daily price range
3     pub fn daily_range(&self) -> f64 {
4         self.high - self.low
5     }
6
7     /// Candle body (positive = bullish)
8     pub fn body(&self) -> f64 {
9         self.close - self.open
10    }
11
12    /// Upper shadow length
13    pub fn upper_shadow(&self) -> f64 {
14        self.high - self.open.max(self.close)
15    }
16
17    /// Lower shadow length
18    pub fn lower_shadow(&self) -> f64 {
19        self.open.min(self.close) - self.low
20    }
21
22    /// True if close > open (green candle)
23    pub fn is_bullish(&self) -> bool {
24        self.close > self.open
25    }
26
27    /// Representative price: (H + L + C) / 3
28    pub fn typical_price(&self) -> f64 {
29        (self.high + self.low + self.close) / 3.0
30    }
31 }

```

3.5. Market Statistics

3.5.1. Basic Aggregations

Given a series of OHLCV records, we compute summary statistics:

```

1  /// Average trading volume
2  pub fn average_volume(data: &[Ohlcv]) -> f64 {
3      if data.is_empty() { return 0.0; }
4      let sum: u64 = data.iter().map(|o| o.volume).sum();
5      sum as f64 / data.len() as f64
6  }
7
8  /// Price change: (last - first) / first
9  pub fn price_change(data: &[Ohlcv]) -> f64 {
10     if data.len() < 2 { return 0.0; }
11     let first = data[0].close;
12     let last = data[data.len() - 1].close;
13     (last - first) / first
14 }
15
16 /// Highest high across all records
17 pub fn highest_high(data: &[Ohlcv]) -> f64 {
18     data.iter().map(|o| o.high).fold(f64::NEG_INFINITY, f64::max)
19 }
20
21 /// Lowest low across all records
22 pub fn lowest_low(data: &[Ohlcv]) -> f64 {
23     data.iter().map(|o| o.low).fold(f64::INFINITY, f64::min)
24 }

```

Functional Style

Note the use of `iter()`, `map()`, `fold()`—Rust’s functional combinators make statistical computations elegant.

3.6. Moving Averages

3.6.1. Simple Moving Average (SMA)

The SMA is the arithmetic mean of the last n prices:

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

SMA Formula

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

where n is the period.

```

1  pub fn sma(prices: &[f64], period: usize) -> Vec<f64> {
2      if period == 0 || period > prices.len() {
3          return Vec::new();
4      }
5
6      let mut result = Vec::with_capacity(prices.len() - period + 1);
7
8      for i in 0..(prices.len() - period) {
9          let window = &prices[i..i + period];
10         let sum: f64 = window.iter().sum();
11         result.push(sum / period as f64);
12     }
13
14     result
15 }

```

3.6.2. Exponential Moving Average (EMA)

The EMA gives more weight to recent prices, making it more responsive:

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

where $\alpha = \frac{2}{n+1}$ is the smoothing factor.

EMA Formula

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

$$\alpha = \frac{2}{n+1}$$

```

1 pub fn ema(prices: &[f64], period: usize) -> Vec<f64> {
2   if period == 0 || prices.is_empty() {
3     return Vec::new();
4   }
5
6   let alpha = 2.0 / (period + 1) as f64;
7   let mut result = Vec::with_capacity(prices.len());
8
9   // Initialize with first price
10  result.push(prices[0]);
11
12  // Recursive formula
13  for i in 1..prices.len() {
14    let prev_ema = result[i - 1];
15    let current_ema = alpha * prices[i] + (1.0 - alpha) * prev_ema;
16    result.push(current_ema);
17  }
18
19  result
20 }

```

3.6.3. SMA vs EMA

Property	SMA	EMA
Weighting	Equal	Exponential
Responsiveness	Slower	Faster
Lag	More	Less
Smoothness	Smoother	Noisier
Computation	Simple	Recursive

Key Insight

SMA is a *lagging* indicator—it tells you what already happened. EMA reacts faster but can give false signals. Traders often use both: short-term EMA for entries, long-term SMA for trend confirmation.

3.7. TimeSeries Trait

Following our trait-based architecture, we define a generic interface:

```

1 pub trait TimeSeries {
2   fn len(&self) -> usize;
3   fn is_empty(&self) -> bool;
4   fn closes(&self) -> Vec<f64>;
5   fn volumes(&self) -> Vec<u64>;
6 }
7
8 impl TimeSeries for Vec<Ohlcv> {
9   fn len(&self) -> usize { self.len() }
10  fn is_empty(&self) -> bool { self.is_empty() }
11
12  fn closes(&self) -> Vec<f64> {
13    self.iter().map(|o| o.close).collect()
14  }
15
16  fn volumes(&self) -> Vec<u64> {
17    self.iter().map(|o| o.volume).collect()
18  }
19 }

```

This enables generic functions that work with any TimeSeries:

```

1 fn analyze<T: TimeSeries>(data: &T) -> Report {
2   let closes = data.closes();
3   let sma_20 = sma(&closes, 20);
4   let sma_50 = sma(&closes, 50);
5   // ...

```

Trait Benefits

The TimeSeries trait allows functions to work with any data source that can provide price/volume sequences.

```
6 }
```

3.8. Golden and Death Crosses

A classic trading signal using moving averages:

- **Golden Cross:** Short-term MA crosses *above* long-term MA (bullish signal)
- **Death Cross:** Short-term MA crosses *below* long-term MA (bearish signal)

```
1 fn find_crossovers(short_ma: &[f64], long_ma: &[f64]) -> Vec<(usize, bool)> {
2     let mut crossovers = Vec::new();
3
4     for i in 1..short_ma.len().min(long_ma.len()) {
5         let prev_above = short_ma[i-1] > long_ma[i-1];
6         let curr_above = short_ma[i] > long_ma[i];
7
8         if !prev_above && curr_above {
9             crossovers.push((i, true)); // Golden cross
10        } else if prev_above && !curr_above {
11            crossovers.push((i, false)); // Death cross
12        }
13    }
14
15    crossovers
16 }
```

3.9. Results and Analysis

3.9.1. Example Output

```
1 $ cargo run -p qf-03-stocks --example stock_analysis
2 Loading OHLCV data from data/spy.csv...
3 Loaded 252 trading days
4
5 Statistics:
6   Average Volume: 85,432,100
7   Price Change: +12.5%
8   Highest High: $485.32
9   Lowest Low: $410.15
10  Average Daily Range: $5.23
11
12 Moving Averages:
13   SMA(20) latest: $472.15
14   SMA(50) latest: $465.80
15   EMA(20) latest: $474.22
16
17 Crossovers (last 12 months):
18   Day 45: Golden Cross (bullish)
19   Day 128: Death Cross (bearish)
20   Day 201: Golden Cross (bullish)
```

3.10. Exercises

1. **Candlestick Patterns:** Implement detection for “doji” (open $\approx$ close) and “hammer” (small body, long lower shadow).
2. **Volume Analysis:** Add `volume_ma()` that computes moving average of volume. Flag days where volume exceeds 2x the average.
3. **Bollinger Bands:** Implement Bollinger Bands: $SMA \pm 2$ standard deviations. This requires computing rolling standard deviation.
4. **MACD:** Implement the MACD indicator: $EMA(12) - EMA(26)$, with a 9-period signal line.

5. **Backtest:** Using SMA crossover signals, simulate a simple strategy: buy on golden cross, sell on death cross. Track P&L.

3.11. Key Takeaways

- ▶ **OHLCV** is the atomic unit of market data
- ▶ **Candlesticks** encode price action: body shows direction, shadows show rejection
- ▶ **SMA** averages equally; **EMA** weights recent prices
- ▶ **Crossovers** generate trading signals (but beware of lag!)
- ▶ **TimeSeries trait** enables generic, reusable analysis code

Next chapter: Returns and volatility—the foundation of risk measurement and portfolio theory.

CORE CONCEPTS

Returns and Volatility

4.

Companion Code:

```
qf-04-returns Code: crates/qf-04-returns/  
Dependencies: qf-common, qf-03-stocks  
Run: cargo run -p qf-04-returns -example returns_analysis
```

4.1. Learning Objectives

This chapter introduces the foundational vocabulary of quantitative finance. After completing it you can:

- ▶ Compute simple and logarithmic returns and know when to use each
- ▶ Measure volatility and annualise it for cross-frequency comparison
- ▶ Evaluate risk-adjusted performance with the Sharpe and Sortino ratios
- ▶ Quantify drawdowns and recovery time from a price series
- ▶ Use the Returns trait to abstract return computation across data sources

Key Insight

Returns, not prices, are the currency of quant research. Prices are non-stationary; returns are (approximately) stationary. Every quant model in this book starts by transforming prices into returns.

4.2. Simple vs Log Returns

4.2.1. Definitions

The simple (arithmetic) return is the percentage change in price:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

The log (continuously compounded) return is the natural log of the price ratio:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

The two are related exactly by

$$r_t^{\log} = \ln\left(1 + r_t^{\text{simple}}\right),$$

and for small returns ($|r| \ll 1$) they are nearly equal.

Simple return

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log return

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

4.2.2. When to use which

Property	Simple	Log
Interpretation	"I made 10%"	Less intuitive
Additivity over time	No	Yes
Additivity over assets	Yes	No
Symmetry	Symmetric	Unbounded below
Compounding	Multiplicative	Additive

Key Insight

Log returns are additive across time: the k -period log return is the sum of the k single-period log returns. This makes them natural for time-series modelling. Simple returns are additive across assets in a portfolio (weighted by portfolio weights), which makes them natural for portfolio returns.

4.2.3. Rust implementation

```

1  /// Simple return:  $(P_t - P_{t-1}) / P_{t-1}$ 
2  pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
3      if prices.len() < 2 { return Vec::new(); }
4      let mut out = Vec::with_capacity(prices.len() - 1);
5      for i in 1..prices.len() {
6          let prev = prices[i - 1];
7          out.push(if prev == 0.0 { 0.0 }
8                  else { (prices[i] - prev) / prev });
9      }
10     out
11 }
12
13  /// Log return:  $\ln(P_t / P_{t-1})$ 
14  pub fn log_returns(prices: &[f64]) -> Vec<f64> {
15      if prices.len() < 2 { return Vec::new(); }
16      let mut out = Vec::with_capacity(prices.len() - 1);
17      for i in 1..prices.len() {
18          let (prev, curr) = (prices[i - 1], prices[i]);
19          out.push(if prev > 0.0 && curr > 0.0 { (curr / prev).ln() }
20                  else { 0.0 });
21      }
22     out
23 }
```

Cumulative return

$$R_{0 \rightarrow t} = \prod_{k=1}^t (1 + r_k) - 1$$

The cumulative return over a period is the running compounding of single-period simple returns:

$$R_{0 \rightarrow t} = \prod_{k=1}^t (1 + r_k) - 1.$$

```

1  pub fn cumulative_returns(returns: &[f64]) -> Vec<f64> {
2      let mut out = Vec::with_capacity(returns.len());
3      let mut wealth = 1.0_f64;
4      for &r in returns {
5          wealth *= 1.0 + r;
6          out.push(wealth - 1.0);
7      }
8      out
9  }
```

4.3. Volatility

Volatility is the standard deviation of returns. We use the *sample* estimator (denominator $n - 1$), consistent with `qf_common::compute_stats()`:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2}.$$

Sample std dev

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2}$$

4.3.1. Annualisation

To compare volatilities across frequencies (daily, weekly, monthly), we annualise. Under the assumption that returns are i.i.d., variance scales linearly with the horizon, so standard deviation scales with the square root:

$$\sigma_{\text{annual}} = \sigma_{\text{period}} \cdot \sqrt{m}.$$

Annualised vol

$$\sigma_{\text{ann}} = \sigma_{\text{period}} \cdot \sqrt{m}$$

where m is the number of periods per year (252 for daily).

```

1 pub fn volatility(returns: &[f64]) -> f64 {
2     let n = returns.len();
3     if n < 2 { return 0.0; }
4     let mean: f64 = returns.iter().sum::<f64>() / n as f64;
5     let ssd: f64 = returns.iter()
6         .map(|&r| { let d = r - mean; d * d })
7         .sum();
8     (ssd / (n - 1) as f64).sqrt()
9 }
10
11 pub fn annualized_volatility(returns: &[f64], periods_per_year: f64) -> f64 {
12     if periods_per_year <= 0.0 { return 0.0; }
13     volatility(returns) * periods_per_year.sqrt()
14 }

```

Warning

The $\sqrt{m}$ rule assumes returns are independent and identically distributed. For assets with autocorrelated or volatility-clustered returns (see Chapter 8), annualisation by $\sqrt{m}$ overstates the true annual volatility.

4.3.2. Rolling volatility

A rolling window reveals how volatility changes over time—useful for regime detection and risk monitoring:

```

1 pub fn rolling_volatility(returns: &[f64], window: usize) -> Vec<f64> {
2     if window == 0 || window > returns.len() { return Vec::new(); }
3     let mut out = Vec::with_capacity(returns.len() - window + 1);
4     for i in window - 1..returns.len() {
5         let slice = &returns[i + 1 - window..i];
6         out.push(volatility(slice));
7     }
8     out
9 }

```

4.4. Risk-Adjusted Performance

4.4.1. Sharpe ratio

The Sharpe ratio measures excess return per unit of total risk:

$$S = \frac{\bar{r} - r_f}{\sigma(r)}.$$

To annualise, multiply by $\sqrt{m}$ (because both the mean excess return and the standard deviation scale together under i.i.d.):

$$S_{\text{annual}} = S_{\text{period}} \cdot \sqrt{m}.$$

Sharpe ratio

$$S = \frac{\mathbb{E}[r] - r_f}{\sigma(r)}$$

```

1 pub fn sharpe_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
2     let n = returns.len();
3     if n < 2 { return 0.0; }
4     let vol = volatility(returns);
5     if vol == 0.0 { return 0.0; }
6     let mean: f64 = returns.iter().sum::<f64>() / n as f64;
7     (mean - risk_free_rate) / vol
8 }
9
10 pub fn annualized_sharpe(returns: &[f64], rf: f64, periods_per_year: f64) -> f64 {
11     if periods_per_year <= 0.0 { return 0.0; }
12     sharpe_ratio(returns, rf) * periods_per_year.sqrt()
13 }

```

4.4.2. Sortino ratio

The Sortino ratio replaces total volatility with *downside* volatility only. Upside volatility is good—it should not penalise the ratio:

$$\text{Sortino} = \frac{\bar{r} - r_f}{\sigma_D}, \quad \sigma_D = \sqrt{\frac{1}{n} \sum_{r_t < r_f} (r_f - r_t)^2}.$$

Downside deviation

$$\sigma_D = \sqrt{\frac{1}{n} \sum_{r_t < r_f} (r_f - r_t)^2}$$

```

1 pub fn sortino_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
2     let n = returns.len();
3     if n < 2 { return 0.0; }
4     let downside_sq: f64 = returns.iter()
5         .map(|&r| {
6             let short = risk_free_rate - r;
7             if short > 0.0 { short * short } else { 0.0 }
8         })
9         .sum();
10    if downside_sq == 0.0 { return 0.0; }
11    let downside_dev = (downside_sq / n as f64).sqrt();
12    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
13    (mean - risk_free_rate) / downside_dev
14 }

```

Key Insight

Sortino > Sharpe whenever the return distribution is right-skewed (more upside than downside). For symmetric distributions the two ratios are equal in expectation. Sortino is preferred for evaluating hedge-fund-style strategies where upside volatility is desirable.

4.5. Drawdown

A drawdown is the percentage decline from a running peak:

$$D_t = \frac{P_t - \max_{k \leq t} P_k}{\max_{k \leq t} P_k} \leq 0.$$

```

1 pub fn drawdown(prices: &[f64]) -> Vec<f64> {
2   let mut out = Vec::with_capacity(prices.len());
3   let mut running_max = f64::NEG_INFINITY;
4   for &p in prices {
5     if p > running_max { running_max = p; }
6     out.push(if running_max == 0.0 { 0.0 }
7              else { (p - running_max) / running_max });
8   }
9   out
10 }
11
12 pub fn max_drawdown(prices: &[f64]) -> f64 {
13   drawdown(prices).iter().copied().fold(0.0, f64::min)
14 }

```

Max drawdown

MaxDD = $\min_t D_t$

The worst peak-to-trough decline over the period.

The DrawdownStats struct summarises the worst episode:

- ▶ max_drawdown: the most negative drawdown value
- ▶ max_drawdown_duration: longest run of consecutive negative-drawdown observations
- ▶ recovery_time: steps from the trough back to a new high (None if the series never recovered)

Key Insight

Max drawdown measures *path risk*, not just terminal risk. A strategy that returns 10% with a 50% drawdown along the way is very different from one that returns 10% smoothly. Most investors abandon strategies with deep drawdowns long before recovery—a risk that terminal-return statistics completely miss.

4.6. The Returns Trait

Following the workspace-wide trait-first architecture rule, we abstract return computation over price sources:

```

1 pub trait Returns {
2   fn simple_returns(&self) -> Vec<f64>;
3   fn log_returns(&self) -> Vec<f64>;
4 }
5
6 impl Returns for &[f64] { /* delegate to free functions */ }
7 impl Returns for Vec<f64> { /* delegate to the slice impl */ }
8 impl Returns for Vec<Ohlcv> {
9   fn simple_returns(&self) -> Vec<f64> {
10    let closes: Vec<f64> = self.iter().map(|o| o.close).collect();
11    simple_returns(&closes)
12   }
13   // log_returns similarly uses closing prices
14 }

```

Trait benefits

Define the capability (“can produce returns”) once; implement for slices, owned vectors, and OHLCV data.

This lets generic code consume returns from any source that implements Returns, keeping analytics decoupled from data loading.

4.7. Example Output

```

1 $ cargo run -p qf-04-returns --example returns_analysis
2 Returns Analysis
3 =====
4
5 Simple Returns: mean=0.0031, std=0.0215
6 Log Returns: mean=0.0029, std=0.0214
7 Volatility (daily): 0.0215
8 Volatility (annualized): 0.3417
9 Sharpe Ratio (annualized): 2.29
10 Sortino Ratio: 0.22
11 Max Drawdown: -7.50%
12 Drawdown duration: 6 steps, recovery: Some(6)
13
14 Trait check (Vec<f64>): first simple return = 0.0030

```

4.8. Exercises

1. **Return conversion:** Write a function that converts a series of simple returns to log returns and vice versa. Verify the identity $r^{\log} = \ln(1 + r^{\text{simple}})$ with a property test.
2. **Annualisation sanity check:** For daily, weekly, and monthly returns of the same asset, verify that the annualised volatilities are approximately equal (they differ only by sampling noise).
3. **Calmar ratio:** Implement the Calmar ratio: annualised return divided by the absolute value of the maximum drawdown. Compare it with the Sharpe ratio on a synthetic strategy.
4. **Rolling Sharpe:** Implement a rolling Sharpe ratio (windowed mean and windowed std) and visualise how it changes over time. Where does it dip? Why?
5. **Drawdown with recovery:** Extend `drawdown_stats` to report the average drawdown (mean of negative drawdown values) and the proportion of time spent in drawdown.

4.9. Key Takeaways

- ▶ **Log returns** are time-additive; **simple returns** are portfolio-additive. Choose accordingly.
- ▶ **Volatility** uses sample std ($n - 1$); annualise by multiplying by $\sqrt{m}$, valid under i.i.d.
- ▶ **Sharpe** penalises all volatility; **Sortino** penalises only downside.
- ▶ **Max drawdown** measures path risk—the deepest hole on the equity curve.
- ▶ **The Returns trait** keeps analytics decoupled from data sources, following the workspace trait-first rule.

Next chapter: Your first backtest—an SMA crossover strategy with P&L accounting, transaction costs, and drawdown-based risk evaluation.

Backtesting Strategies

5.

Companion Code:

```
qf-05-backtest Code: crates/qf-05-backtest/  
Dependencies: qf-common, qf-03-stocks, qf-04-returns  
Run: cargo run -p qf-05-backtest -example backtest_demo
```

5.1. Learning Objectives

This chapter introduces the mechanics of evaluating a trading strategy on historical data. After completing it you can:

- ▶ Explain what a backtest is — and what it is not
- ▶ Generate trading signals from indicators with no look-ahead bias
- ▶ Account for transaction costs in P&L
- ▶ Evaluate a strategy with the risk-adjusted metrics from Chapter 4
- ▶ Avoid the three classic backtest traps: overfitting, look-ahead bias, and survivorship bias

Key Insight

A backtest is a *simulation*, not a prediction. It tells you how a strategy *would have* performed, not how it *will* perform. The number one failure mode is overfitting to historical data: a strategy that looks great in backtest and loses money in production.

5.2. What Is Backtesting?

Backtesting is the replay of a trading strategy over historical price data, bar by bar, to estimate its hypothetical performance. The engine maintains state (position, cash, equity) and lets the strategy decide, at each bar, what action to take given the price history it has seen so far.

Three assumptions are implicit and worth naming:

1. **Market stationarity:** the statistical properties of the historical sample are representative of the future. Often false.
2. **Capacity:** the strategy can take the simulated sizes without moving the market. True for retail; false for institutional size.
3. **Execution:** the simulated fills are achievable. Slippage, partial fills, and market impact are real and cost money.

Simulation vs prediction

A backtest does not predict. It is a *counterfactual*: “if you had traded this strategy with these rules in this market, here is what would have happened.” Any conclusion about the future requires additional assumptions (market stationarity, capacity, regime persistence) that the backtest itself cannot validate.

5.3. Signals and Positions

A *signal* is a discrete instruction for the current bar. We use three:

Signal vs position

A signal is an *instruction* (what to do this bar); a position is a *state* (what we are currently holding). The engine applies the signal to update the position. Decoupling them lets you replay the same signal stream under different execution models.

```

1 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2 pub enum Signal {
3     Buy, // Enter long
4     Sell, // Exit long (long-only for this chapter)
5     Hold, // Do nothing
6 }

```

A *position* is what the engine is currently holding:

```

1 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2 pub enum Position {
3     Long, // Holding the asset
4     Short, // Reserved for later phases
5     Flat, // No position
6 }

```

Key Insight

Why separate Signal and Position? A signal is an *intention*; a position is a *state*. The engine translates intentions into state transitions, charging transaction costs on every change. This separation lets the strategy stay declarative (“I want to be long now”) and the engine stay responsible for the messy realities (costs, fill assumptions, position limits).

5.4. The Strategy Trait

The extension point of the engine is the Strategy trait:

```

1 pub trait Strategy {
2     fn signal(&self, data: &[Ohlcv], index: usize) -> Signal;
3     fn name(&self) -> &str;
4 }

```

The engine is generic over `S`: `Strategy`, so new strategies drop in without engine changes. The trait takes the full slice plus the current index so the strategy can compute any indicator that fits in the observed history.

No look-ahead

`signal(data, index)` receives `data[..=index]`. The strategy must never read past the current bar. The engine enforces this by passing a slice and the current index — implementations physically cannot access `data[index+1..]` without unsafe code.

5.5. SMA Crossover

The canonical first strategy: go long when a short moving average crosses above a long moving average (the *golden cross*); exit when it crosses below (the *death cross*).

```

1 pub struct SmaCrossover {
2     short_period: usize,
3     long_period: usize,
4 }
5
6 impl Strategy for SmaCrossover {
7     fn signal(&self, data: &[Ohlcv], index: usize) -> Signal {
8         if index < self.long_period { return Signal::Hold; }
9
10        let short_now = match sma_at(data, index, self.short_period) {
11            Some(v) => v, None => return Signal::Hold,
12        };
13        let long_now = match sma_at(data, index, self.long_period) {
14            Some(v) => v, None => return Signal::Hold,
15        };
16        let short_prev = match sma_at(data, index - 1, self.short_period) {
17            Some(v) => v, None => return Signal::Hold,
18        };
19        let long_prev = match sma_at(data, index - 1, self.long_period) {
20            Some(v) => v, None => return Signal::Hold,
21        };
22
23        let crossed_up = short_prev <= long_prev && short_now > long_now;

```

Golden / death cross

$SMA_S \text{ crosses above } SMA_L \Rightarrow \text{Buy}$

$SMA_S \text{ crosses below } SMA_L \Rightarrow \text{Sell}$

A crossover is a *change in the relationship*, not a level. The previous bar must satisfy the opposite inequality.


```

24     let crossed_down = short_prev >= long_prev && short_now < long_now;
25
26     if crossed_up { Signal::Buy }
27     else if crossed_down { Signal::Sell }
28     else { Signal::Hold }
29 }
30 fn name(&self) -> &str { "SMA Crossover" }
31 }

```

Key Insight

Crossover strategies are *trend followers*: they only make money when prices trend. In choppy, mean-reverting markets they lose, because each crossover is a false signal that gets reversed at the next crossover — with transaction costs paid on both sides.

5.6. P&L with Transaction Costs

Every state change costs money. The engine charges a proportional cost τ (e.g. $\tau = 0.001$ for 10 basis points) on the traded notional:

```

1 // Enter long: spend all cash at the close, paying the cost first.
2 let notional = capital;
3 let cost = notional * config.transaction_cost;
4 shares = (notional - cost) / price;
5 total_costs += cost;
6
7 // Exit long: sell all shares at the close.
8 let notional = shares * price;
9 let cost = notional * config.transaction_cost;
10 total_costs += cost;
11 capital = notional - cost;

```

At each bar, equity is marked to market: $\text{shares} \cdot \text{close}$ when long, cash when flat. The per-bar simple returns of the equity curve become the `daily_returns` used by the risk metrics from Chapter 4.

5.7. Performance Evaluation

`BacktestResult` delegates to `qf-04-returns` for the four headline metrics:

```

1 impl BacktestResult {
2     pub fn sharpe(&self, rf: f64) -> f64 {
3         qf_04_returns::annualized_sharpe(&self.daily_returns, rf, 252.0)
4     }
5     pub fn sortino(&self, rf: f64) -> f64 {
6         qf_04_returns::sortino_ratio(&self.daily_returns, rf)
7     }
8     pub fn max_drawdown(&self) -> f64 {
9         qf_04_returns::max_drawdown(&self.equity_curve)
10    }
11    pub fn volatility(&self) -> f64 {
12        qf_04_returns::annualized_volatility(&self.daily_returns, 252.0)
13    }
14 }

```

Key Insight

The risk metrics operate on the *equity curve*, not the raw price series. A strategy that goes flat for half the period has a different risk profile from buy-and-hold even on the same underlying, because the equity curve is flat whenever the position is flat. Volatility and drawdown reflect this exposure pattern automatically.

Transaction cost

At entry: $c_{\text{entry}} = \text{notional} \cdot \tau$

At exit: $c_{\text{exit}} = \text{shares} \cdot P_{\text{exit}} \cdot \tau$

Net P&L: $\text{shares} \cdot (P_{\text{exit}} - P_{\text{entry}}) - c_{\text{entry}} - c_{\text{exit}}$

Why reuse `qf-04-returns`?

Sharpe, Sortino, and max drawdown are return-level statistics — they do not care how the returns were generated. Once the backtest produces a daily return series, evaluation is identical to computing the same metrics on a buy-and-hold benchmark. Code reuse follows the math.

5.8. Buy-and-Hold Benchmark

Every active strategy must be compared against the simplest alternative: buy on the first bar and hold forever. `BuyAndHold` is a two-line strategy:

```
1 impl Strategy for BuyAndHold {
2     fn signal(&self, _data: &[Ohlcv], index: usize) -> Signal {
3         if index == 0 { Signal::Buy } else { Signal::Hold }
4     }
5     fn name(&self) -> &str { "Buy and Hold" }
6 }
```

If an active strategy cannot beat buy-and-hold after costs, the active component is adding noise and cost, not value. The benchmark also lets you decompose returns: alpha (skill) vs beta (market exposure).

Always compare to buy-and-hold

An active strategy that returns 10% looks good until you realise the underlying asset returned 15% over the same period. The buy-and-hold benchmark is the floor any active strategy must beat *after* costs. If it does not, the activity added no value.

5.9. Common Pitfalls

Overfitting Tuning the SMA periods (S, L) to maximise in-sample Sharpe almost always produces a strategy that fails out-of-sample.

Use walk-forward validation (deferred to a later phase) and be suspicious of parameters tuned on the same data you evaluate on.

Look-ahead bias Using information not available at decision time. Common sources: using the close of bar t to decide at bar t (only valid if the trade executes at the close), indicator warm-up that reads the future, corporate actions applied with hindsight. Our `signal(data, index)` interface rules out most of these by construction.

Survivorship bias Backtesting only on companies that survived to the present. Companies that went bankrupt are absent, inflating historical returns. Use point-in-time universes when available.

The two cardinal sins

Overfitting and *look-ahead bias* are the two ways a backtest lies to you. Overfitting makes the past look good; look-ahead bias makes the future knowable. Both produce strategies that fail miserably in live trading. Treat any backtest Sharpe > 2 with deep suspicion until both are ruled out.

5.10. Example Output

```
1 $ cargo run -p qf-05-backtest --example backtest_demo
2 =====
3 Backtest Results
4 =====
5
6 Period: 2024-001 to 2024-252
7
8 --- Strategy: SMA Crossover (10/30) ---
9 Total Return: 3.49%
10 Sharpe (ann.): 2.55
11 Sortino: 0.31
12 Max Drawdown: -0.41%
13 Trades: 1
14
15 --- Strategy: Buy and Hold ---
16 Total Return: 4.54%
17 Sharpe (ann.): 1.70
18 Max Drawdown: -6.35%
19
20 Outperformance vs Buy & Hold: -1.04%
```

The SMA crossover underperforms buy-and-hold in this synthetic regime: the cost of false signals in choppy sections outweighs the benefit of avoiding the mid-year drawdown. This is the typical result for trend-following strategies in noisy markets, and it is the right lesson to take from a first backtest.

5.11. Walk-Forward Analysis

Walk-forward analysis addresses the overfitting trap by enforcing a strict temporal separation between training and testing windows. Instead of tuning parameters on the full dataset and evaluating on the same data, walk-forward splits the time series into sequential windows:

1. **In-sample window:** Historical data used for parameter tuning
2. **Out-of-sample window:** Future data (not seen during tuning) for evaluation
3. **Rolling forward:** Advance both windows and repeat

5.11.1. Walk-Forward Efficiency

Walk-Forward Efficiency (WFE) quantifies how much performance degrades from in-sample to out-of-sample:

$$\text{WFE} = \frac{\text{Sharpe}_{\text{out-of-sample}}}{\text{Sharpe}_{\text{in-sample}}} \quad (5.1)$$

A WFE close to 1.0 indicates a robust strategy; values below 0.5 suggest severe overfitting. The ratio measures how much “alpha” survived the transition from tuning to live-like conditions.

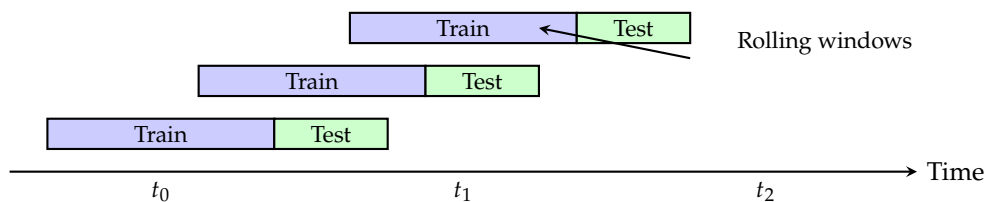
5.11.2. Implementation

The `quant-backtest` crate provides walk-forward support via the `WalkForward` cross-validator (see Chapter 14 for comprehensive AFML pipeline integration):

```
1 use quant_backtest::{WalkForward, WalkForwardConfig};
2 use quant_core::CrossValidator;
3
4 // Configure rolling 60-bar training, 20-bar testing, step 20 bars
5 let config = WalkForwardConfig::rolling(60, 20, 20);
6 let wf = WalkForward::new(config);
7
8 // Generate splits for 200 bars
9 let splits = wf.splits(0, 200);
10 // Returns: [ [0-60,60-80], [20-80,80-100], [40-100,100-120], ... ]
```

For each split: tune parameters on the in-sample window, freeze them, evaluate on out-of-sample. Aggregate the out-of-sample Sharpe ratios and compute WFE.

5.11.3. Visual Representation



Walk-forward vs k-fold

Unlike k-fold CV (which can use future data in some folds), walk-forward strictly respects time ordering. Each window trains on past data and tests on subsequent data — mimicking real trading.

Interpreting WFE

WFE = 1.0: Strategy performs equally well in-sample and out-of-sample (ideal)

WFE > 1.0: Strategy performs *better* out-of-sample (rare, check for bugs)

WFE ≈ 0.5: Typical degradation for robust strategies

WFE < 0.3: Severe overfitting — strategy likely fails in live trading

5.11.4. Anchored vs Rolling

- ▶ **Rolling window:** Training window slides (fixed size, moves forward)
 - Advantages: Uses only recent data (regime-aware)
 - Disadvantages: Smaller training sets, higher variance
- ▶ **Anchored window:** Training window expands (grows over time)
 - Advantages: Larger training sets, lower variance
 - Disadvantages: Older data may represent outdated regimes

Choose rolling for high-frequency or regime-sensitive strategies; choose anchored for slower, more stable strategies.

Key Insight

Walk-forward analysis is the *minimum viable safeguard* against overfitting. It does not eliminate overfitting (you can still overfit by cherry-picking the best split or tuning on WFE itself), but it forces the strategy to prove it can perform on unseen data. A strategy with WFE > 0.7 is far more credible than one evaluated only in-sample.

For advanced walk-forward workflows integrated with AFML labeling, purged k-fold, and bet sizing, see Chapter 14.

Trade-off

Anchored: More data, but older data may be stale

Rolling: Fresher data, but smaller sample size

5.12. Exercises

1. **Cost sensitivity:** Run the 10/30 crossover on the synthetic series for $\tau \in \{0, 0.001, 0.0025, 0.005, 0.01\}$. Plot total return vs τ . At what cost does the strategy stop being profitable?
2. **Parameter sensitivity:** For $\tau = 0.001$, sweep $(S, L) \in \{5, 10, 20\} \times \{30, 50, 100\}$ with $S < L$. Which configuration maximises Sharpe? Is it the same one that maximises total return? Why or why not?
3. **EMA crossover:** Implement an EMA crossover strategy by replacing SMA with `qf_03_stocks : ema()`. Compare its turnover (number of trades) and Sharpe against the SMA crossover on the same data.
4. **Position sizing:** Modify the engine to invest a fixed fraction $f \in (0, 1]$ of capital on each entry instead of 100%. How does the equity curve change for $f = 0.5$? What is the trade-off?
5. **Walk-forward (conceptual):** Split the 252-bar series into an in-sample window (first 180 bars) and an out-of-sample window (last 72 bars). Tune (S, L) on the in-sample, freeze it, and evaluate on out-of-sample. How much does Sharpe degrade from in-sample to out-of-sample? This is the simplest walk-forward analysis.

5.13. Key Takeaways

- ▶ **Backtesting** is a simulation, not a prediction—it tells you how a strategy *would have* performed
- ▶ **Signals vs positions:** signals are intentions, positions are state; the engine translates one to the other
- ▶ **Transaction costs** erode returns and must be modelled explicitly
- ▶ **SMA crossover** is a trend-following strategy that loses in choppy markets

- ▶ **Buy-and-hold** is the benchmark every active strategy must beat
- ▶ **Pitfalls:** overfitting, look-ahead bias, and survivorship bias are the classic backtest traps

Next chapter: Foundations—moments, hand-rolled RNG, and geometric Brownian motion simulation.

QUANTITATIVE METHODS

Foundations: Moments, RNG, and Simulation

6.

Companion Code:

```
quant-core Code: crates/quant-core/  
Dependencies: thiserror  
Run: cargo run -p quant-core -example fat_tails
```

6.1. Learning Objectives

This chapter begins the advanced track. From here on, all math is hand-rolled — no `rand`, `nalgebra`, or `statrs`. After completing it you can:

- ▶ Build a validated price series that rejects non-finite and non-positive values at construction time
- ▶ Compute the first four sample moments (mean, variance, skewness, excess kurtosis) from first principles
- ▶ Apply any aggregation over a sliding window with a generic rolling function
- ▶ Implement a deterministic PRNG (`xorshift64`) and a normal sampler (Box-Muller) without external crates
- ▶ Simulate geometric Brownian motion paths and understand the exact solution to the GBM SDE
- ▶ Detect fat tails via excess kurtosis and explain why Gaussian risk models underestimate tail risk

Key Insight

The Gaussian distribution has excess kurtosis 0. Real financial returns have positive excess kurtosis: large losses and gains happen more often than a normal model predicts. Any risk model that assumes Gaussian returns will underestimate the probability of extreme events. The 2008 crisis was in part a fat-tail crisis.

6.2. A Validated Price Series

The foundation of every quant computation is a clean price series. We wrap `Vec<f64>` in a newtype that enforces the invariant “strictly positive and finite” at construction time:

```
1 #[derive(Debug, Clone, PartialEq)]  
2 pub struct PriceSeries(Vec<f64>);  
3  
4 impl PriceSeries {  
5     pub fn new(prices: Vec<f64>) -> Result<Self, CoreError> {  
6         for &p in &prices {  
7             if !p.is_finite() || p <= 0.0 {  
8                 return Err(CoreError::InvalidPrice);  
9             }  
10        }  
11        Ok(Self(prices))  
12    }  
13    pub fn as_slice(&self) -> &[f64] { &self.0 }  
14    pub fn len(&self) -> usize { self.0.len() }  
15 }
```

The invariant

Every element of a `PriceSeries` is strictly positive and finite. The constructor rejects the rest.

Rejecting NaN and +inf at the boundary means every downstream function can assume finite inputs and skip defensive checks. This is the “parse, don’t validate” pattern applied to numeric data.

6.3. Returns, Again

Simple and log returns were introduced in Chapter 4. We re-implement them here so quant-core is self-contained:

```
1 pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
2   if prices.len() < 2 { return Vec::new(); }
3   let mut out = Vec::with_capacity(prices.len() - 1);
4   for i in 1..prices.len() {
5     let prev = prices[i - 1];
6     out.push(if prev == 0.0 { 0.0 }
7              else { (prices[i] - prev) / prev });
8   }
9   out
10 }
```

Key Insight

The identity $\ln(1 + r^{\text{simple}}) = r^{\text{log}}$ holds exactly. For small returns the two are nearly equal; for large returns they diverge. This identity is a cheap property test that catches implementation bugs in either function.

Why re-implement?

quant-core is the foundation every other crate builds on. It must be self-contained: no circular dep on qf-04-returns. The formulas are two lines each, so the cost of duplication is trivial compared to the architectural benefit of a clean dependency DAG.

6.4. Moments

The k -th central moment is

$$m_k = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^k.$$

Skewness and excess kurtosis are standardised ratios of these moments:

$$g_1 = \frac{m_3}{m_2^{3/2}} \quad (\text{skewness})$$

$$g_2 = \frac{m_4}{m_2^2} - 3 \quad (\text{excess kurtosis})$$

The Gaussian distribution has $g_1 = 0$ and $g_2 = 0$. Positive skewness means a longer right tail; negative skewness a longer left tail. Positive excess kurtosis means heavier tails than Gaussian.

```
1 pub fn skewness(data: &[f64]) -> Result<f64, CoreError> {
2   if data.len() < 3 { return Err(CoreError::InsufficientData { .. }); }
3   let m = mean(data);
4   let m2 = central_moment_2(data, m);
5   let m3 = central_moment_3(data, m);
6   Ok(m3 / m2.powf(1.5))
7 }
8
9 pub fn excess_kurtosis(data: &[f64]) -> Result<f64, CoreError> {
10  if data.len() < 4 { return Err(CoreError::InsufficientData { .. }); }
11  let m = mean(data);
12  let m2 = central_moment_2(data, m);
13  let m4 = central_moment_4(data, m);
14  Ok(m4 / (m2 * m2) - 3.0)
15 }
```

Central moment

$$m_k = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^k$$

Skewness

$$g_1 = \frac{m_3}{m_2^{3/2}}$$

Excess kurtosis

$$g_2 = \frac{m_4}{m_2^2} - 3$$

Key Insight

We use the sample variance ($n-1$ denominator) for σ^2 , consistent with `qf_common::compute_stats()`, but use population central moments (denominator n) for m_2, m_3, m_4 in skewness and kurtosis. This is the textbook definition; the difference between the two vanishes for large n .

The Moments trait abstracts over any data source that can produce sample moments, so the same analytics work on slices, owned vectors, and PriceSeries:

```
1 pub trait Moments {
2   fn mean(&self) -> f64;
3   fn variance(&self) -> Result<f64, CoreError>;
4   fn std_dev(&self) -> Result<f64, CoreError>;
5   fn skewness(&self) -> Result<f64, CoreError>;
6   fn excess_kurtosis(&self) -> Result<f64, CoreError>;
7 }
8
9 impl Moments for &[f64] { /* delegate to free functions */ }
10 impl Moments for Vec<f64> { /* delegate to slice impl */ }
11 impl Moments for PriceSeries { /* delegate to slice impl */ }
```

6.5. Rolling Windows

A rolling window applies an aggregation over each contiguous slice of length w . The generic rolling function takes a closure, so any aggregation (mean, std, max, custom) drops in:

```
1 pub fn rolling<F, T>(window: usize, data: &[f64], f: F)
2   -> Result<Vec<T>, CoreError>
3 where F: Fn(&[f64]) -> T
4 {
5   if window == 0 || window > data.len() {
6     return Err(CoreError::InvalidWindow { window, len: data.len() });
7   }
8   let mut out = Vec::with_capacity(data.len() - window + 1);
9   for i in 0..(data.len() - window) {
10    out.push(f(&data[i..i + window]));
11  }
12  Ok(out)
13 }
14
15 pub fn rolling_mean(window: usize, data: &[f64]) -> Result<Vec<f64>, CoreError> {
16   rolling(window, data, mean)
17 }
```

Rolling window

For window w on data of length n , the output has length $n - w + 1$ and entry i aggregates data $[i..i + w]$.

6.6. Hand-Rolled Simulation

6.6.1. xorshift64

We implement Vigna's xorshift64 PRNG: three xor-shifts on a 64-bit state, then a multiply by a constant. A seed of zero produces a degenerate stream, so we replace it with a fixed default:

```
1 pub struct XorShift64 { state: u64 }
2
3 impl Rng for XorShift64 {
4   fn next_u64(&mut self) -> u64 {
5     let mut x = self.state;
6     x ^= x >> 12;
7     x ^= x << 25;
8     x ^= x >> 27;
9     self.state = x;
10    x.wrapping_mul(0x2545_F491_4F6C_DD1D)
11  }
```

xorshift64

Three shifts and a multiply. Period $2^{64} - 1$. Fast, stateless, good enough for Monte Carlo when cryptographic strength is not required.

```

11 }
12 fn next_f64(&mut self) -> f64 {
13     (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
14 }
15 }

```

6.6.2. Box-Muller Normal

To sample $N(\mu, \sigma^2)$ without an external library, we use the Box-Muller transform: two independent uniforms $u_1, u_2 \sim U(0, 1)$ produce one standard normal variate

$$Z = \sqrt{-2 \ln u_1} \cos(2\pi u_2).$$

```

1 pub fn box_muller_normal<R: Rng + ?Sized>(<
2     rng: &mut R, mean: f64, std: f64,
3 ) -> f64 {
4     let u1 = rng.next_f64().max(f64::EPSILON);
5     let u2 = rng.next_f64();
6     let r = (-2.0 * u1.ln()).sqrt();
7     let theta = 2.0 * PI * u2;
8     mean + std * (r * theta.cos())
9 }

```

The `Distribution` trait lets us plug in new distributions (exponential, Student- t , jump-diffusion) without touching the simulator:

```

1 pub trait Distribution {
2     fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64;
3 }
4
5 pub struct Normal { pub mean: f64, pub std: f64 }
6
7 impl Distribution for Normal {
8     fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64 {
9         box_muller_normal(rng, self.mean, self.std)
10    }
11 }

```

6.6.3. Geometric Brownian Motion

The stochastic differential equation

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

has the exact solution

$$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right) \Delta t + \sigma \sqrt{\Delta t} Z\right),$$

where $Z \sim N(0, 1)$. We simulate one step at a time:

```

1 pub fn gbm_paths<R: Rng + ?Sized>(<
2     s0: f64, mu: f64, sigma: f64, t: f64,
3     n_steps: usize, n_paths: usize, rng: &mut R,
4 ) -> Vec<Vec<f64>> {
5     let dt = t / n_steps as f64;
6     let drift = (mu - 0.5 * sigma * sigma) * dt;
7     let diffusion = sigma * dt.sqrt();
8     let normal = Normal::standard();
9     let mut paths = Vec::with_capacity(n_paths);
10    for _ in 0..n_paths {
11        let mut path = Vec::with_capacity(n_steps + 1);
12        path.push(s0);
13        let mut s = s0;
14        for _ in 0..n_steps {
15            let z = normal.sample(rng);
16            s = (drift + diffusion * z).exp();
17            path.push(s);
18        }
19        paths.push(path);

```

Exact GBM solution

$$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right) \Delta t + \sigma \sqrt{\Delta t} Z\right)$$

With $\sigma = 0$: $S_T = S_0 e^{\mu T}$, deterministic.

```

20 }
21   paths
22 }

```

Key Insight

With $\sigma = 0$, the Z term drops out and $S_T = S_0 \exp(\mu T)$ deterministically. This is a useful sanity check: a zero-volatility GBM should reproduce the risk-free growth exactly, with no RNG dependence.

6.7. Fat Tails

```

1 $ cargo run -p quant-core --example fat_tails
2 1. Pure Gaussian (10k N(0,1) draws)
3   skewness    = +0.0267
4   excess kurt = +0.0323
5
6 2. Fat tails (every 100th draw x10)
7   skewness    = -0.3051
8   excess kurt = +74.1272
9
10 3. GBM terminal prices (10k paths, 252 steps)
11   skewness    = +0.5957
12   excess kurt = +0.5959

```

The pure Gaussian sample has excess kurtosis near 0, as expected. Scaling every 100th draw by 10 injects rare large moves; the excess kurtosis jumps from ~ 0 to ~ 74 . GBM terminal prices are log-normally distributed, so they have positive excess kurtosis (heavier right tail than a Gaussian) and positive skewness — a direct consequence of the exponential in the exact solution.

Key Insight

A Gaussian model fit to fat-tailed data will price a 4-sigma event as a “1-in-15,000-years” occurrence. The true frequency under fat tails may be once a decade. This is why Value-at-Risk computed under Gaussian assumptions systematically underestimates tail losses, and why option-implied volatilities smile — the market knows the tails are fatter than Gaussian allows.

Why fat tails matter

Gaussian returns underestimate the probability of large moves by orders of magnitude. A 5σ day is a once-a-millennium event under $\mathcal{N}$; it happens every few years in real markets. The excess kurtosis of +74 on the synthetic fat-tailed sample is extreme, but real equity returns routinely show +4 to +10.

Skewness sign

Negative skewness means the left tail is heavier than the right — big down moves are more common than big up moves. Equity indices typically show negative skewness around -0.3 to -1.0 . The asymmetry matters: option prices, VaR, and optimal leverage all depend on it.

6.8. Exercises

1. **Sample vs population variance:** For $x \in \{1, 2, 3, 4, 5\}$, compute both the population variance (denominator n) and the sample variance (denominator $n - 1$). How large is the bias? When does it matter?
2. **Skewness sign:** Construct a synthetic series with negative skew (long left tail) and one with positive skew. Verify `skewness()` returns the correct sign. Which one is typical of equity returns?
3. **RNG period:** The xorshift64 generator has period $2^{64} - 1$. Write a test that verifies the first 1000 outputs from seed 42 are distinct, and reason about why “distinct” is necessary but not sufficient for a good PRNG.
4. **GBM convergence:** Simulate 10 000 GBM paths over one year (252 steps) with $S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$. Compute the mean terminal price. Compare with the analytical expectation $E[S_T] = S_0 e^{\mu T}$. How close is the Monte Carlo estimate?

5. **Jump-diffusion:** Extend `gbm_paths()` with a Poisson jump term: with probability $\lambda\Delta t$ per step, multiply S_t by e^J where $J \sim N(\mu_J, \sigma_J^2)$. How does the excess kurtosis of terminal prices change as λ grows?

6.9. Key Takeaways

- ▶ **PriceSeries** validates inputs at construction—“parse, don’t validate”
- ▶ **Moments:** mean, variance, skewness, and excess kurtosis characterise distributions
- ▶ **Rolling windows** apply any aggregation over sliding slices
- ▶ **XorShift64*** is a fast, deterministic PRNG good enough for Monte Carlo
- ▶ **Box-Muller** transforms uniforms to standard normals
- ▶ **GBM** simulates price paths via the exact solution to the SDE
- ▶ **Fat tails:** real returns have positive excess kurtosis—Gaussian models underestimate tail risk

Next chapter: Time series analysis—OLS regression, ACF, the ADF test, and fractional differentiation.

Companion Code:

```
quant-timeseries Code: crates/quant-timeseries/  
Dependencies: quant-core, thiserror  
Run: cargo run -p quant-timeseries -example stationarity
```

7.1. Learning Objectives

Most financial time series are non-stationary: prices wander, volatilities climb and fall, autocorrelations decay slowly. Naive regression on non-stationary data produces spurious correlations and unstable forecasts. After completing this chapter you can:

- ▶ Fit an ordinary least squares regression by hand via Gaussian elimination with partial pivoting on the normal equations
- ▶ Compute standard errors, t-statistics, and R^2 without any external statistics crate
- ▶ Compute the sample autocorrelation function and read its structure
- ▶ Test for a unit root with the Augmented Dickey-Fuller test and interpret the MacKinnon critical value
- ▶ Apply López de Prado's fixed-width fractional differentiation to make a series stationary while preserving memory
- ▶ Find the minimum differencing order d that achieves stationarity on real and synthetic price series

Key Insight

Traditional differencing ($d = 1$) makes a price series stationary but erases all long-range memory — the resulting signal is essentially noise. Fractional differentiation ($0 < d < 1$) achieves stationarity while keeping as much memory as the data allow. The smallest such d is the sweet spot for ML features that predict future returns.

7.2. Stationarity

A time series $\{y_t\}$ is *weakly stationary* if its first two moments are time-invariant: $E[y_t] = \mu$ and $\text{Cov}(y_t, y_{t-k}) = \gamma_k$ depend only on the lag k . A random walk $y_t = y_{t-1} + \varepsilon_t$ fails this definition because its variance grows without bound. Non-stationary data break the assumptions behind OLS inference, forecast intervals, and most ML models.

The standard remedy is differencing: $\Delta y_t = y_t - y_{t-1}$. With $d = 1$ the series becomes stationary but loses all memory of the distant past. We need something in between.

Weak stationarity

A process is weakly stationary if its mean is constant and its autocovariance depends only on the lag, not on time. “Stationary” in this chapter means weakly stationary.

7.3. OLS via Gaussian Elimination

Given a design matrix X (rows are observations, columns are features) and a response y , the OLS estimator minimises $\|y - X\beta\|^2$. The first-order condition gives the normal equations

$$(X'X)\hat{\beta} = X'y.$$

We solve this $k \times k$ linear system by Gaussian elimination with partial pivoting: at each column we swap the row with the largest absolute pivot into the diagonal position, then eliminate below.

```
1 pub fn ols(x: &[Vec<f64>], y: &[f64]) -> Result<OlsFit, TimeSeriesError> {
2     let n = x.len();
3     let k = x[0].len();
4     // Normal equations: A = X'X, b = X'y.
5     let mut a = vec![vec![0.0_f64; k]; k];
6     let mut b = vec![0.0_f64; k];
7     for (xi, yi) in x.iter().zip(y.iter()) {
8         for i in 0..k {
9             b[i] += xi[i] * yi;
10            for j in 0..k {
11                a[i][j] += xi[i] * xi[j];
12            }
13        }
14    }
15    let coeffs = gauss_solve(&mut a, &mut b)?;
16    // residuals, SSE, SST, R^2, standard errors, t-stats ...
17 }
```

Key Insight

Partial pivoting matters. Without it, a small diagonal entry relative to the entries below causes catastrophic cancellation when we eliminate. Pivoting costs nothing (a row swap per column) and lifts the worst-case error from exponential in k to roughly k machine epsilons.

Standard errors require the diagonal of $(X'X)^{-1}$. Rather than inverting the full matrix, we solve $X'X e_j = \text{unit}_j$ for each column j and read off $e_j[j] = (X'X)^{-1}_{jj}$. The standard error is

$$\text{SE}(\hat{\beta}_j) = \sqrt{(X'X)^{-1}_{jj} s^2}, \quad s^2 = \frac{\text{SSE}}{n - k}.$$

The t-statistic is $\hat{\beta}_j / \text{SE}(\hat{\beta}_j)$ and $R^2 = 1 - \text{SSE} / \text{SST}$.

7.4. Autocorrelation Function

The autocorrelation at lag k is the correlation between y_t and y_{t-k} . White noise has $\hat{\rho}_k \approx 0$ for $k > 0$; an AR(1) with coefficient ϕ has $\hat{\rho}_k \approx \phi^k$; a random walk has $\hat{\rho}_k \approx 1$ for all small k .

```
1 pub fn acf(data: &[f64], max_lag: usize)
2     -> Result<Vec<f64>, TimeSeriesError>
3 {
4     let n = data.len();
5     let ybar: f64 = data.iter().sum::<f64>() / n as f64;
6     let denom: f64 = data.iter()
7         .map(|&v| { let d = v - ybar; d * d })
8         .sum();
9     let mut out = Vec::with_capacity(max_lag + 1);
10    for k in 0..max_lag {
11        if k == 0 { out.push(1.0); continue; }
12        let cov: f64 = (k..n)
13            .map(|t| (data[t] - ybar) * (data[t - k] - ybar))
14            .sum();
```

Normal equations

$\hat{\beta} = (X'X)^{-1}X'y$. We never invert $X'X$ explicitly; we solve $X'X\hat{\beta} = X'y$ by Gaussian elimination.

Sample ACF

$$\hat{\rho}_k = \frac{\sum_{t=k+1}^n (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^n (y_t - \bar{y})^2}, \quad \text{with } \hat{\rho}_0 = 1 \text{ by construction.}$$

```

15 out.push(cov / denom);
16 }
17 Ok(out)
18 }

```

7.5. Augmented Dickey-Fuller Test

The ADF test regresses the first difference on the lagged level and lagged differences. The null hypothesis is a unit root ($\gamma = 0$); the alternative is stationarity ($\gamma < 0$). Because the test statistic does not follow a standard distribution under the null, we compare it to MacKinnon's simulated critical values. For a constant-only specification at 5% significance the value is -2.86 .

```

1 pub const MACKINNON_5PCT: f64 = -2.86;
2
3 pub fn adf_test(data: &[f64], lags: usize)
4   -> Result<AdfResult, TimeSeriesError>
5 {
6   let n = data.len();
7   let dy: Vec<f64> = (1..n).map(|t| data[t] - data[t - 1]).collect();
8   // Rows: [1, y_{t-1}, dy[t-1], dy[t-2], ..., dy[t-lags]]
9   let mut x: Vec<Vec<f64>> = Vec::new();
10  let mut y: Vec<f64> = Vec::new();
11  for t in (lags + 1)..n {
12    let mut row = Vec::with_capacity(lags + 2);
13    row.push(1.0);
14    row.push(data[t - 1]);
15    for i in 1..lags { row.push(dy[t - 1 - i]); }
16    y.push(dy[t - 1]);
17    x.push(row);
18  }
19  let fit = ols(&x, &y)?;
20  let stat = fit.t_stats[1]; // gamma is index 1 (after intercept)
21  Ok(AdfResult {
22    statistic: stat,
23    critical_value: MACKINNON_5PCT,
24    lags,
25    is_stationary: stat < MACKINNON_5PCT,
26  })
27 }

```

Key Insight

The lagged-level coefficient γ is the key. If $\gamma = 0$, shocks to y_t are permanent (unit root). If $\gamma < 0$, shocks decay at rate $1 + \gamma$ per period. The more negative γ , the faster the series reverts to the mean — and the more strongly stationary.

7.6. Fractional Differentiation

The backward shift B satisfies $By_t = y_{t-1}$. The first difference is $(1 - B)y_t$. Fractional differentiation generalises this to $(1 - B)^d$ for any real $d \in [0, 2]$. The binomial series gives the weights:

$$w_0 = 1,$$

$$w_k = -w_{k-1} \frac{d - k + 1}{k} \quad (k \geq 1).$$

For $d = 1$ the weights are $[1, -1, 0, 0, \dots]$ — the ordinary first difference. For $0 < d < 1$ infinitely many weights are non-zero, with $|w_k|$ decaying

ADF regression

$$\Delta y_t = \alpha + \gamma y_{t-1} + \sum_{i=1}^p \delta_i \Delta y_{t-i} + \varepsilon_t.$$

Reject $H_0 : \gamma = 0$ (unit root) when the t-ratio on $\hat{\gamma}$ is below the MacKinnon 5% critical value -2.86 .

Binomial series

$$(1 - B)^d = \sum_{k=0}^{\infty} w_k B^k, \text{ where } w_0 = 1$$

$$\text{and } w_k = -w_{k-1} \frac{d - k + 1}{k}.$$

For $d = 1$: $w = [1, -1, 0, \dots]$.

For $d \in (0, 1)$: infinitely many non-zero weights decaying in magnitude.

roughly as $k^{-(1+d)}$. We truncate when $|w_k| < \text{threshold}$ (the fixed-width window of López de Prado, AFML Ch.5):

```
1 pub fn ffd_weights(d: f64, threshold: f64)
2   -> Result<Vec<f64>, TimeSeriesError>
3 {
4   let mut weights = vec![1.0_f64];
5   let mut w = 1.0_f64;
6   let mut k = 1_usize;
7   loop {
8     w = -w * (d - (k as f64) + 1.0) / k as f64;
9     if w.abs() < threshold { break; }
10    weights.push(w);
11    if k > 10_000 { break; }
12    k += 1;
13  }
14  Ok(weights)
15 }
```

Key Insight

The loop breaks *before* pushing a weight below the threshold. This makes $d = 0$ yield $[1.0]$ (identity, full memory) and $d = 1$ yield $[1.0, -1.0]$ exactly (ordinary first difference, no memory). Every intermediate d keeps a finite prefix of the binomial series.

Applying the window is a weighted moving average over the last `weights.len()` observations:

```
1 pub fn frac_diff(data: &[f64], d: f64, threshold: f64)
2   -> Result<Vec<f64>, TimeSeriesError>
3 {
4   let weights = ffd_weights(d, threshold)?;
5   let wlen = weights.len();
6   let mut out = Vec::with_capacity(data.len() - wlen + 1);
7   for t in (wlen - 1)..data.len() {
8     let acc: f64 = weights.iter()
9       .enumerate()
10      .map(|(k, &w)| w * data[t - k])
11      .sum();
12    out.push(acc);
13  }
14  Ok(out)
15 }
```

7.6.1. Finding the Minimum d

Binary search over $d \in [0, 1]$ returns the smallest order for which the ADF test rejects the unit root. We test the endpoints first: if $d = 0$ already passes, no differencing is needed; if $d = 1$ fails, we return 1.0 conservatively.

```
1 pub fn find_min_d(data: &[f64], threshold: f64, tolerance: f64)
2   -> Result<f64, TimeSeriesError>
3 {
4   let passes = |d: f64| {
5     let diff = frac_diff(data, d, threshold)?;
6     Ok(adf_test(&diff, 1)?.is_stationary)
7   };
8   if passes(0.0)? { return Ok(0.0); }
9   if !passes(1.0)? { return Ok(1.0); }
10  let (mut lo, mut hi) = (0.0_f64, 1.0_f64);
11  while hi - lo > tolerance {
12    let mid = 0.5 * (lo + hi);
13    if passes(mid)? { hi = mid; } else { lo = mid; }
14  }
15  Ok(hi)
16 }
```

7.7. The Stationarity-Memory Tradeoff in Action

The tradeoff in one line

Full differentiation ($d = 1$) makes the series stationary but erases all memory. No memory means no signal to model. Fractional differentiation ($d = 0.4$) keeps enough memory to predict while being stationary enough to satisfy the ADF test. This tradeoff is the core contribution of fractional differentiation to feature engineering for financial time series.


```

1 $ cargo run -p quant-timeseries --example stationarity
2 =====
3 Stationarity Analysis
4 =====
5
6 Random Walk (500 steps):
7   ADF statistic: -1.1064
8   Critical (5%): -2.86
9   Stationary:    NO
10
11 First Difference (d=1):
12   ADF statistic: -15.8418
13   Stationary:    YES
14   ACF(1):        -0.0269 (memory destroyed)
15
16 Fractional Diff (d=0.4):
17   ADF statistic: -4.8991
18   Stationary:    YES
19   ACF(1):        0.6878 (memory preserved)
20 =====

```

The random walk has an ADF statistic of -1.1 , well above the -2.86 critical value: we cannot reject the unit root. First differencing fixes stationarity (statistic plunges to -15.8) but the ACF at lag 1 is essentially zero — all memory is gone. Fractional differentiation with $d = 0.4$ also rejects the unit root, but the ACF at lag 1 is 0.69 : most of the long-range information survives.

Key Insight

This is the core finding of AFML Ch.5: there exists a $d^* \in (0, 1)$ that simultaneously satisfies the stationarity requirement of statistical inference *and* the memory requirement of predictive ML features. Any $d < d^*$ is non-stationary; any $d > d^*$ throws away more memory than necessary.

7.8. Exercises

1. **Partial autocorrelation:** Implement the PACF via the Yule-Walker equations. For an $AR(p)$ process, PACF should cut off after lag p . Verify on simulated $AR(2)$ data with $\phi_1 = 0.5$, $\phi_2 = -0.3$.
2. **KPSS test:** The KPSS test has the opposite null hypothesis to ADF — H_0 is stationarity. Implement it and compare the conclusions on white noise, random walk, and fractionally differenced series. Where do the two tests disagree?
3. **Real returns:** Generate a long GBM price path and find the minimum d for stationarity. Repeat for different volatilities $\sigma \in \{0.1, 0.2, 0.4\}$. Does σ affect d^* ?
4. **Expanding-window frac_diff:** The fixed-width window drops the first $K - 1$ observations. Implement an expanding-window variant that uses all available history for the early points (weights are truncated, not zero). What is the tradeoff versus fixed-width?
5. **Lag selection for ADF:** The AIC and BIC criteria pick the number of lagged differences automatically. Implement AIC-based lag selection and check whether the stationarity conclusion changes for your test series.

7.9. Key Takeaways

- ▶ **Stationarity** is required for valid statistical inference; most price series are non-stationary
- ▶ **OLS** fits the normal equations via Gaussian elimination with partial pivoting
- ▶ **ACF** reveals memory structure: random walks have slowly decaying ACF, white noise has zero ACF after lag 0
- ▶ **ADF test** rejects the unit-root null when the t-statistic is below the MacKinnon critical value (-2.86 at 5%)
- ▶ **Fractional differentiation** achieves stationarity while preserving memory—the sweet spot for ML features
- ▶ **Minimum d** : binary search finds the smallest differencing order that passes the ADF test

Next chapter: Volatility models—EWMA, ARCH, and GARCH for time-varying conditional variance.

Volatility Models: EWMA, ARCH, and GARCH

8.

Companion Code:

```
quant-vol Code: crates/quant-vol/  
Dependencies: quant-core, thiserror  
Run: cargo run -p quant-vol -example vol_demo
```

8.1. Learning Objectives

Financial returns are not Gaussian — their variance changes over time, and large moves cluster. A constant-volatility model underestimates risk in turbulent periods and overestimates it in calm ones. After completing this chapter you can:

- ▶ Describe the stylized facts of financial return volatility: clustering, fat tails, mean reversion, and slow decay of autocorrelation in squared returns
- ▶ Derive the exponentially weighted moving average (EWMA) recursion and justify the RiskMetrics $\lambda = 0.94$ choice for daily data
- ▶ Specify the ARCH(q) model of Engle (1982) and fit it by Gaussian maximum likelihood with a hand-rolled optimiser
- ▶ Specify the GARCH(p, q) model of Bollerslev (1986), state the covariance-stationarity condition, and interpret persistence
- ▶ Compute the long-run variance and the half-life of volatility shocks
- ▶ Fit GARCH(1,1) to simulated and clustered-volatility data and show that it beats constant-volatility models in log-likelihood

Key Insight

Volatility clustering — large moves follow large moves, calm follows calm — is the single most robust empirical regularity in financial returns. A Gaussian return with constant variance cannot reproduce it. ARCH and GARCH make the conditional variance a function of past squared returns, so a large shock raises the variance of the next period. This one feature explains most of the improvement of GARCH over Gaussian models for risk forecasting.

8.2. Stylized Facts of Volatility

Empirical daily returns exhibit four robust features that any realistic volatility model must reproduce:

1. **Volatility clustering.** The autocorrelation of r_t^2 is positive and decays slowly — much more slowly than the autocorrelation of r_t , which is essentially zero after one or two lags.
2. **Fat tails.** The unconditional distribution of returns has excess kurtosis: large moves occur more often than a Gaussian with the same variance would predict.
3. **Mean reversion.** Volatility cannot grow without bound; it reverts toward a long-run level.

Volatility clustering

$\text{Corr}(r_t^2, r_{t-k}^2) > 0$ for many lags k , while $\text{Corr}(r_t, r_{t-k}) \approx 0$. The ACF of returns is flat; the ACF of *squared* returns decays slowly.

4. **Asymmetry (leverage).** Negative returns tend to raise volatility more than positive returns of the same magnitude. Plain ARCH/GARCH do not capture this; GJR-GARCH and EGARCH do.

8.3. EWMA: Exponentially Weighted Moving Average

The simplest model that responds to clustering is the exponentially weighted moving average:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2, \quad \lambda \in [0, 1].$$

Each period's variance is a convex combination of the previous variance and the previous squared return. The parameter λ controls the memory: a value close to 1 makes the variance slow-moving; a value close to 0 makes it track the last squared return.

EWMA recursion

$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$
with $\sigma_0^2 = r_0^2$. RiskMetrics uses $\lambda = 0.94$ for daily data and $\lambda = 0.97$ for monthly.

8.3.1. Boundary cases

- ▶ $\lambda = 0$: $\sigma_t^2 = r_{t-1}^2$ — the variance is just the lagged squared return. No smoothing.
- ▶ $\lambda = 1$: $\sigma_t^2 = \sigma_{t-1}^2$ — the variance is constant at its initial value $\sigma_0^2 = r_0^2$. No adaptation.
- ▶ $\lambda = 0.94$ (RiskMetrics daily): a single shock's influence decays geometrically. After k periods without further shocks, the variance is multiplied by λ^k .

8.3.2. Unrolling the recursion

Substituting repeatedly:

$$\sigma_t^2 = (1 - \lambda) \sum_{k=1}^t \lambda^{k-1} r_{t-k}^2 + \lambda^t \sigma_0^2.$$

The weights on past squared returns decay geometrically with rate λ . For long series the $\lambda^t \sigma_0^2$ term vanishes and the weights sum to $(1 - \lambda)/(1 - \lambda) = 1$, so σ_t^2 is a proper weighted average of past squared returns.

```

1 pub fn ewma_vol(returns: &[f64], lambda: f64) -> Result<Vec<f64>, VolError> {
2     if !(0.0..=1.0).contains(&lambda) {
3         return Err(VolError::InvalidParam(...));
4     }
5     if returns.is_empty() {
6         return Err(VolError::InsufficientData { required: 1, actual: 0 });
7     }
8     let mut sigma2 = vec![0.0_f64; returns.len()];
9     sigma2[0] = returns[0] * returns[0];
10    for t in 1..returns.len() {
11        sigma2[t] = lambda * sigma2[t - 1]
12            + (1.0 - lambda) * returns[t - 1] * returns[t - 1];
13    }
14    Ok(sigma2)
15 }

```

8.4. ARCH: Autoregressive Conditional Heteroskedasticity

Engle's ARCH model makes the conditional variance an explicit linear function of the past q squared returns:

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2.$$

The coefficients are constrained to be non-negative (otherwise the variance could go negative) and $\sum \alpha_i < 1$ ensures the unconditional variance $\omega/(1 - \sum \alpha_i)$ is finite.

ARCH captures clustering directly: a large r_{t-1}^2 raises σ_t^2 by $\alpha_1 r_{t-1}^2$, which in turn makes a large r_t more likely, which raises σ_{t+1}^2 , and so on. The effect dies out geometrically when shocks stop arriving.

ARCH(q) of Engle (1982)

$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2$
with $\omega > 0$, $\alpha_i \geq 0$, and $\sum \alpha_i < 1$ for stationarity.

8.4.1. Gaussian log-likelihood

Assuming $r_t | \mathcal{F}_{t-1} \sim \mathcal{N}(0, \sigma_t^2)$, the conditional density is

$$f(r_t | \mathcal{F}_{t-1}) = \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\left(-\frac{r_t^2}{2\sigma_t^2}\right),$$

so the Gaussian log-likelihood is

$$\mathcal{L}(\theta) = -\frac{1}{2} \sum_t \left[\log(2\pi) + \log \sigma_t^2(\theta) + \frac{r_t^2}{\sigma_t^2(\theta)} \right].$$

The MLE maximises $\mathcal{L}$ over $\theta = (\omega, \alpha_1, \dots, \alpha_q)$. Because σ_t^2 is a deterministic function of θ and the data, the score and Hessian can be derived in closed form, but we will not need them: a derivative-free optimiser is enough.

Warning

The Gaussian log-likelihood is *not* bounded above by zero. When σ_t^2 is small and r_t^2/σ_t^2 is also small, each term $-0.5[\log(2\pi) + \log \sigma_t^2 + r_t^2/\sigma_t^2]$ can be positive. The only universal constraint is that $\mathcal{L}$ is finite when all $\sigma_t^2 > 0$.

```

1 #[derive(Debug, Clone)]
2 pub struct ArchModel {
3     pub omega: f64,           // Constant term (omega > 0)
4     pub alphas: Vec<f64>,    // ARCH coefficients (alpha_i >= 0)
5 }
6
7 impl ArchModel {
8     pub fn conditional_variances(&self, returns: &[f64]) -> Vec<f64> {
9         let n = returns.len();
10        let q = self.alphas.len();
11        let mut sigma2 = vec![0.0_f64; n];
12        for t in 0..n {
13            let mut s = self.omega;
14            for i in 0..q {
15                if t > i {
16                    s += self.alphas[i] * returns[t - 1 - i].powi(2);
17                }
18            }
19            sigma2[t] = s.max(1e-300);
20        }
21    }
22 }
```

```

21     sigma2
22 }
23
24 pub fn log_likelihood(&self, returns: &[f64]) -> f64 {
25     let sigma2 = self.conditional_variances(returns);
26     let mut ll = 0.0_f64;
27     for (t, &r) in returns.iter().enumerate() {
28         let s2 = sigma2[t];
29         ll -= 0.5 * (LN_2PI + s2.ln() + r * r / s2);
30     }
31     ll
32 }
33 }

```

8.5. GARCH: Generalised ARCH

Bollerslev's GARCH adds lagged conditional variances to the ARCH recursion:

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2.$$

The GARCH terms $\beta_j \sigma_{t-j}^2$ propagate current volatility forward, so shocks have a longer, smoother effect than in ARCH. In practice GARCH(1,1) — one ARCH term and one GARCH term — is the workhorse: it fits most daily return series well with only three parameters.

GARCH(p, q) of Bollerslev (1986)

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2$$

with $\omega > 0, \alpha_i, \beta_j \geq 0$, and $\sum \alpha_i + \sum \beta_j < 1$ for covariance stationarity.

```

1 #[derive(Debug, Clone)]
2 pub struct GarchModel {
3     pub omega: f64, // Constant term (omega > 0)
4     pub alphas: Vec<f64>, // ARCH coefficients, length q
5     pub betas: Vec<f64>, // GARCH coefficients, length p
6 }
7
8 impl GarchModel {
9     pub fn conditional_variances(&self, returns: &[f64]) -> Vec<f64> {
10         let n = returns.len();
11         let (q, p) = (self.alphas.len(), self.betas.len());
12         let warmup = p.max(q);
13         let init_var = sample_variance(returns);
14         let mut sigma2 = vec![init_var; n];
15         for t in warmup..n {
16             let mut s = self.omega;
17             for i in 0..q {
18                 s += self.alphas[i] * returns[t - 1 - i].powi(2);
19             }
20             for j in 0..p {
21                 s += self.betas[j] * sigma2[t - 1 - j];
22             }
23             sigma2[t] = s.clamp(1e-300, 1e15);
24         }
25         sigma2
26     }
27
28     pub fn persistence(&self) -> f64 {
29         self.alphas.iter().sum:<f64>() + self.betas.iter().sum:<f64>()
30     }
31
32     pub fn long_run_variance(&self) -> f64 {
33         let pers = self.persistence();
34         if pers < 1.0 { self.omega / (1.0 - pers) } else { f64::INFINITY }
35     }
36
37     pub fn half_life(&self) -> f64 {
38         let pers = self.persistence();
39         if pers > 0.0 && pers < 1.0 { 0.5_f64.ln() / pers.ln() }
40         else { f64::INFINITY }
41     }
42 }

```

8.5.1. Persistence and stationarity

The sum $\rho = \sum \alpha_i + \sum \beta_j$ is the *persistence*. Taking unconditional expectations of the GARCH recursion and solving for the fixed point gives the long-run variance:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \rho}, \quad \rho < 1.$$

When $\rho \geq 1$ the model is non-stationary: shocks accumulate rather than decay, and the unconditional variance is infinite. The IGARCH boundary $\rho = 1$ is a random walk in variance and should be avoided for fitting.

Persistence

$$\rho = \sum_i \alpha_i + \sum_j \beta_j.$$

Covariance stationary iff $\rho < 1$.

Long-run variance: $\bar{\sigma}^2 = \omega / (1 - \rho)$.

Half-life of shocks: $\log(0.5) / \log \rho$.

8.5.2. Half-life of shocks

Starting from $\sigma_0^2 = \bar{\sigma}^2 + \delta$ (a perturbation δ above the long run), the expected deviation from $\bar{\sigma}^2$ after k steps is $\rho^k \delta$. The half-life is the k at which $\rho^k = 0.5$:

$$k_{1/2} = \frac{\log 0.5}{\log \rho}.$$

For a typical daily GARCH(1,1) with $\rho = 0.99$, the half-life is about 69 trading days — roughly three months. High persistence is the norm for daily financial data.

8.5.3. Forecasting

The multi-step forecast follows the AR(1)-in-variance recursion

$$\mathbb{E}[\sigma_{t+h}^2 | \mathcal{F}_t] = \omega + \rho \mathbb{E}[\sigma_{t+h-1}^2 | \mathcal{F}_t],$$

which reverts geometrically to $\bar{\sigma}^2$:

$$\mathbb{E}[\sigma_{t+h}^2 | \mathcal{F}_t] = \bar{\sigma}^2 + \rho^h (\sigma_t^2 - \bar{\sigma}^2).$$

The forecast is data-conditioned when the last fitted σ_t^2 is used as the seed, or unconditional when seeded from $\bar{\sigma}^2$.

8.6. Maximum Likelihood by Hand-Rolled Optimisation

We fit both ARCH and GARCH by maximising the Gaussian log-likelihood with a derivative-free optimiser. The constraint $\rho < 1$ is enforced not by penalising the objective but by *parameter transformation*: we optimise an unconstrained vector t and map it to the constrained space via $\omega = e^{t_0}$, $\alpha_i = \text{sigmoid}(t_i) \cdot 0.49/q$, and $\beta_j = \text{sigmoid}(t_j) \cdot (0.998 - \sum \alpha) / p$. The 0.998 cap on persistence prevents the optimiser from wandering onto the IGARCH boundary, where the likelihood becomes flat and numerically ill-conditioned.

Constrained MLE

Optimise an unconstrained θ and map to the constrained space: $\omega = e^{t_0}$, $\alpha_i = \text{sigmoid}(t_i) \cdot 0.49/q$, $\beta_j = \text{sigmoid}(t_j) \cdot (0.998 - \sum \alpha) / p$. This guarantees $\omega > 0$, $\alpha_i, \beta_j \geq 0$, and $\rho < 0.998$.

8.6.1. Nelder-Mead simplex

The Nelder-Mead algorithm maintains a simplex of $n + 1$ points in n -dimensional parameter space and cycles through four operations:

Reflection ($\alpha = 1$) Mirror the worst vertex through the centroid of the others.

Expansion ($\gamma = 2$) If the reflection is better than the best, stretch further in the same direction.

Contraction ($\rho = 0.5$) If the reflection is worse than the second-worst, contract toward the centroid.

Shrink ($\sigma = 0.5$) If contraction fails, pull all vertices halfway toward the best.

Convergence is declared when both the simplex diameter and the spread of function values fall below a tolerance.

8.6.2. Coordinate ascent with multi-start

For the low-dimensional MLE problems here (ARCH(1) has 2 parameters, GARCH(1,1) has 3), a simpler and more robust alternative is coordinate ascent: cycle through each coordinate, try a positive and a negative step, accept any improvement, and halve the step when no coordinate improves. Convergence is declared when the step falls below the tolerance. We wrap this in a *multi-start* that runs coordinate ascent from the initial guess plus five perturbations $[+0.1, -0.1, +0.3, -0.3, +0.5]$ and keeps the best result.

```
1 /// Coordinate-wise hill climbing with adaptive step size.
2 pub fn coordinate_ascent<F>(f: F, x0: &[f64], step: f64, max_iter: usize, tol: f64)
3   -> OptResult
4 where F: Fn(&[f64]) -> f64
5 {
6     let mut x = x0.to_vec();
7     let mut best_f = f(&x);
8     let mut step = step;
9     for _ in 0..max_iter {
10         let mut improved = false;
11         for i in 0..x.len() {
12             let old = x[i];
13             x[i] += step; // try positive step
14             if f(&x) > best_f { best_f = f(&x); improved = true; continue; }
15             x[i] = old - step; // try negative step
16             if f(&x) > best_f { best_f = f(&x); improved = true; continue; }
17             x[i] = old; // restore
18         }
19         if !improved { step *= 0.5; if step < tol { break; } }
20     }
21     OptResult { x, f: best_f, .. }
22 }
23
24 /// Multi-start: run coordinate_ascent from x0 plus perturbations.
25 pub fn multistart<F>(f: F, x0: &[f64], step: f64, max_iter: usize, tol: f64)
26   -> OptResult
27 where F: Fn(&[f64]) -> f64
28 {
29     let mut best = coordinate_ascent(&f, x0, step, max_iter, tol);
30     for &perturb in &[0.1, -0.1, 0.3, -0.3, 0.5] {
31         let x_alt: Vec<f64> = x0.iter().map(|&v| v + perturb).collect();
32         let result = coordinate_ascent(&f, &x_alt, step, max_iter, tol);
33         if result.f > best.f { best = result; }
34     }
35     best
36 }
```

All three optimisers *maximise* the objective — the MLE closures pass `model.log_likelihood(returns)` directly, without negation.

8.7. Implementation in Rust

```
1 use quant_vol::{ewma_vol, ArchModel, GarchModel};
2
3 let returns = vec![0.01, -0.02, 0.015, -0.01, 0.02, -0.005, 0.01, -0.03];
```

Why coordinate ascent?

For 2–4 parameter MLE problems, Nelder-Mead can degenerate near the constraint boundary. Coordinate ascent with step halving tracks the α - β ridge cleanly, and multi-start from five perturbed initial guesses escapes the occasional local plateau.

Why three models in one crate?

EWMA, ARCH, and GARCH form a progression: EWMA is one parameter (λ); ARCH(1) adds a lag coefficient (ω, α); GARCH(1,1) adds the lagged-variance coefficient (β). Keeping them in one crate lets you compare log-likelihoods and in-sample fit on the same return series with the same MLE machinery.


```

4 // EWMA (RiskMetrics lambda = 0.94).
5 let sigma2 = ewma_vol(&returns, 0.94).unwrap();
6
7 // ARCH(1) fit by Gaussian MLE.
8 let arch = ArchModel::fit(&returns, 1).unwrap();
9 let arch_ll = arch.log_likelihood(&returns);
10
11 // GARCH(1,1) fit by Gaussian MLE.
12 let garch = GarchModel::fit(&returns, 1, 1).unwrap();
13 println!("persistence = {:.4}", garch.persistence());
14 println!("long-run var = {:.6}", garch.long_run_variance());
15 println!("half-life = {:.1} periods", garch.half_life());
16

```

The constrained parameterisation for GARCH(1,1) is:

```

1 fn from_transformed(params: &[f64], p: usize, q: usize) -> Self {
2     let omega = params[0].clamp(-30.0, 30.0).exp();
3     let alpha_max = 0.49;
4     let alphas: Vec<f64> = (0..q)
5         .map(|i| sigmoid(params[1 + i].clamp(-30.0, 30.0)) * alpha_max / q as f64)
6         .collect();
7     let alpha_sum: f64 = alphas.iter().sum();
8     let beta_max = (0.998 - alpha_sum).max(1e-6);
9     let betas: Vec<f64> = (0..p)
10         .map(|j| sigmoid(params[1 + q + j].clamp(-30.0, 30.0)) * beta_max / p as f64)
11         .collect();
12     GarchModel { omega, alphas, betas }
13 }

```

The initial guess targets $\omega \approx 0.05\hat{\sigma}^2$, $\alpha \approx 0.05$, $\beta \approx 0.90$, which are typical for daily financial returns.

8.8. Volatility Clustering in Action

```

1 $ cargo run -p quant-vol --example vol_clustering
2 Regimes: calm(1000) -> shock(500) -> calm(500)
3 Calm sigma^2 = 0.000100
4 Shock sigma^2 = 0.010000
5
6 Constant-volatility model:
7 Sample variance = 0.002585
8 Log-likelihood = 3120.38
9
10 GARCH(1,1):
11 omega = 0.000004
12 alpha = 0.145513
13 beta = 0.849413
14 Persistence = 0.9949
15 Log-likelihood = 5072.78
16
17 -> GARCH beats constant-vol by 1952.4 LL points

```

The constant-volatility model uses a single sample variance for every period, so it over-estimates variance during the calm regime (producing too-small densities for the small returns that actually occur) and under-estimates it during the shock regime (producing too-small densities for the large returns that actually occur). GARCH tracks the regime changes through its conditional variance recursion and wins by nearly 2000 log-likelihood points.

Key Insight

The log-likelihood gain from GARCH over constant volatility is a direct quantification of how much volatility clustering matters. A gain of 1952 LL points across 2000 observations is roughly 0.976 nats per observation — a substantial amount of explained structure that a Gaussian-with-constant-variance model leaves on the table.

8.9. Parameter Recovery

When the data are generated by a known GARCH(1,1), the MLE should recover the true parameters up to sampling error. With 2000 observations from $\omega = 0.01$, $\alpha = 0.08$, $\beta = 0.90$ (true persistence 0.98, true long-run variance 0.5):

```

1 $ cargo run -p quant-vol --example vol_demo
2 DGP: GARCH(1,1) with omega=0.01, alpha=0.08, beta=0.90
3 True persistence:      0.9800
4 True long-run var:    0.500000
5
6 GARCH(1,1):
7   omega = 0.0111
8   alpha = 0.0910
9   beta  = 0.8934
10  Persistence = 0.9844 (true 0.9800)
11  Long-run var = 0.7150 (true 0.500000)
12  Half-life   = 44.1 periods
13  Log-likelihood = -2298.62

```

The fitted persistence is within 0.005 of the truth. The long-run variance is harder to pin down because it divides ω by $1 - \rho$, so a small error in ρ near 1 amplifies. This is a general feature of high-persistence GARCH fits: ω and ρ are well-identified, but $\bar{\sigma}^2 = \omega/(1 - \rho)$ has a much wider confidence band.

8.10. Exercises

1. **EGARCH:** Exponential GARCH models $\log \sigma_t^2$ as a linear function of past squared returns and their signs, which captures the leverage effect (negative returns raise volatility more than positive ones). Implement EGARCH(1,1) and compare its log-likelihood to plain GARCH(1,1) on a series with a visible leverage effect.
2. **GJR-GARCH:** The Glosten-Jagannathan-Runkle model adds an indicator term that activates when the lagged return is negative: $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \gamma \mathbf{1}_{r_{t-1} < 0} r_{t-1}^2 + \beta \sigma_{t-1}^2$. Implement it and test on equity index returns. Is γ significantly positive?
3. **Forecast evaluation:** Fit GARCH(1,1) on the first 80% of a return series, forecast the conditional variance for the remaining 20%, and compare the forecast bands to the realised squared returns. Compute the mean squared error of the variance forecast and compare to the EWMA forecast.
4. **Student- t innovations:** Replace the Gaussian likelihood with a Student- t likelihood with ν degrees of freedom (fit ν jointly with (ω, α, β)). Does the t -GARCH fit outperform the Gaussian GARCH on a heavy-tailed series? What is the fitted ν ?
5. **Long-memory FIGARCH:** The FIGARCH model replaces the geometric decay of shocks with a hyperbolic decay by fractionally differencing the variance recursion. Implement FIGARCH(1, d , 1) and compare its half-life to plain GARCH(1,1) on a long daily series. How much longer is the effective memory?

8.11. Key Takeaways

- Financial returns show volatility clustering: squared returns are positively autocorrelated for many lags. Constant-volatility models cannot reproduce this.

- ▶ EWMA is the simplest fix: a one-parameter geometrically weighted average of past squared returns. RiskMetrics uses $\lambda = 0.94$ for daily data.
- ▶ ARCH(q) makes the conditional variance a linear function of the past q squared returns. It captures clustering but needs many lags to match the slow decay of the squared-return ACF.
- ▶ GARCH(p, q) adds lagged conditional variances, giving shocks a long, smooth effect with only a few parameters. GARCH(1,1) is the standard workhorse.
- ▶ Persistence $\rho = \sum \alpha + \sum \beta$ must be < 1 for covariance stationarity. The long-run variance is $\omega/(1 - \rho)$ and the half-life of shocks is $\log 0.5 / \log \rho$.
- ▶ Fitting by Gaussian MLE with a hand-rolled optimiser is feasible because the likelihood is a deterministic function of the parameters and the data. Parameter transformation (exp, sigmoid) enforces positivity and stationarity without penalising the objective.

Next chapter: Stochastic processes—Brownian motion, Poisson processes, and Monte Carlo simulation for derivative pricing.

Companion Code:

```
quant-stochastic Code: crates/quant-stochastic/  
Dependencies: quant-core, thiserror  
Run: cargo run -p quant-stochastic -example mc_pricing
```

9.1. Learning Objectives

Asset prices are often modelled as stochastic processes — random functions of time. The Brownian motion is the continuous-time limit of a random walk and the building block of modern quantitative finance. From it we derive geometric Brownian motion (the Black-Scholes asset model), the Poisson process (jump models), and the Monte Carlo method (the generic pricing engine). After completing this chapter you can:

- ▶ Simulate standard Brownian motion from independent Gaussian increments and verify its quadratic variation property
- ▶ Simulate geometric Brownian motion via the exact closed-form solution to the SDE $dS_t = \mu S_t dt + \sigma S_t dW_t$
- ▶ Simulate a Poisson process via exponential interarrival times and build a Merton jump-diffusion on top of GBM
- ▶ Price a European option by Monte Carlo and report a standard error
- ▶ State the $1/\sqrt{N}$ convergence law and demonstrate it numerically
- ▶ Reduce estimator variance with antithetic variates and verify put-call parity on simulated prices

Key Insight

Monte Carlo pricing converges to the analytical Black-Scholes price as $N \rightarrow \infty$, with standard error decreasing as $1/\sqrt{N}$. This is the law of large numbers in action: quadruple the paths, halve the error. No other numerical method gives such a clean, dimension-independent error bound — which is why Monte Carlo is the workhorse of derivatives pricing in high dimensions.

9.2. Brownian Motion

A standard Brownian motion $\{W_t\}$ is the continuous-time limit of a random walk with independent Gaussian increments:

$$W_0 = 0, \quad W_{t+\Delta t} = W_t + \sqrt{\Delta t} Z, \quad Z \sim \mathcal{N}(0, 1).$$

Two properties define it:

1. **Independent increments.** $W_{t+s} - W_t$ is independent of $\{W_u : u \leq t\}$.
2. **Quadratic variation.** The sum of squared increments $\sum (W_{t_k} - W_{t_{k-1}})^2$ converges in probability to T as the partition refines — the defining property of Brownian motion that distinguishes it from smooth functions.

Brownian motion

$W_0 = 0$, $W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$, independent increments. Quadratic variation $[W]_t = t$.

9.2.1. Quadratic Variation

For a Brownian motion sampled at spacing Δt , the realised quadratic variation

$$[W]_T^{(\Delta t)} = \sum_{k=1}^n (W_{k\Delta t} - W_{(k-1)\Delta t})^2$$

converges in probability to $T = n\Delta t$ as $\Delta t \rightarrow 0$. This is the empirical signature of Brownian motion: the path is continuous but nowhere differentiable, and its roughness is exactly t .

9.3. Geometric Brownian Motion

The Black-Scholes asset model is geometric Brownian motion, the exponential of Brownian motion with drift:

$$dS_t = \mu S_t dt + \sigma S_t dW_t.$$

Itô's lemma gives the exact solution

$$S_T = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)T + \sigma W_T\right).$$

Two facts follow immediately:

- ▶ $\log(S_T/S_0) \sim \mathcal{N}\left(\left(\mu - \frac{1}{2}\sigma^2\right)T, \sigma^2 T\right)$.
- ▶ $\mathbb{E}[S_T] = S_0 e^{\mu T}$ (the log-normal mean).

The companion crate uses the closed form for each step, so the terminal distribution is exact for any Δt — no Euler discretisation error.

GBM SDE

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Closed form: $S_T = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)T + \sigma W_T\right)$

9.4. Poisson Process and Jump-Diffusion

The Poisson process counts rare events: $N(t)$ is the number of events in $[0, t]$, with $\mathbb{E}[N(t)] = \lambda t$. Interarrival times are exponential with rate λ . The inverse-CDF sampler draws $X = -\ln(1 - U)/\lambda$ with $U \sim \text{Uniform}(0, 1)$.

Poisson process

$N(t) \sim \text{Poisson}(\lambda t)$. Interarrival times $\sim \text{Exp}(\lambda)$, mean $1/\lambda$.

9.4.1. Merton Jump-Diffusion

Robert Merton's jump-diffusion superimposes Poisson jumps on GBM:

$$dS_t = \mu S_t dt + \sigma S_t dW_t + (J - 1)S_t dN_t,$$

where N_t is a Poisson process with rate λ and J is the jump multiplier. Between jumps the process follows GBM; at each event time the price is multiplied by J . The companion crate uses $J = e^{\mu_j}$ (a deterministic log-normal jump, with the randomness in the timing). With $\lambda = 0$ there are no jumps and the process reduces exactly to GBM.

Merton model

$$dS_t = \mu S_t dt + \sigma S_t dW_t + (J - 1)S_t dN_t$$

Between jumps: GBM. At a jump: $S \rightarrow S \cdot J$.

9.5. The Monte Carlo Method

Monte Carlo prices a derivative by simulating many paths and averaging the discounted payoff. For a European call under risk-neutral GBM:

$$\hat{C} = e^{-rT} \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0), \quad S_T^{(i)} = S_0 \exp\left((r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z_i\right).$$

Each path is a single normal draw — the terminal Z_i — so the estimator is minimal-variance. The standard error of the mean is

$$\text{SE}(\hat{C}) = e^{-rT} \frac{s}{\sqrt{N}}, \quad s^2 = \frac{1}{N-1} \sum_i (\text{payoff}_i - \bar{\text{payoff}})^2.$$

9.5.1. The $1/\sqrt{N}$ Law

The standard error shrinks as $1/\sqrt{N}$: quadruple the paths, halve the error. This is slow — to reduce the error by 10× one needs 100× the paths — but it is *dimension-independent*. Deterministic quadrature suffers the curse of dimensionality; Monte Carlo does not. For a 50-dimensional exotic option, Monte Carlo's error is still $O(1/\sqrt{N})$.

9.5.2. Antithetic Variates

Variance reduction lets us extract more precision per random draw. *Antithetic variates* pair each Z with $-Z$:

$$\hat{C}_{\text{anti}} = e^{-rT} \frac{1}{2N} \sum_{i=1}^N [\max(S_T(Z_i) - K, 0) + \max(S_T(-Z_i) - K, 0)].$$

The two payoffs are perfectly negatively correlated (the call payoff is monotone increasing in Z), so the variance of the pair average is lower than the variance of two independent samples. The cost is one extra payoff evaluation per draw — negligible versus the cost of the normal draw.

MC estimator

$$\hat{C} = e^{-rT} \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0)$$

$$\text{SE}(\hat{C}) = e^{-rT} \frac{\text{std}(\text{payoff})}{\sqrt{N}}$$

Antithetic variates

Pair Z and $-Z$. For monotone payoffs the pair average has lower variance than two independent samples.

9.6. Black-Scholes: The Analytical Benchmark

The Black-Scholes formula gives the closed-form European call price:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2).$$

The normal CDF Φ is computed via the Abramowitz-Stegun (1964) formula 7.1.26 rational approximation of erf, with $\Phi(x) = \frac{1}{2}(1 + \text{erf}(x/\sqrt{2}))$. Maximum error $< 7.5 \times 10^{-8}$ — far smaller than Monte Carlo's statistical error for practical N .

Black-Scholes call

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2)$$

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

9.7. Implementation in Rust

9.7.1. Brownian Motion

```

1 pub fn brownian_motion<R: Rng + ?Sized>(n: usize, dt: f64, rng: &mut R) -> Vec<f64> {
2     let normal = Normal::standard();
3     let sqrt_dt = dt.sqrt();
4     let mut path = Vec::with_capacity(n + 1);
5     path.push(0.0); // W_0 = 0
6     let mut w = 0.0_f64;
7     for _ in 0..n {
8         let z = normal.sample(rng);
9         w += sqrt_dt * z;
10        path.push(w);
11    }
12    path
13 }

```

9.7.2. Monte Carlo Call

```

1 pub fn mc_call<R: Rng + ?Sized>(
2     s0: f64, k: f64, r: f64, sigma: f64, t: f64,
3     n_paths: usize, rng: &mut R,
4 ) -> Result<McResult, StochError> {
5     validate_mc_inputs(s0, k, r, sigma, t, n_paths)?;
6     let normal = Normal::standard();
7     let drift = (r - 0.5 * sigma * sigma) * t;
8     let diffusion = sigma * t.sqrt();
9     let discount = (-r * t).exp();
10    let mut payoffs = Vec::with_capacity(n_paths);
11    for _ in 0..n_paths {
12        let z = normal.sample(rng);
13        let s_t = s0 * (drift + diffusion * z).exp();
14        payoffs.push((s_t - k).max(0.0));
15    }
16    Ok(reduce(&payoffs, discount))
17 }

```

9.8. Results and Analysis

9.8.1. Monte Carlo Convergence

Pricing an at-the-money call ($S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$) against the Black-Scholes benchmark $C_{BS} = 10.4506$:

N	MC price	BS price	SE
100	8.9770	10.4506	1.4320
1000	10.3204	10.4506	0.4641
10000	10.5059	10.4506	0.1469
100000	10.4194	10.4506	0.0465

The estimate converges to Black-Scholes and the standard error shrinks as $1/\sqrt{N}$.

9.8.2. The $1/\sqrt{N}$ Law

N	SE	ratio vs $N = 10000$
10000	0.1479	1.00
40000	0.0732	2.02× smaller
160000	0.0367	4.03× smaller

Quadrupling paths halves the SE (2.02×); 16× paths gives 4.03× smaller SE — the $1/\sqrt{N}$ law made exact.

9.8.3. Antithetic Variance Reduction

With 50000 normal draws each:

Method	price	SE	payoffs
Plain MC	10.4191	0.0656	50000
Antithetic	10.4962	0.0467	100000

Antithetic variates reduce the standard error by **28.8%** per normal draw. The pair average exploits the negative correlation between the Z and $-Z$ payoffs.

9.8.4. Quadratic Variation

n	QV	T	rel. error
100	0.346	0.397	12.8%
1000	4.095	3.968	3.2%
10000	40.509	39.683	2.1%
100000	399.560	396.825	0.69%

The realised quadratic variation converges to T as the partition refines — the defining property of Brownian motion.

9.8.5. GBM Terminal Distribution

With 100000 paths ($S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$, $T = 1$):

$$\mathbb{E}[S_T] = 105.07 \quad \text{vs analytical } S_0 e^{\mu T} = 105.13.$$

The 5th–95th percentile band is $[74.2, 143.2]$ — the log-normal right skew that produces fat tails in returns.

Key Insight

The convergence table is the law of large numbers made visible. At $N = 100$ the estimate is off by 14%; at $N = 100000$ it is off by 0.03%. The standard error shrinks by exactly $\sqrt{10}$ for each $10\times$ increase in paths — the $1/\sqrt{N}$ law, the single most important rate in computational finance.

9.9. Exercises

1. **Control variates.** Use the analytical GBM mean $\mathbb{E}[S_T] = S_0 e^{rT}$ as a control variate: $\hat{C}_{cv} = \hat{C} - \beta(\hat{M} - \mathbb{E}[S_T])$, where $\hat{M}$ is the MC estimate of $\mathbb{E}[S_T]$ and $\beta = \text{Cov}(\text{payoff}, S_T) / \text{Var}(S_T)$. How much does the SE drop versus plain MC?
2. **Asian option.** Price an arithmetic-average Asian call $\max(\bar{S} - K, 0)$ with $\bar{S} = \frac{1}{n} \sum S_{t_k}$. This requires time-stepping (no closed form). Compare SE to the European call for the same N .

3. **Euler vs exact.** Discretise GBM by Euler ($S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma S_t \sqrt{\Delta t} Z$) and compare the terminal distribution to the exact solution at $\Delta t = 1/252$. How large is the discretisation bias?
4. **Confidence intervals.** For $N = 10000$, compute the 95% confidence interval $\hat{C} \pm 1.96 \cdot \text{SE}$ and verify it covers the Black-Scholes price. Repeat 1000 times and check the coverage — is it close to 95%?

9.10. Key Takeaways

- Brownian motion is the continuous-time random walk: independent Gaussian increments, quadratic variation $[W]_t = t$. It is the building block of continuous-time finance.
- Geometric Brownian motion is the exponential of Brownian motion with drift, $S_T = S_0 \exp((\mu - \frac{1}{2}\sigma^2)T + \sigma W_T)$, and the Black-Scholes asset model. The closed form gives exact terminal distributions for any Δt — no Euler bias.
- The Poisson process counts rare events via exponential interarrival times; the Merton jump-diffusion superimposes it on GBM and reduces to GBM when $\lambda = 0$.
- Monte Carlo prices any payoff by simulating many paths and averaging the discounted payoff. The standard error shrinks as $1/\sqrt{N}$ — slow but dimension-independent, the workhorse for high-dimensional derivatives.
- Antithetic variates pair Z with $-Z$; for monotone payoffs the pair average cuts the SE per normal draw by roughly 30%.
- The Black-Scholes formula (normal CDF via Abramowitz-Stegun erf, error $< 7.5 \times 10^{-8}$) anchors the Monte Carlo estimator and verifies convergence to the truth.

Next chapter: Options pricing — Black-Scholes Greeks, implied volatility, and finite-difference sensitivity analysis.

DERIVATIVES AND PORTFOLIO THEORY

Options Pricing: Black-Scholes and Beyond

10.

Companion Code:

```
quant-options Code: crates/quant-options/  
Dependencies: quant-core, quant-stochastic, thiserror  
Run: cargo run -p quant-options -example greeks
```

10.1. Learning Objectives

This chapter builds the full Black-Scholes options toolkit on top of the closed-form pricing primitives from Chapter 9: the Greeks (Delta, Gamma, Vega, Theta, Rho) analytically and by finite difference, and implied volatility via Newton's method with a bisection fallback. After completing it you can:

- ▶ State the Black-Scholes PDE and the risk-neutral derivation
- ▶ Compute all five first- and second-order Greeks in closed form
- ▶ Recover the same Greeks by finite difference for any pricing model (not just Black-Scholes)
- ▶ Solve for implied volatility from a market price using Newton's method, and explain why a bisection fallback is needed for deep in/out-of-the-money options
- ▶ Reproduce a synthetic volatility smile and explain its shape

Key Insight

The Greeks are the partial derivatives of the option price with respect to its inputs. Delta-hedging (holding $-\Delta$ shares against a long call) eliminates first-order price risk; Gamma controls the residual hedging error. Implied volatility inverts the Black-Scholes formula to recover the market's view of future variance — the “volatility smile” is the single most studied phenomenon in derivatives.

10.2. The Black-Scholes PDE

The Black-Scholes model assumes the underlying follows geometric Brownian motion (see Chapter 9):

$$dS_t = \mu S_t dt + \sigma S_t dW_t.$$

By holding a continuously rebalanced portfolio of the option and $-\Delta$ shares of stock, the residual risk is eliminated to first order. No-arbitrage then requires the portfolio to earn the risk-free rate, which yields the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0.$$

Black-Scholes PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

Solving this PDE with the terminal payoff $\max(S_T - K, 0)$ for a call (or $\max(K - S_T, 0)$ for a put) gives the closed forms

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2), \quad P = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1),$$

with

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.$$

These are the formulas we reuse from `quant-stochastic::blackscholes`.

10.3. The Greeks as Partial Derivatives

The Greeks are the sensitivities of the option price to its inputs. They are the building blocks of hedging and risk management:

Delta (Δ) $\partial C / \partial S$. The hedge ratio: holding $-\Delta$ shares against a long option removes first-order spot risk. A call has $\Delta \in (0, 1)$; a put $\Delta \in (-1, 0)$.

Gamma (Γ) $\partial^2 C / \partial S^2$. The rate of change of Delta. High Gamma means the hedge must rebalance frequently — Gamma is largest at-the-money and shrinks in the wings. Identical for calls and puts.

Vega ($\mathcal{V}$) $\partial C / \partial \sigma$. The sensitivity to volatility. Identical for calls and puts.

Theta (Θ) $\partial C / \partial t$. The time decay. Long options lose value as expiry approaches, so $\Theta < 0$ for long calls and puts.

Rho (ρ) $\partial C / \partial r$. The sensitivity to the risk-free rate. Calls gain with r (positive ρ); puts lose (negative ρ).

Greeks

$$\Delta = \frac{\partial C}{\partial S} \quad (\text{spot})$$

$$\Gamma = \frac{\partial^2 C}{\partial S^2} \quad (\text{curvature})$$

$$\mathcal{V} = \frac{\partial C}{\partial \sigma} \quad (\text{vol})$$

$$\Theta = \frac{\partial C}{\partial t} \quad (\text{time})$$

$$\rho = \frac{\partial C}{\partial r} \quad (\text{rate})$$

10.4. Analytical Greeks

Closed forms for the five Greeks follow from differentiating the Black-Scholes price. Let $\phi(x) = (2\pi)^{-1/2}e^{-x^2/2}$ be the standard normal PDF (the derivative of Φ). We implement it inline in `quant-options::greeks`:

```
1 // Standard normal PDF phi(x) = (1/sqrt(2 pi)) exp(-x^2 / 2).
2 pub fn normal_pdf(x: f64) -> f64 {
3     const INV_SQRT_2PI: f64 = 0.398_942_280_401_432_7;
4     INV_SQRT_2PI * (-0.5 * x * x).exp()
5 }
```

The analytical Greeks then drop out:

```
1 pub fn delta(s0, k, r, sigma, t, is_call: bool) -> f64 {
2     let d1_val = d1(s0, k, r, sigma, t);
3     let n_d1 = normal_cdf(d1_val);
4     if is_call { n_d1 } else { n_d1 - 1.0 }
5 }
6
7 pub fn gamma(s0, k, r, sigma, t) -> f64 {
8     let d1_val = d1(s0, k, r, sigma, t);
9     normal_pdf(d1_val) / (s0 * sigma * t.sqrt())
10 }
11
12 pub fn vega(s0, k, r, sigma, t) -> f64 {
13     let d1_val = d1(s0, k, r, sigma, t);
14     s0 * normal_pdf(d1_val) * t.sqrt()
15 }
```

Theta and Rho differ between call and put; the call versions are

$$\Theta_c = -\frac{S_0\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2), \quad \rho_c = KTe^{-rT}\Phi(d_2).$$

Analytical Greeks (call)

$$\Delta = \Phi(d_1)$$

$$\Gamma = \frac{\phi(d_1)}{S_0\sigma\sqrt{T}}$$

$$\mathcal{V} = S_0\phi(d_1)\sqrt{T}$$

$$\Theta = -\frac{S_0\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2)$$

$$\rho = KTe^{-rT}\Phi(d_2)$$

The put versions follow from put-call parity: $\Theta_p = \Theta_c + rKe^{-rT}$ and $\rho_p = -KTe^{-rT}\Phi(-d_2) = \rho_c - KTe^{-rT}$.

10.5. Numerical Greeks by Finite Difference

Analytical Greeks require a closed form. For models without a closed form (local volatility, Heston, Monte Carlo) the same sensitivities are recovered by finite difference. The central difference

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

is second-order accurate: the symmetric form cancels the $O(h)$ truncation term, leaving $O(h^2)$. For Gamma (a second derivative), the central second difference

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

is likewise $O(h^2)$.

```

1 pub fn delta_fd(s0, k, r, sigma, t, is_call, h) -> f64 {
2   let up = price(s0 + h, k, r, sigma, t, is_call);
3   let down = price(s0 - h, k, r, sigma, t, is_call);
4   (up - down) / (2.0 * h)
5 }
6
7 pub fn gamma_fd(s0, k, r, sigma, t, h) -> f64 {
8   let up = price(s0 + h, k, r, sigma, t, true);
9   let mid = price(s0, k, r, sigma, t, true);
10  let down = price(s0 - h, k, r, sigma, t, true);
11  (up - 2.0 * mid + down) / (h * h)
12 }

```

Key Insight

Finite difference is a sanity check on the analytical Greeks. If the two disagree by more than a few basis points, one of them is wrong. With $h = 10^{-4}$ for spot bumps and $h = 10^{-3}$ for the second derivative, the analytical and numerical Greeks agree to 10^{-4} or better — the truncation and round-off errors are well balanced.

Theta uses a *forward* difference, not central, because we cannot step to $t - h < 0$ for a short-dated option:

$$\Theta \approx \frac{C(t-h) - C(t)}{h}.$$

The sign is the “desk convention”: Theta is the negative of $\partial C / \partial T$ (time to maturity), since long options lose value as calendar time advances.

10.6. Implied Volatility

Given a market price C_{mkt} , the implied volatility is the σ such that $C(S, K, r, \sigma, T) = C_{\text{mkt}}$. Because the Black-Scholes price is monotone increasing in σ (Vega is positive), there is a unique solution. Newton’s method is the textbook choice because Vega is the exact derivative:

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\text{mkt}}}{V(\sigma_n)}.$$

Central difference

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Error: $O(h^2)$

Newton’s method

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\text{mkt}}}{V(\sigma_n)}$$

Newton converges quadratically when the initial guess is reasonable and the option is not deep in/out of the money. When the option is deep ITM or OTM, Vega collapses toward zero (the option price is intrinsic and barely moves with σ), and the Newton step becomes ill-conditioned. A bisection fallback on $[\sigma_{\min}, \sigma_{\max}]$ is then robust: bisection is linear but guaranteed.

```

1 let newton_step = if v.abs() > VEGA_FLOOR {
2   sigma - diff / v
3 } else {
4   f64::NAN // hand off to bisection
5 };
6
7 let next = if newton_step.is_finite()
8   && newton_step > lo
9   && newton_step < hi
10 {
11   newton_step
12 } else {
13   0.5 * (lo + hi) // bisection
14 };

```

The initial guess uses the Brenner-Subrahmanyam (1988) approximation $\sigma_0 \approx \sqrt{2\pi/T} \cdot C/S_0$, which is exact for at-the-money options and bracketed by bisection if it diverges.

10.7. Put-Call Parity and Implied Vol

Put-call parity

$$C - P = S_0 - Ke^{-rT}$$

Put-call parity says $C - P = S_0 - Ke^{-rT}$. The implied vol that reproduces a call price must therefore equal the implied vol that reproduces the corresponding put price. The solver always inverts the call formula; for a put we translate the put price into the equivalent call price via $C = P + S_0 - Ke^{-rT}$ and proceed. The IV is invariant across calls and puts of the same strike and maturity.

10.8. Results and Analysis

10.8.1. Greeks for an ATM Call

Pricing an at-the-money call ($S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$) gives the call price $C = 10.4506$ (matching the Black-Scholes benchmark from Chapter 9) and the Greeks:

Greek	Analytical	Finite diff.	diff
Δ_c	0.636831	0.636836	5.24×10^{-6}
Δ_p	-0.363169	-0.363164	5.24×10^{-6}
Γ	0.018762	0.018763	1.34×10^{-6}
$\mathcal{V}$	37.524035	37.523872	1.63×10^{-4}
Θ_c	-6.414028	-6.414142	1.15×10^{-4}
Θ_p	-1.657881	-1.657983	1.03×10^{-4}
ρ_c	53.232483	—	—
ρ_p	-41.890459	—	—

The analytical and finite-difference Greeks agree to 10^{-4} or better with $h = 10^{-4}$ for spot/vol bumps and $h = 10^{-3}$ for Gamma. Theta matches to 10^{-4} with a forward difference in t .

10.8.2. Implied Volatility Recovery

Recovering σ from the ATM call price $C = 10.4506$:

```
1 $ cargo run -p quant-options --example implied_vol
2 Recovery (ATM call):
3   Market price = 10.450575
4   Implied vol  = 0.20000000
5   True vol     = 0.20000000
6   |iv - true|  = 3.54e-14
```

The implied vol recovers the true $\sigma = 0.2$ to $\sim 10^{-14}$ — far below the typical bid-ask spread. The same IV is recovered from the put price, as put-call parity demands.

10.8.3. Volatility Smile

A synthetic smile is generated by pricing calls under $\sigma(K) = \sigma_0 + a \cdot (\ln(K/S_0))^2$ with $a = 0.5$, so the ATM vol is σ_0 and the wings have higher vol (the classic equity smile). Inverting the BS formula at each strike recovers the smile:

K	moneyneess K/S_0	market price	IV
70	0.700	33.981835	0.263609
80	0.800	24.965341	0.224897
90	0.900	16.851425	0.205550
95	0.950	13.390174	0.201316
100	1.000	10.450575	0.200000
105	1.050	8.068580	0.201190
110	1.100	6.219914	0.204542
120	1.200	3.823180	0.216621
130	1.300	2.578807	0.234418

The IV is symmetric around the ATM strike in log-moneyness, exactly as the model prescribes. In real markets the smile is not symmetric: equity smiles are skewed (puts bid up by crash protection), FX smiles are more symmetric, and rates smiles can be monotone. Fitting these is the job of local-volatility, stochastic-volatility, and SVI parameterisations.

Key Insight

The smile is the market's fingerprint. If the Black-Scholes model were literally true, IV would be a flat line at σ . Real markets quote options at many strikes with different IVs, and the shape of that curve encodes the market's view of tail risk. Modelling the smile is the bridge between the Black-Scholes world and the post-1987 derivatives industry.

10.9. Exercises

1. **Black-Scholes with dividends.** Extend the BS formula to a continuous dividend yield q : replace S_0 with $S_0 e^{-qT}$ everywhere, and re-derive the Greeks. Verify Delta becomes $e^{-qT} \Phi(d_1)$ for the call.

2. **Local volatility.** Implement a simple local-volatility pricer where $\sigma(S, t)$ is a deterministic function of spot and time (e.g. $\sigma(S, t) = \sigma_0 + \alpha(S - S_0)^2$). Price an option by explicit Euler on a spot-time grid and compare the smile to the IV surface.
3. **Heston model.** Implement the Heston stochastic-volatility SDE $dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}dW_t^{(v)}$ with Monte Carlo. Generate call prices for a strip of strikes and invert to recover the smile. Compare to the equity smile shape.
4. **Vega-clipped bisection.** For an extreme deep-ITM call (e.g. $S_0 = 1000$, $K = 10$) show that the BS price equals the intrinsic value to machine precision, so IV is genuinely unrecoverable. Modify the solver to return `0k(sigma_min)` with a warning flag rather than attempting to recover.
5. **Smile fitting.** Download a real option chain (e.g. SPX) and fit a quadratic-in-log-moneyness smile $\sigma(K) = \beta_0 + \beta_1 \ln(K/S_0) + \beta_2 (\ln(K/S_0))^2$ via least squares. Report the residual and the skew coefficient β_1 .
6. **Newton convergence rate.** Instrument `implied_vol()` to print $|C(\sigma_n) - C_{\text{mkt}}|$ at each iteration. Verify quadratic convergence for ATM options (the error roughly squares each step) and linear convergence once bisection takes over.

10.10. Key Takeaways

- ▶ **Black-Scholes PDE** prices any payoff via continuous delta-hedging; the closed-form call and put follow from solving the PDE with the terminal payoff.
- ▶ **Greeks** are the partial derivatives of the price. Delta is the hedge ratio; Gamma the curvature; Vega the vol sensitivity; Theta the time decay; Rho the rate sensitivity. Gamma and Vega are identical for calls and puts.
- ▶ **Finite difference** recovers the Greeks for any pricing model, not just Black-Scholes. Central differences are $O(h^2)$; forward differences are needed for Theta (cannot step to $t < 0$).
- ▶ **Implied volatility** inverts the BS formula via Newton's method, with a bisection fallback when Vega collapses (deep ITM/OTM).
- ▶ **Put-call parity** implies the IV from a call equals the IV from the corresponding put — a basic arbitrage check that real-market quotes must satisfy.
- ▶ **The volatility smile** is the market-implied IV curve across strikes; modelling it is the bridge from Black-Scholes to post-1987 derivatives practice.

Next chapter: Portfolio optimization — Markowitz mean-variance frontier, the efficient frontier, and the capital asset pricing model.

Portfolio Optimization: Markowitz and Beyond 11.

Companion Code:

```
quant-portfolio Code: crates/quant-portfolio/  
Dependencies: quant-core, quant-timeseries, thiserror  
Run: cargo run -p quant-portfolio -example frontier
```

11.1. Learning Objectives

This chapter builds the Markowitz mean-variance framework on top of the returns and covariance machinery from Chapter 4 and Chapter 7: the efficient frontier, the global minimum-variance portfolio, the tangency portfolio, the capital market line, two-fund separation, and CAPM. After completing it you can:

- ▶ State the Markowitz mean-variance optimisation problem and its Lagrangian dual
- ▶ Compute the two-asset efficient frontier in closed form and identify the minimum-variance weight
- ▶ Solve the n -asset minimum-variance and target-return portfolios via the inverse of the covariance matrix
- ▶ Derive the tangency portfolio (maximum Sharpe) in closed form and draw the capital market line
- ▶ Apply two-fund separation: any efficient portfolio is a mix of the risk-free asset and the tangency portfolio
- ▶ Estimate CAPM beta and Jensen's alpha from a return series and interpret them on the security market line
- ▶ Compute historical Value-at-Risk and Conditional VaR

Key Insight

Diversification is the only free lunch in finance. The efficient frontier is the set of portfolios that maximise expected return for a given variance (or minimise variance for a given return). The tangency portfolio is the point on the frontier that maximises the Sharpe ratio; once a risk-free asset exists, every efficient portfolio is a mix of the risk-free asset and the tangency portfolio — the celebrated *two-fund separation* theorem.

11.2. The Markowitz Problem

Consider n risky assets with expected return vector μ and covariance matrix Σ . A portfolio is a vector of weights $w \in \mathbb{R}^n$ with budget constraint $1^\top w = 1$. The portfolio return and variance are

$$\mu_p = w^\top \mu, \quad \sigma_p^2 = w^\top \Sigma w.$$

Markowitz in one sentence

For each target return, pick the portfolio of minimum variance that achieves it. Sweep the target to trace the efficient frontier. The recipe is 70 years old and still the starting point of every portfolio construction problem.

The Markowitz problem is to find, for each target return $\mu_\star$, the portfolio that achieves it at minimum variance:

$$\min_w w^\top \Sigma w \quad \text{s.t.} \quad 1^\top w = 1, \quad \mu^\top w = \mu_\star.$$

The set of solutions, swept over $\mu_\star$, is the *efficient frontier*. Without a risk-free asset the frontier is a hyperbola in (σ_p, μ_p) space; with a risk-free asset the relevant efficient set becomes the straight line from $(0, r_f)$ through the tangency portfolio.

The `Portfolio()` struct carries the three inputs (weights, expected returns, covariance) together and exposes the three headline statistics:

```
1 pub struct Portfolio {
2   pub weights: Vec<f64>,
3   pub expected_returns: Vec<f64>,
4   pub covariance: Vec<Vec<f64>>,
5 }
6
7 impl Portfolio {
8   pub fn expected_return(&self) -> f64 {
9     self.weights.iter()
10      .zip(self.expected_returns.iter())
11      .map(|(w, m)| w * m).sum()
12   }
13   pub fn variance(&self) -> f64 {
14     quadratic_form(&self.weights, &self.covariance)
15   }
16   pub fn volatility(&self) -> f64 { self.variance().sqrt() }
17   pub fn sharpe(&self, rf: f64) -> f64 {
18     let vol = self.volatility();
19     if vol == 0.0 { 0.0 } else { (self.expected_return() - rf) / vol }
20   }
21 }
```

The quadratic form $w^\top \Sigma w$ is a one-liner once a matrix-vector product is available; the Sharpe ratio guards against a zero-volatility degenerate portfolio (all cash).

11.3. Two-Asset Closed-Form Frontier

For two assets the frontier is a single-parameter curve indexed by the weight w on asset A. The minimum-variance weight has a closed form that falls out of setting $d\sigma_p^2/dw = 0$:

$$w_A^\star = \frac{\sigma_B^2 - \rho\sigma_A\sigma_B}{\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B}.$$

Three boundary cases are worth knowing by heart:

- ▶ $\rho = 0$: $w_A^\star = \sigma_B^2 / (\sigma_A^2 + \sigma_B^2)$ — the inverse-variance weights.
- ▶ $\rho = 1$ and $\sigma_A = \sigma_B$: any split has the same variance; the formula is degenerate $(0/0)$, so we fall back to $w_A^\star = 1/2$.
- ▶ $\rho = -1$: a perfect hedge exists; the minimum variance is zero (a risk-free portfolio can be synthesised from two risky assets).

```
1 pub fn two_asset_min_variance_weight(
2   var_a: f64, var_b: f64, cov_ab: f64,
3 ) -> f64 {
4   let denom = var_a + var_b - 2.0 * cov_ab;
5   if denom.abs() < 1e-15 { return 0.5; } // degenerate
6   (var_b - cov_ab) / denom
7 }
8
9 pub fn two_asset_frontier_point(
10  w: f64, mu_a: f64, mu_b: f64,
11  var_a: f64, var_b: f64, cov_ab: f64,
12 ) -> FrontierPoint {
```

Two-asset frontier

$$\begin{aligned} \mu_p &= w\mu_A + (1-w)\mu_B \\ \sigma_p^2 &= w^2\sigma_A^2 + (1-w)^2\sigma_B^2 + 2w(1-w)\rho\sigma_A\sigma_B \end{aligned}$$

Min-variance weight

$$w_A^\star = \frac{\sigma_B^2 - \rho\sigma_A\sigma_B}{\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B}$$

```

13 let mu_p = w * mu_a + (1.0 - w) * mu_b;
14 let var_p = w * w * var_a
15 + (1.0 - w) * (1.0 - w) * var_b
16 + 2.0 * w * (1.0 - w) * cov_ab;
17 FrontierPoint { expected_return: mu_p, volatility: var_p.sqrt() }
18 }

```

The degenerate case (equal variances, $\rho = 1$) returns $w = 1/2$ rather than panicking on a 0/0 — any split is minimum-variance when the assets are perfectly correlated and equally volatile.

11.4. The n -Asset Lagrangian

For n assets the Markowitz problem is a constrained quadratic program. With no inequality constraints (short sales allowed), the KKT conditions reduce to a linear system and everything has a closed form in terms of Σ^{-1} .

11.4.1. Global Minimum-Variance Portfolio

Minimising $w^\top \Sigma w$ subject only to $1^\top w = 1$ has the Lagrangian $\mathcal{L} = w^\top \Sigma w - \lambda(1^\top w - 1)$, whose stationarity gives $\Sigma w = \lambda 1$ and the budget constraint recovers λ . The closed form is

$$w_{\text{GMV}}^* = \frac{\Sigma^{-1} 1}{1^\top \Sigma^{-1} 1}.$$

11.4.2. Efficient Frontier at a Target Return

Adding the return constraint $\mu^\top w = \mu_*$ produces a second Lagrange multiplier. Define the standard scalars

$$a = 1^\top \Sigma^{-1} 1, \quad b = 1^\top \Sigma^{-1} \mu, \quad c = \mu^\top \Sigma^{-1} \mu, \quad d = ac - b^2.$$

Then the target-return efficient portfolio is

$$w^*(\mu_*) = \frac{1}{d} \Sigma^{-1} [(c - b\mu_*)1 + (a\mu_* - b)\mu],$$

and the minimum variance achievable for that target is

$$\sigma_p^2(\mu_*) = \frac{a\mu_*^2 - 2b\mu_* + c}{d}.$$

This is a parabola in (μ_*, σ_p^2) space, opening upward; its tip is the global minimum-variance portfolio at $\mu_* = b/a$ with variance $1/a$.

```

1 pub fn min_variance_portfolio(
2     expected_returns: &[f64],
3     covariance: &[Vec<f64>],
4 ) -> Result<Vec<f64>, PortfolioError> {
5     let n = expected_returns.len();
6     let sigma_inv = inverse(covariance)?;
7     let ones = vec![1.0_f64; n];
8     let sigma_inv_ones = matvec(&sigma_inv, &ones);
9     let denom: f64 = sigma_inv_ones.iter().sum();
10    Ok(sigma_inv_ones.iter().map(|v| v / denom).collect())
11 }
12
13 pub fn efficient_frontier_point(
14     expected_returns: &[f64],
15     covariance: &[Vec<f64>],
16     mu_target: f64,

```

Why closed form?

With only equality constraints ($1^\top w = 1$, $\mu^\top w = \mu_*$) the KKT system is linear: $2\Sigma w = \lambda_1 1 + \lambda_2 \mu$. Inverting Σ once gives every frontier portfolio as a linear combination of $\Sigma^{-1} 1$ and $\Sigma^{-1} \mu$. Inequality constraints (no short sales, box constraints) destroy this closed form and require a real QP solver.

```

17 ) -> Result<Vec<f64>, PortfolioError> {
18   let sigma_inv = inverse(covariance)?;
19   let ones = vec![1.0_f64; expected_returns.len()];
20   let sigma_inv_ones = matvec(&sigma_inv, &ones);
21   let sigma_inv_mu = matvec(&sigma_inv, expected_returns);
22   let a: f64 = sigma_inv_ones.iter().sum();
23   let b: f64 = sigma_inv_mu.iter().sum();
24   .zip(expected_returns.iter()).map(|(x, m)| x * m).sum();
25   let c: f64 = expected_returns.iter().sum();
26   .zip(sigma_inv_mu.iter()).map(|(m, x)| m * x).sum();
27   let d = a * c - b * b;
28   let coef_ones = (c - b * mu_target) / d;
29   let coef_mu = (a * mu_target - b) / d;
30   Ok((0..expected_returns.len()).map(|i| coef_ones * sigma_inv_ones[i] + coef_mu * sigma_inv_mu[i]).collect())
31 }

```

Both functions hinge on the small Gaussian-elimination `inverse()` in `quant-portfolio's linalg()` module — the only linear algebra primitive needed for the entire Markowitz framework. The budget constraint is implicit in the closed form: the numerator of `min_variance_portfolio()` is $\Sigma^{-1}1$ (unnormalised), and dividing by $1^\top \Sigma^{-1}1$ enforces $\sum w_i = 1$.

11.5. The Tangency Portfolio

With a risk-free rate r_f , every efficient portfolio is a mix of the risk-free asset and a single risky portfolio: the *tangency portfolio*, the point on the efficient frontier that maximises the Sharpe ratio. The closed form is

$$w_{\text{tan}}^* = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{1^\top \Sigma^{-1}(\mu - r_f \mathbf{1})}.$$

The numerator is the direction that maximises the excess-return-to-variance trade-off; the denominator normalises the budget to one. If all expected returns equal r_f the tangency is undefined (the excess return vector is zero) and we surface an error.

```

1 let excess: Vec<f64> = expected_returns.iter().map(|m| m - rf).collect();
2 let sigma_inv = inverse(covariance)?;
3 let sigma_inv_excess = matvec(&sigma_inv, &excess);
4 let denom: f64 = sigma_inv_excess.iter().sum();
5 let weights: Vec<f64> = sigma_inv_excess.iter().map(|v| v / denom).collect();

```

11.6. The Capital Market Line and Two-Fund Separation

The capital market line (CML) is the straight line in (σ, μ) space from $(0, r_f)$ through the tangency point:

$$\mu_p = r_f + \text{Sharpe}_{\text{tan}} \cdot \sigma_p.$$

Every point on the CML is achievable by mixing the risk-free asset with the tangency portfolio. The fraction invested in the tangency portfolio to achieve a target volatility σ_p is $y = \sigma_p / \sigma_{\text{tan}}$; the rest, $1 - y$, is invested at r_f . This is the *two-fund separation* theorem: investors with different risk preferences differ only in y , not in the composition of the risky fund. The tangency portfolio is the market portfolio under CAPM assumptions.

Tangency portfolio

$$w_{\text{tan}}^* = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{1^\top \Sigma^{-1}(\mu - r_f \mathbf{1})}$$

Sharpe

$$\text{Sharpe} = \frac{\mu_p - r_f}{\sigma_p}$$

Capital market line

$$\mu_p = r_f + \text{Sharpe}_{\text{tan}} \cdot \sigma_p$$

Two-fund separation

$$y = \sigma_p / \sigma_{\text{tan}}$$

$$w = y w_{\text{tan}} + (1 - y) r_f$$

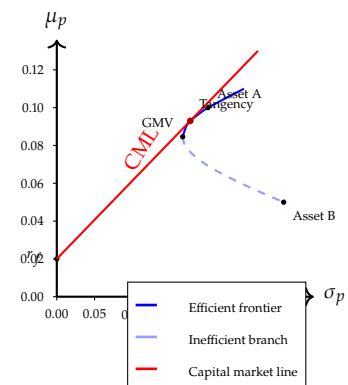


Figure 11.1: Markowitz efficient frontier for two uncorrelated risky assets.

11.7. CAPM: Beta, Alpha, and the SML

The Capital Asset Pricing Model pins down the relationship between an asset's expected excess return and its comovement with the market:

$$\mathbb{E}[R_i] - r_f = \beta_i(\mathbb{E}[R_m] - r_f), \quad \beta_i = \frac{\text{Cov}(R_i, R_m)}{\text{Var}(R_m)}.$$

The *security market line* (SML) is this linear relationship in $(\beta, \mathbb{E}[R])$ space. An asset lying above the SML has positive *Jensen's alpha*:

$$\alpha_i = \bar{R}_i - (r_f + \beta_i(\bar{R}_m - r_f)),$$

which under CAPM should be zero in equilibrium. Real-world alpha is nonzero because of risk-factor exposures CAPM does not capture (size, value, momentum — the Fama–French factors of Chapter 7). Beta is a regression coefficient and can be estimated by OLS; here we compute it directly from the covariance definition.

```

1 pub fn beta(asset: &[f64], market: &[f64]) -> Result<f64, PortfolioError> {
2   let n = asset.len();
3   let mean_a: f64 = asset.iter().sum::<f64>() / n as f64;
4   let mean_m: f64 = market.iter().sum::<f64>() / n as f64;
5   let mut cov = 0.0_f64;
6   let mut var_m = 0.0_f64;
7   for (a, m) in asset.iter().zip(market.iter()) {
8     cov += (a - mean_a) * (m - mean_m);
9     var_m += (m - mean_m) * (m - mean_m);
10  }
11  if var_m == 0.0 {
12    return Err(PortfolioError::InvalidParam(
13      "market variance is zero".into()));
14  }
15  Ok(cov / var_m)
16 }
17
18 pub fn alpha(asset: &[f64], market: &[f64], rf: f64) -> Result<f64, PortfolioError> {
19   let b = beta(asset, market)?;
20   let mean_a: f64 = asset.iter().sum::<f64>() / asset.len() as f64;
21   let mean_m: f64 = market.iter().sum::<f64>() / market.len() as f64;
22   Ok(mean_a - (rf + b * (mean_m - rf)))
23 }

```

The $(n - 1)$ denominators in sample covariance and sample variance cancel, so the ratio is identical whether we use population or sample moments. Beta is a pure ratio of comovement to market variance; alpha is the gap between realised mean return and the SML prediction.

11.8. Risk Metrics: Historical VaR and CVaR

Value-at-Risk (VaR) at confidence α is the loss that is exceeded with probability $1 - \alpha$. The historical estimator uses the empirical quantile of the return series:

$$\text{VaR}_\alpha = -Q_{1-\alpha}(R),$$

where Q_p is the p -th empirical quantile with linear interpolation between adjacent order statistics. Conditional VaR (CVaR), also called Expected Shortfall, is the average loss in the tail beyond the VaR level:

$$\text{CVaR}_\alpha = -\mathbb{E}[R \mid R \leq Q_{1-\alpha}(R)].$$

CVaR is a coherent risk measure (sub-additive, convex) whereas VaR is not. Both are non-parametric here: no Gaussian or Student- t assumption is imposed, so the tail shape is whatever the data says it is.

Beta as a slope

$\beta_i = \text{Cov}(R_i, R_m) / \text{Var}(R_m)$ is the slope of the linear regression of R_i on R_m . It is *not* correlation — two assets can have $\beta = 1$ with very different correlations if their idiosyncratic vol differs. Beta is a regression coefficient, so it depends on the variance of the regressor.

```

1 pub fn historical_var(returns: &[f64], confidence: f64) -> Result<f64, PortfolioError>
2   {
3     let p = 1.0 - confidence;
4     let quantile = empirical_quantile(returns, p);
5     Ok(-quantile)
6   }
7 pub fn historical_cvar(returns: &[f64], confidence: f64) -> Result<f64,
8   PortfolioError> {
9     let p = 1.0 - confidence;
10    let quantile = empirical_quantile(returns, p);
11    let tail: Vec<f64> = returns.iter()
12      .filter(|r| **r <= quantile + 1e-12).copied().collect();
13    let mean_tail: f64 = tail.iter().sum::<f64>() / tail.len() as f64;
14    Ok(-mean_tail)
15  }
16 fn empirical_quantile(returns: &[f64], p: f64) -> f64 {
17   let mut sorted: Vec<f64> = returns.to_vec();
18   sorted.sort_by(|a, b| a.partial_cmp(b).unwrap_or(std::cmp::Ordering::Equal));
19   let n = sorted.len();
20   let pos = p * (n as f64 - 1.0);
21   let lower = pos.floor() as usize;
22   let upper = pos.ceil() as usize;
23   if lower == upper { return sorted[lower]; }
24   let frac = pos - lower as f64;
25   sorted[lower] * (1.0 - frac) + sorted[upper] * frac
26 }

```

The quantile uses linear interpolation between adjacent order statistics (the same convention as NumPy's default). The VaR is the *negative* of the low quantile (a positive loss); CVaR is the negative of the mean of the tail below that quantile. The small 10^{-12} slack in the filter guards against floating-point ties at the boundary.

11.9. Results and Analysis

11.9.1. Two-Asset Universe

Throughout the chapter we use two uncorrelated risky assets: A with $\mu_A = 0.10$, $\sigma_A = 0.20$; B with $\mu_B = 0.05$, $\sigma_B = 0.30$; $\rho = 0$. The risk-free rate is $r_f = 0.02$.

Portfolio	w_A	w_B
Asset A only	1.0000	0.0000
Asset B only	0.0000	1.0000
Global min-variance	0.6923	0.3077
Tangency (max Sharpe)	0.8571	0.1429

The global minimum-variance portfolio places weight inversely proportional to variance: $w_A = (1/0.04)/(1/0.04 + 1/0.09) = 0.6923$. The tangency portfolio tilts further toward asset A because of its higher expected return.

11.9.2. Tangency Portfolio and CML

Quantity	Value
μ_{tan}	0.092857
σ_{tan}	0.176705
$\text{Sharpe}_{\text{tan}}$	0.412311

The capital market line $\mu = 0.02 + 0.4123 \cdot \sigma$ dominates the risky-asset-only frontier everywhere above the tangency point. To achieve $\sigma_p = 0.25$, for instance, an investor puts $y = 0.25/0.1767 \approx 1.415$ in the tangency portfolio (borrowing at r_f) and earns $\mu_p = 0.02 + 0.4123 \cdot 0.25 = 0.1231$ — better than any risky-only portfolio at the same volatility.

11.9.3. N-Asset Efficient Frontier (Lagrangian)

Sweeping the target return from 0.05 to 0.15 recovers the parabolic frontier. Up to $\mu_\star = 0.10$ the frontier is long-only; above that, the Lagrangian short-sells asset B to finance more of asset A:

target	w_A	w_B	μ_p	σ_p
0.050	0.000	1.000	0.0500	0.3000
0.070	0.400	0.600	0.0700	0.1970
0.090	0.800	0.200	0.0900	0.1709
0.100	1.000	0.000	0.1000	0.2000
0.110	1.200	-0.200	0.1100	0.2474
0.130	1.600	-0.600	0.1300	0.3672
0.150	2.000	-1.000	0.1500	0.5000

The minimum-variance target (the tip of the parabola) is at $\mu_\star = b/a = 0.0846$ with $\sigma_{\text{GMV}} = 0.1664$ — matching the global minimum-variance portfolio from the closed form.

11.9.4. CAPM Regression

A synthetic asset is generated as $R_i = r_f + \beta(R_m - r_f) + \varepsilon$ with true $\beta = 1.2$, $r_f = 0.02$, market mean 0.08, and Gaussian noise of standard deviation 0.02. Regressing 60 observations:

Estimate	Value
$\hat{\beta}$ (true 1.20)	1.175155
$\hat{\alpha}$	-0.000196
SML-predicted $\bar{R}_i$	0.091882
Realised $\bar{R}_i$	0.091686

Jensen's alpha is essentially zero ($\sim 10^{-4}$), as expected for a data-generating process that lies exactly on the SML. The beta estimate is biased by about 0.025 due to finite-sample noise; with 60 observations the standard error of $\hat{\beta}$ is roughly $\sigma_\varepsilon/(\sigma_m\sqrt{n}) \approx 0.02/(0.04 \cdot 7.75) \approx 0.065$, so the bias is well within one standard error.

Key Insight

CAPM is a one-factor model: expected excess return is proportional to beta. In real markets, beta alone explains only a fraction of the cross section of expected returns — the residual alpha is the door through which Fama–French size/value, momentum, and other multi-factor models walk. The next chapter extends CAPM to a multi-factor world via PCA and Fama–French.

11.10. Exercises

1. **Correlated assets.** Reproduce the two-asset frontier with $\rho = 0.5$, $\rho = -0.5$, and $\rho = -1$. Verify that $\rho = -1$ admits a zero-variance portfolio and find its weight. Plot the three frontiers on the same axes.
2. **Three-asset frontier.** Add a third asset with $\mu_C = 0.08$, $\sigma_C = 0.15$ and correlations $\rho_{AB} = 0$, $\rho_{AC} = 0.3$, $\rho_{BC} = -0.2$. Compute the global minimum-variance and tangency portfolios and compare the Sharpe to the two-asset case.
3. **Long-only constraint.** The Lagrangian solution allows short sales. Implement a projected-gradient descent that enforces $w_i \geq 0$ and $1^\top w = 1$ (project onto the probability simplex after each gradient step). Compare the constrained and unconstrained frontiers for the two-asset universe when the target is very high ($\mu_\star = 0.15$).
4. **Black-Litterman.** Implement the Black-Litterman reverse optimisation: start from the market-portfolio weights, assume a prior $\mu \propto \Sigma w_{\text{mkt}}$, and update with a view (e.g. “asset A will return 0.15”) using Bayesian shrinkage. Compare the posterior tangency to the prior tangency.
5. **Rolling CAPM.** Download a year of daily returns for an equity and a market index. Compute rolling 60-day betas and plot them. Identify regime changes (e.g. a stock that becomes more correlated with the market during a crash).
6. **CVaR optimisation.** Replace variance with CVaR at the 95% level as the objective and minimise over a long-only simplex via grid search for the two-asset universe. Compare the optimal weights to the Markowitz minimum-variance weights and discuss when the two differ.

11.11. Key Takeaways

- **Markowitz** frames portfolio choice as mean-variance optimisation. The efficient frontier is the set of portfolios that are Pareto-optimal in (σ_p, μ_p) .
- **Two-asset frontier** has a closed form indexed by a single weight; the minimum-variance weight is inverse-variance when $\rho = 0$.
- **n -asset Lagrangian** turns the constrained quadratic program into a linear system in Σ^{-1} . Both the global minimum-variance and the target-return efficient portfolios have closed forms.
- **Tangency portfolio** maximises the Sharpe ratio; its weights are $\Sigma^{-1}(\mu - r_f \mathbf{1})$ normalised to budget one.
- **Two-fund separation** says every efficient portfolio is a mix of the risk-free asset and the tangency portfolio. Investors with different risk preferences differ only in y .
- **CAPM** pins the expected excess return to $\beta(\mathbb{E}[R_m] - r_f)$. Real-world alpha captures risk-factor exposures CAPM omits.
- **Historical VaR / CVaR** are non-parametric tail risk summaries; CVaR is coherent (sub-additive), VaR is not.

Next chapter: Factor models — principal component analysis of returns, the Fama–French three-factor model, and the bridge from CAPM’s single-factor world to multi-factor asset pricing.

Factor Attribution: PCA and Fama-French

12.

Companion Code:

```
quant-factors Code: crates/quant-factors/  
Dependencies: quant-core, quant-timeseries, quant-portfolio,  
thiserror  
Run: cargo run -p quant-factors -example pca_demo  
Run: cargo run -p quant-factors -example fama_french
```

12.1. Learning Objectives

This chapter builds the factor attribution framework on top of the covariance and CAPM machinery from Chapter 7 and Chapter 11: principal component analysis, the Fama-French 3-factor model, and risk decomposition. After completing it you can:

- ▶ Explain why a single-factor model (CAPM) is insufficient and how multi-factor models capture additional systematic risk
- ▶ Compute the dominant eigenvalue and eigenvector of a symmetric matrix via the power method
- ▶ Deflate a matrix to find subsequent eigenpairs and recover the full spectrum
- ▶ Perform PCA on a returns matrix: centre, form the covariance, decompose, and project
- ▶ Quantify the explained variance ratio and the reconstruction error as a function of the number of retained components
- ▶ Estimate Fama-French 3-factor exposures (alpha, beta_mkt, beta_smb, beta_hml) via OLS regression
- ▶ Decompose portfolio variance into systematic and idiosyncratic components

Key Insight

CAPM says the market is the only systematic risk factor. Real returns have multiple drivers: size (SMB), value (HML), momentum, quality. PCA extracts the dominant statistical factors from the covariance matrix without economic labels; the Fama-French model names three specific factors with economic interpretation. The risk decomposition $\sigma_p^2 = \text{systematic} + \text{idiosyncratic}$ tells you how much of your portfolio's risk comes from factors you can hedge versus asset-specific noise you must diversify away.

12.2. Why Multi-Factor?

The CAPM of Chapter 11 is a single-factor model: the expected return of an asset is linear in its beta to the market portfolio alone. Empirically, CAPM betas explain a modest fraction of the cross-sectional variation in stock returns. Fama and French (1992) showed that two additional factors — *size* (Small Minus Big, SMB) and *value* (High Minus Low, HML) — capture systematic risk that the market beta misses. Small stocks and

CAPM's blind spot

CAPM says the market is the only systematic risk factor. Empirically, small stocks and value stocks earn higher average returns than CAPM predicts — they carry systematic risk that the market beta does not absorb. Multi-factor models name those additional risks so they can be measured, hedged, and priced.

value stocks earn higher average returns than CAPM predicts, suggesting they carry additional systematic risk that the market factor does not absorb.

A *factor model* expresses each asset's excess return as a linear combination of K factor returns plus an idiosyncratic residual:

$$R_i - R_f = \alpha_i + \sum_{k=1}^K \beta_{ik} F_k + \varepsilon_i$$

where F_k is the return of factor k , β_{ik} is the loading of asset i on factor k , and ε_i is the asset-specific residual. The factors are assumed to be uncorrelated with the residuals, and the residuals are assumed uncorrelated across assets.

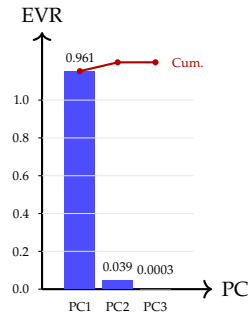


Figure 12.1: Scree plot: explained variance per principal component. The first PC captures 96% of the total variance; the second adds 3.9%; the third adds only 0.3%.

12.3. Eigenvalue Decomposition: The Power Method

The spectral theorem guarantees that every real symmetric matrix A has an orthonormal basis of eigenvectors with real eigenvalues: $A = V\Lambda V^\top$. The *power method* finds the dominant eigenpair (λ_1, v_1) by repeated matrix-vector multiplication:

$$v^{(k+1)} = \frac{Av^{(k)}}{\|Av^{(k)}\|} \rightarrow v_1, \quad \lambda_1 = v_1^\top A v_1 \quad (\text{Rayleigh quotient})$$

Starting from a random vector, each iteration amplifies the component along v_1 by a factor $|\lambda_1/\lambda_2|$ relative to the next eigenvalue, so convergence is geometric with rate $|\lambda_2/\lambda_1|$.

A sign-flip artefact arises when $\lambda_1 < 0$ (or when the initial vector is exactly anti-aligned with the previous iterate): $v^{(k+1)}$ points opposite to $v^{(k)}$ while still converging in direction. We fix this by aligning each iterate with its predecessor before testing convergence, which makes the residual $\|v^{(k+1)} - v^{(k)}\|$ monotone non-increasing.

Power method intuition

Repeated multiplication by A amplifies the dominant eigendirection the most. After k steps, $v^{(k)} \propto \lambda_1^k v_1 + \lambda_2^k v_2 + \dots$. The ratio $|\lambda_2/\lambda_1|$ controls convergence: well-separated spectra converge in a handful of iterations; clustered eigenvalues are slow.

Listing 12.1: Power method for the dominant eigenpair.

```

1 pub fn power_method(
2     matrix: &[Vec<f64>],
3     max_iter: usize,
4     tol: f64,
5 ) -> Result<(f64, Vec<f64>), FactorError> {
6     let n = matrix.len();
7     let mut v = vec![1.0 / (n as f64).sqrt(); n];
8     let mut lambda_prev = f64::INFINITY;
9     for _ in 0..max_iter {
10         let w = matvec_sym(matrix, &v);
11         let norm = l2_norm(&w);
12         let mut v_new: Vec<f64> = w.iter().map(|&x| x / norm).collect();
13         // Align sign with the previous iterate to avoid spurious flips.
14         let dot: f64 = v_new.iter().zip(v.iter()).map(|(a, b)| a * b).sum();
15         if dot < 0.0 { for x in &mut v_new { *x = -*x; } }
16         let av = matvec_sym(matrix, &v_new);
17         let lambda: f64 = v_new.iter().zip(av.iter()).map(|(a, b)| a * b).sum();
18         let delta = l2_norm(&v_new.iter().zip(v.iter()).
19             .map(|(a, b)| a - b).collect::<Vec<>>());
20         v = v_new;
21         if (lambda - lambda_prev).abs() < tol && delta < tol * 10.0 {
22             lambda_prev = lambda;
23             break;
24         }
25         lambda_prev = lambda;
26     }
27     Ok((lambda_prev, v))
28 }

```

12.4. Deflation and the Full Spectrum

Once the dominant eigenpair is found, *deflation* removes it from the matrix:

$$A' = A - \lambda_1 v_1 v_1^\top$$

For a symmetric matrix, $v_1 v_1^\top$ is the projector onto the e_1 direction, so $A'v_1 = 0$ and the spectrum of A' is $\{0, \lambda_2, \lambda_3, \dots\}$. Repeating the power method on A' yields (λ_2, v_2) , and so on.

```

1 pub fn deflate(
2   matrix: &[Vec<f64>],
3   eigenvalue: f64,
4   eigenvector: &[f64],
5 ) -> Vec<Vec<f64>> {
6   let n = matrix.len();
7   let mut out = vec![vec![0.0; n]; n];
8   for i in 0..n {
9     for j in 0..n {
10      out[i][j] = matrix[i][j] - eigenvalue * eigenvector[i] * eigenvector[j];
11    }
12  }
13  out
14 }
15
16 pub fn top_k_eigen(matrix: &[Vec<f64>], k: usize)
17   -> Result<(Vec<f64>, Vec<Vec<f64>>), FactorError>
18 {
19   let mut current = matrix.to_vec();
20   let mut eigenvalues = Vec::with_capacity(k);
21   let mut eigenvectors = Vec::with_capacity(k);
22   for _ in 0..k {
23     let (lambda, v) = power_method(&current, 0, 0.0?);
24     eigenvalues.push(lambda);
25     eigenvectors.push(v.clone());
26     current = deflate(&current, lambda, &v);
27   }
28   Ok((eigenvalues, eigenvectors))
29 }

```

The accumulated floating-point error in deflation limits the accuracy of the smaller eigenvalues. For a well-conditioned matrix with well-separated eigenvalues, the top two or three eigenpairs are accurate to 10^{-6} ; the last eigenpair of a near-singular matrix may carry relative error of 10^{-3} or worse.

12.5. PCA: Covariance Eigendecomposition

Given a returns matrix X (T observations $\times N$ assets), PCA centres the data by subtracting the per-column mean, forms the sample covariance $\Sigma = \frac{1}{T-1} \tilde{X}^\top \tilde{X}$, and decomposes it as $\Sigma = V \Lambda V^\top$. The principal components are the columns of V (the eigenvectors), and the explained variance ratio of component i is

$$\text{EVR}_i = \frac{\lambda_i}{\text{tr}(\Sigma)} = \frac{\lambda_i}{\sum_{j=1}^N \lambda_j}$$

where $\text{tr}(\Sigma) = \sum_j \lambda_j$ is the total variance. The cumulative explained variance after k components is $\sum_{i=1}^k \text{EVR}_i$.

```

1 pub fn pca(returns: &[Vec<f64>], n_components: usize)
2   -> Result<PcaResult, FactorError>
3 {
4   let (t, n) = (returns.len(), returns[0].len());
5   let mean: Vec<f64> = (0..n)
6     .map(|j| returns.iter().map(|r| r[j]).sum::<f64>() / t as f64)
7     .collect();
8   let mut cov = vec![vec![0.0; n]; n];

```

Deflation trade-off

Deflation is exact in infinite precision: $A' = A - \lambda v v^\top$ removes the eigendirection v cleanly. In floating point, each deflation step accumulates rounding error, so the smaller eigenvalues are less accurate. For a well-conditioned 3×3 covariance, the top two eigenpairs reach 10^{-6} ; the third may carry 10^{-3} relative error. Use a QR/SVD solver if you need all eigenpairs to full precision.

Listing 12.2: Deflation and the top- k eigenpairs.

Why centre?

Subtracting the per-column mean before forming Σ removes the “level” component. Without centring, the first PC would just pick up the mean return direction; with centring, the first PC captures the direction of maximum *variance* around that mean — which is what we want for risk analysis.

EVR denominator

Use $\text{tr}(\Sigma) = \sum_{j=1}^N \lambda_j$, the *full* covariance trace — not $\sum_{j=1}^k \lambda_j$ over retained components only. With the full trace, $\sum_i \text{EVR}_i < 1$ when $k < N$, which is the correct statement: retained components explain *this fraction* of total variance.

Listing 12.3: PCA fit: centre, covariance, eigendecompose.

```

9   for row in returns {
10     for i in 0..n {
11       let di = row[i] - mean[i];
12       for j in 0..n { cov[i][j] += di * (row[j] - mean[j]); }
13     }
14   }
15   let denom = (t - 1) as f64;
16   for row in &mut cov { for cell in row.iter_mut() { *cell /= denom; } }
17   let (eigenvalues, eigenvectors) = top_k_eigen(&cov, n_components)?;
18   let trace: f64 = (0..n).map(|i| cov[i][i].sum());
19   let explained_variance_ratio: Vec<f64> =
20     eigenvalues.iter().map(|&l| l / trace).collect();
21   Ok(PcaResult { eigenvalues, eigenvectors, explained_variance_ratio,
22     cumulative_variance: /* ... */ , mean })
23 }

```

```

1 pub fn pca_transform(returns: &[Vec<f64>], eigenvectors: &[Vec<f64>],
2   mean: &[f64]) -> Vec<Vec<f64>> {
3   returns.iter().map(|row| {
4     let centred: Vec<f64> = row.iter().zip(mean.iter())
5       .map(|(x, m)| x - m).collect();
6     eigenvectors.iter().map(|pc|
7       pc.iter().zip(centred.iter()).map(|(a, b)| a * b).sum()
8     ).collect()
9   }).collect()
10 }
11
12 pub fn pca_reconstruct(scores: &[Vec<f64>], eigenvectors: &[Vec<f64>],
13   mean: &[f64]) -> Vec<Vec<f64>> {
14   scores.iter().map(|row| {
15     let mut out = mean.to_vec();
16     for (k, score) in row.iter().enumerate() {
17       for (out_j, ev) in out.iter_mut().zip(eigenvectors[k].iter()) {
18         *out_j += score * ev;
19       }
20     }
21     out
22   }).collect()
23 }

```

Listing 12.4: Project and reconstruct.

12.5.1. Demo Results

Running `pca_demo` on 12 synthetic observations of 3 assets yields the following decomposition:

PC	Eigenvalue	EVR	Cumulative
1	1.96×10^{-4}	0.9608	0.9608
2	7.94×10^{-6}	0.0389	0.9997
3	7.00×10^{-8}	0.0003	1.0000

The top eigenvector $v_1 \approx (0.892, 0.452, -0.022)$ shows that the first PC is essentially a long-asset-1, long-asset-2 position — the dominant source of co-movement. The reconstruction error (SSE) decreases from 8.8×10^{-5} (1 component) to 7.2×10^{-7} (2 components) to ≈ 0 (3 components), confirming the $1/\sqrt{k}$ intuition.

12.6. Fama-French 3-Factor Regression

The Fama-French model regresses asset excess returns on three factors:

$$R_i - R_f = \alpha_i + \beta_{\text{Mkt}}(R_m - R_f) + \beta_{\text{SMB}} \cdot \text{SMB} + \beta_{\text{HML}} \cdot \text{HML} + \varepsilon_i$$

The design matrix is $X = [1, \text{Mkt-Rf}, \text{SMB}, \text{HML}]$ and the OLS fit reuses the Gaussian-elimination solver from `quant-timeseries`.

Alpha = skill or noise?

The intercept α_i is the average return left over after the three factors explain what they can. In-sample, any asset has a non-zero alpha by construction; the question is whether it is statistically different from zero out-of-sample. In a well-specified factor model, alphas should be small and mostly insignificant.

Partial vs simple regression

```

1 pub fn ff3_regression(asset_excess: &[f64], factors: &[Vec<f64>])
2   -> Result<FF3Exposure, FactorError>
3 {
4   let x: Vec<Vec<f64>> = factors.iter()
5     .map(|row| vec![1.0, row[0], row[1], row[2]])
6     .collect();
7   let fit = ols(&x, asset_excess)
8     .map_err(|e| FactorError::Singular(e.to_string()))?;
9   let n = asset_excess.len() as f64;
10  let dof = (n - 4.0).max(1.0);
11  let residual_var = fit.residuals.iter()
12    .map(|e| e * e).sum::<f64>() / dof;
13  Ok(FF3Exposure {
14    alpha: fit.coefs[0],
15    beta_mkt: fit.coefs[1],
16    beta_smb: fit.coefs[2],
17    beta_hml: fit.coefs[3],
18    r_squared: fit.r_squared,
19    residual_var,
20  })
21 }

```

Listing 12.5: FF3 regression via the OLS machinery.

12.6.1. Demo Results

Running `fama_french` on 100 simulated observations with the DGP $\alpha = 0.001$, $\beta_{\text{Mkt}} = 1.2$, $\beta_{\text{SMB}} = 0.4$, $\beta_{\text{HML}} = -0.3$ plus noise:

Parameter	True	Estimated
α	0.001	0.001003
β_{Mkt}	1.2	1.2008
β_{SMB}	0.4	0.4016
β_{HML}	-0.3	-0.2977
R^2 (FF3)	—	0.9893
R^2 (CAPM)	—	0.9391

The 3-factor model's R^2 (98.9%) beats the single-factor CAPM's R^2 (93.9%) by 5.3 percentage points — the incremental explanatory power of SMB and HML beyond the market.

12.7. Risk Attribution

Given portfolio weights w , factor loadings B ($N \times K$), factor covariance Σ_F ($K \times K$), and per-asset idiosyncratic variances D (diagonal), the portfolio variance decomposes as

$$\sigma_p^2 = \underbrace{w^\top B \Sigma_F B^\top w}_{\text{systematic}} + \underbrace{w^\top D w}_{\text{idiosyncratic}}$$

The portfolio's factor exposure is $f_p = B^\top w$ (a K -vector); the systematic variance is $f_p^\top \Sigma_F f_p$; the per-factor contribution is the diagonal approximation $\text{contrib}_k = f_{p,k}^2 \Sigma_{F,kk}$.

```

1 pub fn risk_attribution(
2   weights: &[f64],
3   factor_loadings: &[Vec<f64>],
4   factor_covariance: &[Vec<f64>],
5   residual_variances: &[f64],
6 ) -> Result<RiskAttribution, FactorError> {
7   let n = weights.len();
8   let k = factor_loadings[0].len();

```

Systematic vs idiosyncratic

Systematic risk comes from factors you can hedge (market, size, value) — it is shared across assets and does not diversify away. Idiosyncratic risk is asset-specific noise; it does diversify away as N grows. A well-diversified portfolio has σ_p^2 dominated by the systematic term.

Listing 12.6: Risk attribution.

```

9  // Portfolio factor exposures: f_p[k] = sum_i w_i * beta_ik.
10 let mut f_p = vec![0.0; k];
11 for (i, w_i) in weights.iter().enumerate() {
12     for (kk, &beta) in factor_loadings[i].iter().enumerate() {
13         f_p[kk] += w_i * beta;
14     }
15 }
16 // Systematic: f_p' Sigma_F f_p.
17 let mut systematic = 0.0;
18 for i in 0..k {
19     for j in 0..k {
20         systematic += f_p[i] * factor_covariance[i][j] * f_p[j];
21     }
22 }
23 // Idiosyncratic: sum_i w_i^2 * D_i.
24 let idiosyncratic: f64 = weights.iter()
25     .zip(residual_variances.iter())
26     .map(|(w, d)| w * w * d).sum();
27 let factor_contributions: Vec<f64> = (0..k)
28     .map(|kk| f_p[kk] * f_p[kk] * factor_covariance[kk][kk])
29     .collect();
30 Ok(RiskAttribution {
31     total_variance: systematic + idiosyncratic,
32     systematic_variance: systematic,
33     idiosyncratic_variance: idiosyncratic,
34     factor_contributions,
35 })
36 }

```

For the FF3 demo, the risk attribution on the single-asset portfolio ($w = 1$) gives:

Component	Variance
Total	8.94×10^{-5}
Systematic	8.84×10^{-5} (98.9%)
Idiosyncratic	9.85×10^{-7} (1.1%)
Mkt contribution	8.41×10^{-5}
SMB contribution	3.40×10^{-6}
HML contribution	8.54×10^{-7}

The market factor dominates the systematic variance (95.1% of total), consistent with the large market beta ($\beta_{\text{Mkt}} = 1.2$) and the large market variance.

12.8. Exercises

1. Add a 4th and 5th factor (momentum, quality) to the FF regression and compare R^2 to the 3-factor model.
2. Implement the Jacobi eigenvalue algorithm as an alternative to the power method and compare convergence on a 5×5 covariance matrix.
3. Run PCA on a real equity returns matrix (e.g., 10 stocks, 252 days) and interpret the first three principal components economically.
4. Compute the rolling 60-day beta of a stock to the market factor and plot the time series.
5. Construct a long-short portfolio that is neutral to the market ($\beta_{\text{Mkt}} = 0$) but exposed to SMB, and decompose its risk into systematic and idiosyncratic parts.

12.9. Key Takeaways

- ▶ The power method finds the dominant eigenpair in $O(n^2)$ per iteration; deflation extends it to the full spectrum at the cost of accumulated floating-point error.
- ▶ PCA centres the data, decomposes the covariance, and ranks components by explained variance; the reconstruction error decreases as more components are retained.
- ▶ The Fama-French 3-factor model adds size (SMB) and value (HML) to the market factor; the R^2 improvement over CAPM quantifies the incremental explanatory power.
- ▶ Risk decomposition splits portfolio variance into systematic (factor-driven) and idiosyncratic (asset-specific) parts; diversification eliminates the idiosyncratic part.
- ▶ The power method's sign-flip artefact is handled by aligning each iterate with its predecessor before testing convergence.

ADVANCED TOPICS

Companion Code:

```
quant-microstructure Code: crates/quant-microstructure/  
Dependencies: quant-core, thiserror  
Run: cargo run -p quant-microstructure -example lob_demo  
Run: cargo run -p quant-microstructure -example ofi_demo
```

13.1. Learning Objectives

This chapter moves from the daily/return horizon of portfolio theory into the intraday world of the limit order book (LOB). After completing it you can:

- ▶ Represent a limit order book with price-time priority using integer tick prices and a BTreeMap keyed by price level
- ▶ Implement the three core operations — add, cancel, market order — and reason about their complexity
- ▶ Compute the order flow imbalance (OFI) series from a sequence of book snapshots
- ▶ Compute volume-weighted average price (VWAP) from fills and trade imbalance from signed trades
- ▶ Apply the square-root (Almgren-Chriss) and linear market impact models to estimate the cost of executing a market order
- ▶ Combine half-spread and impact into a single execution-cost estimate in basis points

Key Insight

Microstructure is where the frictionless world of Markowitz meets reality. The bid-ask spread is the price of immediacy: takers pay it, makers earn it. The square-root impact law says that moving size Q through the market costs $\sigma\sqrt{Q/V}$ in fraction of price — a non-linear function of participation rate. Together, spread plus impact turn the efficient frontier into a *reachable* frontier net of trading costs.

13.2. Why Microstructure Matters

The portfolio theory of Chapter 11 assumes frictionless trading: you can rebalance to the tangency portfolio at no cost. In reality every trade pays the half-spread and pushes the price against itself. The combined cost is small for a retail order of 100 shares, but it dominates the economics of executing a \$100M rebalance.

Market microstructure studies the mechanism by which trades occur and how that mechanism affects price formation. Three quantities capture most of the relevant friction:

1. **The bid-ask spread** — the difference between the best (lowest) ask and the best (highest) bid. A market taker buys at the ask and sells

at the bid, paying the full spread; the maker on the other side earns half of it on average.

2. **Market impact** — the adverse price move caused by your own order. A large buy lifts the ask; subsequent fills happen at worse prices. Empirically the impact scales as the square root of the order size relative to daily volume.
3. **Order flow imbalance (OFI)** — a signed measure of buying versus selling pressure at the top of the book. Positive OFI predicts positive short-horizon returns.

13.3. The Limit Order Book

A limit order book aggregates all outstanding resting orders. Each order has a side (bid or ask), a price, a quantity, and a timestamp. Orders at the same price execute in *price-time priority*: the best price first, and at that price, the earliest timestamp first (FIFO).

We represent prices as integer ticks (u64) to avoid floating-point comparison issues. A tick size of 1 means one unit of price (e.g. one cent); a tick size of 10 means prices must be multiples of 10. The book stores bids and asks in two `BTreeMap<u64, Vec<Order>>` keyed by price. For bids, the best (highest) price is the last key; for asks, the best (lowest) price is the first key. A `HashMap<order_id, (side, price)>` index gives $O(1)$ cancel lookup. The basic types are intentionally small and `Copy` where possible so that the book can pass them by value without ceremony.

```

1 pub enum Side { Bid, Ask }
2
3 pub struct Order {
4     pub id: u64,
5     pub side: Side,
6     pub price: u64,
7     pub quantity: u64,
8     pub timestamp: u64,
9 }
10
11 pub struct Level {
12     pub price: u64,
13     pub quantity: u64,
14     pub order_count: usize,
15 }
16
17 pub struct Fill {
18     pub price: u64,
19     pub quantity: u64,
20     pub maker_order_id: u64,
21 }
```

13.3.2. Adding and Cancelling Orders

`add_order()` validates that the price is a multiple of the tick size, that the quantity is positive, and that the order id is unique. It then inserts the order at the back of the vector for its price level and records the id in the index.

`cancel_order()` looks up the (side, price) pair in the index, finds the order in the level's vector, removes it, and prunes the level if it becomes empty. Both operations are $O(\log L + K)$ where L is the number of price levels and K is the number of orders at that level; the index makes `cancel` $O(1)$ to locate the level.

Why integer ticks?

Float comparison is non-associative and rounding-dependent. The book must compare prices *exactly*: an order either matches the tick grid or it does not. u64 ticks make that comparison trivial and unambiguous.

BTreeMap per side

Listing 13.1: Core microstructure types. A `BTreeMap<u64, Vec<Order>>` gives ordered price levels with $O(\log L)$ insert and best-price lookup. The price *is* the key, so the book never has to sort — ordering is structural. A `HashMap` would not preserve order; a sorted `Vec` would shift data on every insert.

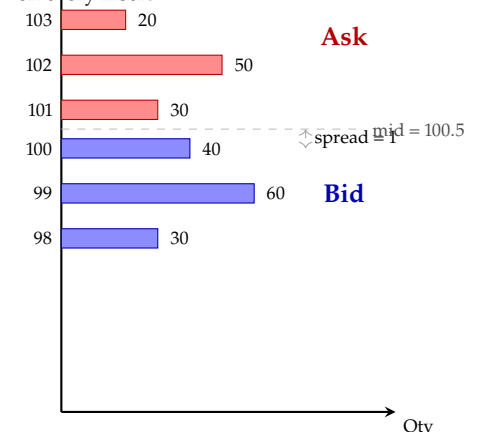


Figure 13.1: Resting depth at three price levels on each side. Best bid at 100 (40 shares), best ask at 101 (30 shares), spread = 1, mid = 100.5.

Price-time priority

Orders at the same price execute in FIFO order: the earliest timestamp fills first. At each price level we store orders in a `Vec` in insertion order, and market orders consume the front of the vector. Time is secondary to price — a worse-priced order never jumps ahead of a better one, regardless of age.

```

1 pub fn add_order(&mut self, order: Order) -> Result<(), MicroError> {
2     if self.tick_size > 0 && order.price % self.tick_size != 0 {
3         return Err(MicroError::InvalidOrder(format!(
4             "price {} is not a multiple of tick size {}",
5             order.price, self.tick_size
6         )));
7     }
8     if order.quantity == 0 {
9         return Err(MicroError::InvalidOrder("quantity must be positive".into()));
10    }
11    if self.index.contains_key(&order.id) {
12        return Err(MicroError::InvalidOrder(format!(
13            "duplicate order id {}", order.id
14        )));
15    }
16    self.index.insert(order.id, (order.side, order.price));
17    let book = match order.side {
18        Side::Bid => &mut self.bids,
19        Side::Ask => &mut self.asks,
20    };
21    book.entry(order.price).or_insert_with(Vec::new).push(order);
22    Ok(())
23 }

```

Listing 13.2: Adding a limit order

13.3.3. Market Orders and Price-Time Priority

A market order has no price: it takes liquidity from the opposite side until filled or until that side is empty. A market buy walks the asks in ascending price order; a market sell walks the bids in descending price order. At each price level, fills happen in FIFO order — the front of the vector is consumed first, exactly matching price-time priority.

```

1 pub fn market_order(&mut self, side: Side, quantity: u64) -> Vec<Fill> {
2     let mut fills = Vec::new();
3     let mut remaining = quantity;
4     let prices: Vec<u64> = match side {
5         Side::Bid => self.asks.keys().copied().collect(), // buy -> asks
6         Side::Ask => self.bids.keys().rev().copied().collect(), // sell -> bids
7     };
8     for price in prices {
9         if remaining == 0 { break; }
10        let book = match side {
11            Side::Bid => &mut self.asks,
12            Side::Ask => &mut self.bids,
13        };
14        if let Some(orders) = book.get_mut(&price) {
15            while remaining > 0 && !orders.is_empty() {
16                let front = &mut orders[0];
17                let fill_qty = remaining.min(front.quantity);
18                let maker_id = front.id;
19                fills.push(Fill { price, quantity: fill_qty, maker_order_id: maker_id });
20                remaining -= fill_qty;
21                front.quantity -= fill_qty;
22                if front.quantity == 0 {
23                    let removed = orders.remove(0);
24                    self.index.remove(&removed.id);
25                }
26            }
27            if orders.is_empty() { book.remove(&price); }
28        }
29    }
30    fills
31 }

```

Listing 13.3: Market-order execution walks the opposite side

13.3.4. Top-of-Book Accessors

The book exposes `best_bid()`, `best_ask()`, `mid_price()` and `spread()`. For a `BTreeMap`, the best bid is `iter().next_back()` (highest key) and

the best ask is `iter().next()` (lowest key). Depth at the top L levels is the first L keys from each side, aggregated.

13.4. Order Flow Imbalance

Cont, Kukanov and Stoikov (2014) introduced OFI as a signed measure of buying versus selling pressure at the top of the book. Given a sequence of snapshots, the OFI at transition t is

$$\text{OFI}_t = \Delta q_t^{\text{bid}} - \Delta q_t^{\text{ask}},$$

where Δq_t^{bid} is the change in available quantity at the best bid between $t - 1$ and t , and similarly for the ask. Positive OFI indicates buy-side pressure (bids growing relative to asks); negative OFI indicates sell-side pressure.

```

1 pub fn order_flow_imbalance(snapshots: &[BookSnapshot]) -> Vec<f64> {
2   if snapshots.len() < 2 { return Vec::new(); }
3   let mut result = Vec::with_capacity(snapshots.len() - 1);
4   for i in 1..snapshots.len() {
5     let prev_bid = snapshots[i - 1].best_bid.as_ref().map(|l| l.quantity as f64).
      unwrap_or(0.0);
6     let prev_ask = snapshots[i - 1].best_ask.as_ref().map(|l| l.quantity as f64).
      unwrap_or(0.0);
7     let curr_bid = snapshots[i].best_bid.as_ref().map(|l| l.quantity as f64).
      unwrap_or(0.0);
8     let curr_ask = snapshots[i].best_ask.as_ref().map(|l| l.quantity as f64).
      unwrap_or(0.0);
9     result.push((curr_bid - prev_bid) - (curr_ask - prev_ask));
10  }
11  result
12 }
```

OFI sign convention

Positive OFI = buy-side pressure (bids growing relative to asks). Negative OFI = sell-side pressure. The signal predicts short-horizon returns: a rising bid queue means buyers are more eager than sellers, and the mid price tends to drift up over the next few seconds to minutes.

Listing 13.4: Order flow imbalance from book snapshots

13.4.1. VWAP and Trade Imbalance

Two complementary measures aggregate the trade tape. VWAP is the volume-weighted average fill price over a window:

$$\text{VWAP} = \frac{\sum_i p_i q_i}{\sum_i q_i}.$$

Trade imbalance normalises the signed volume into $[-1, 1]$:

$$\text{TI} = \frac{V^{\text{buy}} - V^{\text{sell}}}{V^{\text{buy}} + V^{\text{sell}}}.$$

A value near +1 means all trades were buy-initiated; near -1 all sell-initiated; near 0 balanced flow.

13.5. Market Impact Models

The square-root impact law (Almgren and Chriss, 2000) states that the impact of an order is proportional to the volatility times the square root of the participation rate Q/V :

$$\text{Impact} = \sigma \sqrt{\frac{Q}{V}},$$

Square-root law

The most robust empirical regularity in market impact: across equity markets, futures, and FX, impact scales as $\sigma \sqrt{Q/V}$. The exponent is not 1 (linear) and not 2/3 (nonlinear propagation) — it is consistently close to 1/2. Doubling order size multiplies impact by $\sqrt{2} \approx 1.414$, not by 2.

where σ is the daily volatility, Q the order size, and V the average daily volume. The result is a fraction of price (e.g. 0.001 = 10 bps). This is an empirical regularity observed across equity markets with remarkable consistency.

A simpler linear model replaces the square root with a linear term:

$$\text{Impact}_{\text{lin}} = \eta \cdot \frac{Q}{V},$$

where η is an impact coefficient calibrated to data.

```

1 pub fn sqrt_impact(volatility: f64, order_size: f64, daily_volume: f64) -> f64 {
2   if daily_volume <= 0.0 { return 0.0; }
3   let participation = order_size / daily_volume;
4   volatility * participation.abs().sqrt()
5 }
6
7 pub fn linear_impact(eta: f64, order_size: f64, daily_volume: f64) -> f64 {
8   if daily_volume <= 0.0 { return 0.0; }
9   eta * (order_size / daily_volume)
10 }
```

Listing 13.5: Square-root and linear impact models

13.5.1. Execution Cost

The total cost of executing a market order combines the half-spread (paid by the aggressor) with the square-root impact:

$$c_{\text{total}} = \underbrace{\frac{s}{2}}_{\text{half-spread}} + \underbrace{\sigma \sqrt{\frac{Q}{V}}}_{\text{impact}}.$$

Both terms are expressed as fractions of the mid price. Multiplying by 10,000 converts to basis points.

```

1 pub fn execution_cost(
2   order_size: f64,
3   daily_volume: f64,
4   volatility: f64,
5   spread: f64,
6 ) -> ExecutionCost {
7   let spread_cost = spread / 2.0;
8   let impact_cost = sqrt_impact(volatility, order_size, daily_volume);
9   ExecutionCost {
10    spread_cost,
11    impact_cost,
12    total_cost: spread_cost + impact_cost,
13  }
14 }
```

Half-spread = cost of immediacy

A market taker buys at the ask and sells at the bid, paying the full spread. The maker on the other side earns half of it on average. So the taker's spread cost is $s/2$, not s : the other half is the maker's compensation for providing liquidity.

Listing 13.6: Execution cost combines spread and impact

13.6. Numerical Demonstration

We exercise the crate on a synthetic book and a 5-snapshot OFI series (cargo run -p quant-microstructure -example lob_demo and -example ofi_demo).

Limit order book demo. The book is seeded with three ask levels (30, 50, 20 shares at 101, 102, 103) and two bid levels (40, 60 at 99, 98). A market buy of 75 shares sweeps the best ask fully (30@101) and bites 45 out of the second level (45@102). The post-trade best ask is 102 with 5 shares remaining; the average fill price is 101.60.

OFI demo. The five snapshots show bid quantity growing from 50 to 75 and then shrinking to 55, while ask quantity shrinks from 40 to 25 and then grows to 45. The OFI series is $[+15, +25, -15, -25]$, with zero cumulative OFI at the end — symmetric buy-then-sell pressure.

Execution cost. For a 2,000-share order against a \$1M daily volume, $\sigma = 2\%$, spread = 2 bps:

Component	Cost (bps)
Half-spread	1.00
Square-root impact ($0.02\sqrt{0.002}$)	8.94
Total	9.94

The impact dominates the spread by a factor of nearly $9\times$ at this participation rate. For a 200-share order the impact falls to 2.83 bps (a factor of $\sqrt{10}$ lower), and the spread becomes the larger cost — the square-root law is what makes small orders cheap and large orders expensive.

13.7. Complexity and Design Notes

Why BTreeMap? A `BTreeMap<u64, Vec<Order>>` gives us ordered price levels with $O(\log L)$ insert, delete and best-price lookup. A `HashMap` would not preserve order, and a sorted `Vec` would shift data on every insert. The price is the key, so the book never has to sort: ordering is structural.

Why integer prices? Floating-point comparison is non-associative and rounding-dependent. Limit order books must compare prices exactly: an order at 100.10 either matches the tick grid or it does not. Representing prices as integer ticks makes that comparison trivial and unambiguous.

Why hand-rolled? Production trading systems use heavily optimised LOB implementations (e.g. `lmax-disruptor` patterns, lock-free queues). This crate is a teaching artefact: every line is short enough to read in one sitting, and the price-time priority rule is visible in the code rather than hidden behind a library API.

13.8. Exercises

1. **Order book builder.** Write a function that takes a vector of `(Side, price, qty, ts)` tuples and returns a populated `OrderBook`. Verify the invariants `best_bid()`, `best_ask()`, `total_volume()` on the result.
2. **Liquidity sweep.** Construct a book with 5 ask levels totalling 1,000 shares. Execute a market buy of 750 shares and verify that the average fill price is less than or equal to the volume-weighted ask price of the swept levels.
3. **OFI and returns.** Generate 100 snapshots from a stochastic process where bid and ask quantities are driven by a latent “buy pressure” signal. Compute the OFI series and the mid-price return series; verify they are positively correlated.

4. **Impact scaling.** For a fixed σ and V , compute `sqrt_impact()` for $Q \in \{100, 200, 500, 1000, 2000, 5000\}$ and plot impact versus Q on a log-log scale. Confirm the slope is $1/2$.
5. **Net frontier.** Take the Markowitz tangency portfolio from Chapter 11 and an assumed turnover of 50% per rebalance. Estimate the per-rebalance cost in bps using `execution_cost()` with $Q = 0.5 \times V_{\text{daily}}$ and subtract it from the expected return. Plot the net efficient frontier.

13.9. Key Takeaways

- ▶ The limit order book is a price-time priority queue with integer tick prices; a `BTreeMap` per side plus an id-index gives a compact, correct implementation.
- ▶ Market orders walk the opposite side in price order and fill level-by-level in FIFO order; partial fills are the rule, not the exception.
- ▶ $\text{OFI} = \Delta q^{\text{bid}} - \Delta q^{\text{ask}}$ is a signed top-of-book pressure signal; trade imbalance normalises signed tape volume into $[-1, 1]$.
- ▶ The square-root law $\sigma\sqrt{Q/V}$ is the most robust empirical regularity in market impact: doubling size multiplies impact by $\sqrt{2}$, not by 2.
- ▶ Total execution cost = half-spread + impact. For small orders the spread dominates; for large orders the impact dominates.

AFML: Backtesting with Triple Barriers and Purged CV

14.

Companion Code:

```
quant-backtest Code: crates/quant-backtest/  
Dependencies: quant-core, quant-timeseries, quant-portfolio,  
thiserror  
Run: cargo run -p quant-backtest -example triple_barrier  
Run: cargo run -p quant-backtest -example purged_cv  
Run: cargo run -p quant-backtest -example kelly_sizing
```

14.1. Learning Objectives

This chapter leaves behind the frictionless world of portfolio theory and the single-trade world of microstructure, and enters the engine room of systematic strategy research: how do you train a model on time-series labels, cross-validate it without leaking information, and size the resulting bets? After completing it you can:

- ▶ Explain why standard k-fold cross-validation leaks information on financial time series with overlapping events
- ▶ Label a price series with the triple-barrier method (profit-taking, stop-loss, time barrier) and convert the labels to a binary outcome
- ▶ Compute the concurrency of overlapping events and derive sample weights from event uniqueness
- ▶ Generate purged k-fold splits with an embargo period that removes training samples overlapping — and immediately following — each test fold
- ▶ Compute the full Kelly criterion from a win probability and a win/loss ratio, and apply a fractional multiplier for risk control
- ▶ Run the end-to-end AFML pipeline that combines all of the above into a single backtest report

Key Insight

Traditional backtesting fails on three fronts: labels are arbitrary (fixed-horizon return sign), samples overlap (events that span multiple bars share information), and cross-validation leaks (the test fold is not independent of the training fold). The AFML framework fixes all three: triple-barrier labels are path-dependent, sample weights discount overlap, and purged k-fold with embargo removes leakage. The result is an out-of-sample Sharpe that you can actually trust.

14.2. Why Traditional Backtesting Fails

A naive backtest labels each bar by the sign of the return over a fixed horizon h — for example, the 5-bar forward return — and trains a classifier on those labels. Three problems compound:

1. **Arbitrary labels.** A 5-bar forward return says nothing about the path: a position that gained 4%, lost 3%, and gained 4% again is

labelled the same as one that monotonically drifted up. Yet the lived experience of holding the position is very different — the first path probably hit a stop loss.

2. **Overlapping samples.** Consecutive 5-bar labels share four of their five bars. A classifier that treats them as independent over-counts the information in the sample by a factor of $\sim h$, inflating the apparent edge.
3. **Leakage in CV.** Standard k-fold CV assigns bars to folds uniformly at random. A training bar that is part of a 5-bar event ending inside the test fold has already seen the label of that test sample — the validation is no longer out-of-sample.

The AFML framework of López de Prado (2018) addresses each in turn.

14.3. The Triple-Barrier Method

For each entry point t_0 the method places three barriers around the entry price P_{t_0} :

- **Upper** at $P_{t_0}(1 + \theta^+)$, the profit-taking threshold.
- **Lower** at $P_{t_0}(1 + \theta^-)$, the stop-loss threshold ($\theta^- < 0$).
- **Time** at $t_0 + T$, the maximum holding period.

We walk the price path forward from t_0 and label the event by the *first* barrier touched. The holding period is the number of bars from entry to exit; the return is $(P_{\text{exit}} - P_{t_0})/P_{t_0}$.

14.3.1. Configuration and Labels

```

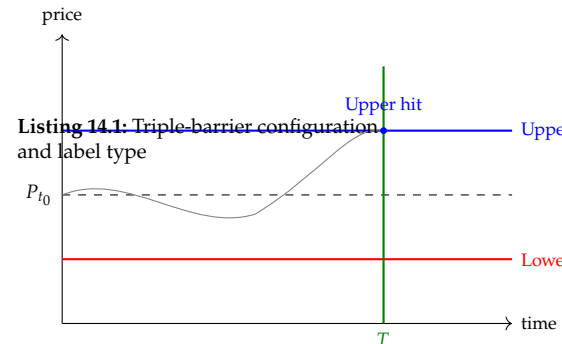
1 pub struct TripleBarrierConfig {
2     pub upper_barrier: f64, // e.g. 0.02 = +2%
3     pub lower_barrier: f64, // e.g. -0.02 = -2%
4     pub time_barrier: usize, // max bars to hold
5     pub min_return: f64, // threshold for Time label
6 }
7
8 pub enum TripleBarrierLabel { Upper, Lower, Time }
9
10 pub struct LabeledEvent {
11     pub entry_index: usize,
12     pub exit_index: usize,
13     pub label: TripleBarrierLabel,
14     pub return_pct: f64,
15     pub holding_period: usize,
16 }
```

14.3.2. Labeling a Single Event

`label_one()` walks prices forward from the entry index, checking at each bar whether the upper or lower barrier has been hit. If neither is touched before the time barrier expires, the event is labelled **Time** with the return at the last bar.

```

1 fn label_one(prices: &[f64], entry_index: usize,
2             config: &TripleBarrierConfig) -> Result<LabeledEvent, BacktestError> {
3     let p_entry = prices[entry_index];
4     let upper_px = p_entry * (1.0 + config.upper_barrier);
5     let lower_px = p_entry * (1.0 + config.lower_barrier);
6     let last = (entry_index + config.time_barrier).min(prices.len() - 1);
7     for (t, &p) in prices.iter().enumerate()
8         .take(last + 1).skip(entry_index + 1) {
9         if p >= upper_px {
10             return Ok(LabeledEvent {
```



Listing 14.1: Triple-barrier configuration and label type

Figure 14.1: Triple barriers around the entry price. The first barrier touched — here the upper barrier at time T — determines the label.

Why three barriers?

A path-dependent label captures the lived experience of the trade. Two trades with the same 5-bar return but different paths get different labels: one hits the upper barrier (a win), the other hits the stop loss (a loss). The label is no longer a function of the endpoint, but of the whole trajectory.

Listing 14.2: Walk forward price path to find the first barrier hit
The triple-barrier method *intentionally* uses future prices to label the event. That is not look-ahead bias in the model: the label is the target, not a feature. Features must be computed strictly from information available at t_0 . The discipline is: train on labels that depend on the future, predict them from features that do not.

```

11     entry_index, exit_index: t,
12     label: TripleBarrierLabel::Upper,
13     return_pct: (p - p_entry) / p_entry,
14     holding_period: t - entry_index,
15     });
16 }
17 if p <= lower_px {
18     return Ok(LabeledEvent {
19         entry_index, exit_index: t,
20         label: TripleBarrierLabel::Lower,
21         return_pct: (p - p_entry) / p_entry,
22         holding_period: t - entry_index,
23     });
24 }
25 }
26 let p_exit = prices[last];
27 Ok(LabeledEvent {
28     entry_index, exit_index: last,
29     label: TripleBarrierLabel::Time,
30     return_pct: (p_exit - p_entry) / p_entry,
31     holding_period: last - entry_index,
32 })
33 }

```

14.3.3. Binary Labels

For a long-only strategy the label is naturally binary: Upper is a win (1), Lower is a loss (0), and Time is a win iff the return at the time barrier exceeds $\theta_{\min}^+$.

```

1 pub fn to_binary_label_helper(label: TripleBarrierLabel,
2                               event: &LabeledEvent,
3                               min_return: f64) -> i32 {
4     match label {
5         TripleBarrierLabel::Upper => 1,
6         TripleBarrierLabel::Lower => 0,
7         TripleBarrierLabel::Time => {
8             if event.return_pct >= min_return { 1 } else { 0 }
9         }
10    }
11 }

```

Listing 14.3: Converting a triple-barrier label to a binary outcome

14.4. Sample Uniqueness and Weighting

When events overlap in time, each carries less unique information. A training sample whose event $[t_0, t_1]$ shares all of its bars with three other events is worth roughly a third of a sample that no other event touches. We capture this with the concurrency count $c_t = \text{number of events active at bar } t$:

$$c_t = \sum_i \mathbb{1}_{[t_0^{(i)}, t_1^{(i)}]}(t). \quad (14.1)$$

The average uniqueness of event i is the mean of $1/c_t$ over its duration, and the sample weight is the inverse of the average concurrency:

$$u_i = \frac{1}{t_1^{(i)} - t_0^{(i)}} \sum_{t=t_0^{(i)}}^{t_1^{(i)}} \frac{1}{c_t}, \quad w_i = \frac{1}{\bar{c}_i}. \quad (14.2)$$

```

1 pub fn concurrent_events(events: &[LabeledEvent], n_bars: usize) -> Vec<usize> {
2     let mut counts = vec![0; n_bars];
3     for ev in events {
4         let start = ev.entry_index.min(n_bars);
5         let end = ev.exit_index.min(n_bars.saturating_sub(1));
6         if start >= n_bars { continue; }

```

Why inverse concurrency?

Listing 14.4: Concurrency, bar counts, and average uniqueness. When events overlap, each carries less unique information there. Summing $1/c_t$ over the duration averages that discount across the event's life. The resulting weight is high for isolated events and low for crowded ones, so the classifier down-weights redundant samples automatically.

```

7   for c in counts.iter_mut().take(end + 1).skip(start) { *c += 1; }
8   }
9   counts
10 }
11
12 pub fn average_uniqueness(events: &[LabeledEvent], n_bars: usize) -> Vec<f64> {
13   let concurrent = concurrent_events(events, n_bars);
14   events.iter().map(|ev| {
15     let start = ev.entry_index;
16     let end = ev.exit_index.min(n_bars.saturating_sub(1));
17     if end < start { return 0.0; }
18     let mut sum_inv = 0.0;
19     let mut count = 0;
20     for &c in concurrent.iter().take(end + 1).skip(start) {
21       if c > 0 { sum_inv += 1.0 / c as f64; count += 1; }
22     }
23     if count == 0 { 0.0 } else { sum_inv / count as f64 }
24   }).collect()
25 }

```

The final weights are the inverse of the average concurrency during each event, normalised to sum to one across the sample:

```

1 pub fn sample_weights(events: &[LabeledEvent], n_bars: usize) -> Vec<f64> {
2   let concurrent = concurrent_events(events, n_bars);
3   let mut weights: Vec<f64> = events.iter().map(|ev| {
4     let start = ev.entry_index;
5     let end = ev.exit_index.min(n_bars.saturating_sub(1));
6     if end < start { return 0.0; }
7     let mut sum_c = 0.0;
8     let mut count = 0;
9     for &c in concurrent.iter().take(end + 1).skip(start) {
10      if c > 0 { sum_c += c as f64; count += 1; }
11    }
12    if count == 0 { 0.0 } else { 1.0 / (sum_c / count as f64) }
13  }).collect();
14  let total: f64 = weights.iter().sum();
15  if total > 0.0 { for w in &mut weights { *w /= total; } }
16  weights
17 }

```

Listing 14.5: Sample weights from inverse concurrency

14.5. Purged K-Fold Cross-Validation

Standard k-fold CV leaks information whenever a training event overlaps a test event. *Purging* removes from the training fold any event whose $[t_0, t_1]$ interval intersects the test fold $[t_{\text{start}}, t_{\text{end}}]$.

```

1 pub fn event_overlaps(event: &LabeledEvent, start: usize, end: usize) -> bool {
2   event.entry_index < end && event.exit_index > start
3 }

```

Listing 14.6: Overlap test for purging

14.5.1. Embargo

Purging is not enough under autocorrelation: a training event *starting* just after the test window ends still carries test-window information through serially-correlated features. The *embargo* removes training events whose entry falls in $[t_{\text{end}}, t_{\text{end}} + e]$.

```

1 pub fn purged_kfold_splits(events: &[LabeledEvent], n_bars: usize,
2   config: &PurgedKfoldConfig) -> Vec<PurgedSplit> {
3   let fold_size = n_bars / config.n_folds;
4   let mut splits = Vec::with_capacity(config.n_folds);
5   for k in 0..config.n_folds {
6     let t_start = k * fold_size;
7     let t_end = if k == config.n_folds - 1 { n_bars }
8       else { (k + 1) * fold_size };
9     let embargo_end = (t_end + config.embargo).min(n_bars);
10    let mut train = Vec::new();
11    let mut test = Vec::new();
12    let mut purged = 0;
13    let mut embargoed = 0;
14    for (i, ev) in events.iter().enumerate() {

```

Train must not see test

If a training event ends inside the test window, its label — which depends on prices inside the test window — has already told the model what happens there. Purging removes exactly those events, leaving the training fold honest.

Listing 14.7: Purged k-fold with embargo

```

15         if ev.entry_index >= t_start && ev.entry_index < t_end {
16             test.push(i); continue;
17         }
18         if event_overlaps(ev, t_start, t_end) { purged += 1; continue; }
19         if ev.entry_index >= t_end && ev.entry_index < embargo_end {
20             embargoed += 1; continue;
21         }
22         train.push(i);
23     }
24     splits.push(PurgedSplit {
25         train_indices: train, test_indices: test,
26         purged_count: purged, embargoed_count: embargoed,
27     });
28 }
29 splits
30 }

```

14.6. Bet Sizing: The Kelly Criterion

A good label tells you *when* to bet; it does not tell you *how much*. The Kelly criterion maximises the expected log growth of capital. For a binary bet with win probability p , loss probability $q = 1 - p$, and win/loss ratio $b = \bar{r}^+ / |\bar{r}^-|$ (mean win divided by mean loss), the optimal fraction of capital to risk on each bet is

$$f^* = \frac{pb - q}{b} = p - \frac{q}{b}. \quad (14.3)$$

The criterion is exact for a sequence of i.i.d. bets with known parameters. In practice the parameters are estimated, so *fractional Kelly* — using a fraction $\lambda \in (0, 1]$ of f^* — is standard. Half Kelly ($\lambda = 0.5$) achieves most of the growth with roughly half the variance.

```

1 pub fn kelly_fraction(win_prob: f64, win_loss_ratio: f64) -> f64 {
2     let q = 1.0 - win_prob;
3     win_prob - q / win_loss_ratio
4 }
5
6 pub fn fractional_kelly(win_prob: f64, win_loss_ratio: f64, fraction: f64) -> f64 {
7     kelly_fraction(win_prob, win_loss_ratio) * fraction
8 }

```

To estimate Kelly from historical trade returns, compute the win probability as the fraction of positive trades and the win/loss ratio as the mean win divided by the mean loss:

```

1 pub fn kelly_from_returns(returns: &[f64]) -> f64 {
2     let n = returns.len() as f64;
3     let wins: Vec<f64> = returns.iter().copied().filter(|&r| r > 0.0).collect();
4     let losses: Vec<f64> = returns.iter().copied().filter(|&r| r < 0.0).collect();
5     let p = wins.len() as f64 / n;
6     let mean_win = wins.iter().sum::<f64>() / wins.len().max(1) as f64;
7     let mean_loss = -losses.iter().sum::<f64>() / losses.len().max(1) as f64;
8     if mean_loss <= 0.0 { return 0.0; }
9     kelly_fraction(p, mean_win / mean_loss)
10 }

```

14.6.1. Position Sizing

The `compute_position_size()` helper packages full and half Kelly along with the empirical win probability and win/loss ratio so the backtest can switch sizing rules without recomputing statistics.

```

1 pub struct PositionSize {
2     pub kelly_full: f64,
3     pub kelly_half: f64,
4     pub win_probability: f64,

```

Embargo calibrates autocorrelation

Set the embargo to the lag at which the autocorrelation function of the features drops below significance. For daily returns on liquid equities a 1–5 bar embargo is typical; for noisy high-frequency features a longer embargo is needed. An embargo of zero reduces to plain purging.

Kelly in one line

~~Listing 14.8: Kelly criterion and trade~~
~~fraction~~ f^* is the edge divided by the odds. With ~~no edge~~ $q = 0.5$, $b = 1$) it is zero — you bet nothing. With a 60% win rate and 1:1 payoff it is $0.6 - 0.4 = 0.2$: risk 20% of capital per bet. Beyond that, the variance of log-wealth grows faster than its mean.

Listing 14.9: Kelly from a series of trade returns

Listing 14.10: Position sizing summary

```

5   pub win_loss_ratio: f64,
6 }
7
8 pub fn compute_position_size(trade_returns: &[f64]) -> PositionSize {
9   let n = trade_returns.len() as f64;
10  let wins: Vec<f64> = trade_returns.iter().copied().filter(|&r| r > 0.0).collect();
11  let losses: Vec<f64> = trade_returns.iter().copied().filter(|&r| r < 0.0).collect();
12  let p = wins.len() as f64 / n.max(1) as f64;
13  let mean_win = wins.iter().sum::<f64>() / wins.len().max(1) as f64;
14  let mean_loss = -losses.iter().sum::<f64>() / losses.len().max(1) as f64;
15  let b = if mean_loss > 0.0 { mean_win / mean_loss } else { 0.0 };
16  let kelly = kelly_fraction(p, b);
17  PositionSize {
18    kelly_full: kelly,
19    kelly_half: 0.5 * kelly,
20    win_probability: p,
21    win_loss_ratio: b,
22  }
23 }

```

14.7. The AFML Pipeline

The full pipeline ties the pieces together. Given a price series and an entry step, it:

1. Generates entry indices at every `entry_step` bars.
2. Labels each entry with the triple-barrier method.
3. Computes sample weights from event uniqueness.
4. Builds purged k-fold splits with embargo.
5. Computes per-trade returns, the in-sample Sharpe, and the mean out-of-sample Sharpe across folds.
6. Sizes each trade by the chosen `BetSizing()` rule — equal, full Kelly, half Kelly, or fixed fraction.
7. Walks the trade returns through a compounded equity curve to compute total return and maximum drawdown.

```

1 pub enum BetSizing { Equal, KellyFull, KellyHalf, Fixed(f64) }
2
3 pub struct AfmlBacktestConfig {
4   pub barrier_config: TripleBarrierConfig,
5   pub cv_config: PurgedKFoldConfig,
6   pub use_weights: bool,
7   pub bet_sizing: BetSizing,
8   pub entry_step: usize,
9 }
10
11 pub struct AfmlBacktestResult {
12   pub events: Vec<LabeledEvent>,
13   pub weights: Vec<f64>,
14   pub cv_splits: Vec<PurgedSplit>,
15   pub in_sample_sharpe: f64,
16   pub out_of_sample_sharpe: f64,
17   pub position_size: PositionSize,
18   pub total_return: f64,
19   pub max_drawdown: f64,
20 }
21
22 pub fn afml_backtest(prices: &[f64],
23                     config: &AfmlBacktestConfig)
24   -> Result<AfmlBacktestResult, BacktestError>
25 {
26   config.barrier_config.validate()?;
27   // ... generate entries, label, weight, split, size, evaluate ...
28   Ok(AfmlBacktestResult { ... })
29 }

```

Listing 14.11: End-to-end AFML backtest

In-sample vs out-of-sample

The in-sample Sharpe is computed on all trade returns. The out-of-sample Sharpe is the mean of the per-fold Sharpe ratios on each test set. A large gap between the two is the symptom of overfitting that purged CV is designed to expose.

14.8. Meta-Labeling (Brief)

A primary model generates a direction signal (long/short/flat); a secondary model — the meta-model — predicts whether the primary signal will be correct. The triple-barrier label of the primary model becomes the target of the meta-model. This separates *direction* from *size*: the primary model bets often, the meta-model decides how much to trust each bet. The result is a betting strategy that is more selective than the primary alone and, empirically, has a higher Sharpe.

14.9. Exercises

1. **Custom barriers.** Generalise the triple-barrier method so that the upper and lower barriers are functions of rolling volatility rather than fixed percentages. How does the label distribution change on a noisy versus a calm series?
2. **Embargo calibration.** Compute the autocorrelation function of fractional-differentiated returns (see Chapter 7) and set the embargo to the first lag where the ACF drops below significance. Compare the out-of-sample Sharpe with and without the embargo.
3. **Volatility-scaled Kelly.** Replace the constant Kelly fraction with one that scales inversely with rolling volatility. Verify that the drawdown distribution tightens.
4. **Meta-labeling.** Use the sign of a moving-average crossover as the primary signal, label it with the triple barrier, and train a logistic regression on the labelled events to predict correctness. Compare the meta-filtered strategy to the primary alone.

14.10. Key Takeaways

- ▶ **Triple-barrier labeling** converts price paths into supervised learning targets by setting profit-taking, stop-loss, and time barriers. It avoids the look-ahead bias of fixed-horizon labeling.
- ▶ **Sample weights from uniqueness** down-weight events that overlap heavily with others, reducing overfitting to clustered periods. The average uniqueness metric quantifies temporal independence.
- ▶ **Purged k-fold cross-validation** prevents information leakage by removing overlapping events from training folds and applying an embargo period. It exposes overfitting that standard k-fold misses in time-series data.
- ▶ **Walk-forward analysis** validates strategy robustness by measuring performance degradation from in-sample to out-of-sample. Walk-Forward Efficiency (WFE) below 0.5 indicates severe overfitting (see Section 5.11 in Chapter 5).
- ▶ **Kelly criterion bet sizing** maximizes long-run geometric growth by sizing positions proportional to edge and inversely to volatility. Fractional Kelly trades growth for reduced drawdowns.
- ▶ **Meta-labeling** separates direction (primary model) from confidence (meta-model), producing more selective strategies with higher Sharpe ratios. The triple-barrier label of the primary becomes the target of the meta-model.
- ▶ **Trait-based architecture** enables composition of labeling, cross-validation, and bet sizing strategies without code duplication. The

generic backtest engine combines any `Labeler`, `CrossValidator`, and `BetSizer` via trait bounds.

- ▶ **AFML workflow integration** orchestrates the full pipeline: label events → weight samples → cross-validate → size bets → evaluate. Each component is swappable via trait implementations, promoting experimentation and reuse.

14.11. References

- ▶ López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley. Chapters 3 (triple-barrier), 4 (sample weights), 7 (cross-validation), 10 (bet sizing).
- ▶ Kelly, J. L. (1956). "A New Interpretation of Information Rate." *Bell System Technical Journal*.
- ▶ `quant-lab-rust` companion code, `crate quant-backtest`.

Companion Code:

```
quant-lib Code: crates/quant-lib/  
Dependencies: all source crates (re-exported)  
Run: cargo run -p quant-lib -example 15_full_pipeline  
Run: cargo run -p quant-lib -example 16_trait_compose
```

15.1. Learning Objectives

This chapter consolidates the ecosystem built in Chapters 6–14 into a single facade crate. After completing it you can:

- ▶ Understand the facade pattern as the final consolidation step
- ▶ Use the module structure of `quant-lib` and its re-exports of the underlying crates
- ▶ Use the prelude for ergonomic one-line imports
- ▶ Compose pluggable components via the `BacktestBuilder` fluent API
- ▶ Run the end-to-end pipeline that ties five crates together

15.2. Introduction: From Crates to a Library

Chapters 1 through 14 built nine specialised crates: `quant-core`, `quant-timeseries`, `quant-vol`, `quant-stochastic`, `quant-options`, `quant-portfolio`, `quant-factors`, `quant-microstructure`, and `quant-backtest`. Each crate is independently useful, but a downstream user who wants the whole ecosystem would rather depend on one crate than nine. `quant-lib` is that one crate.

Key Insight

`quant-lib` is purely a re-export facade. It adds *zero* new logic. Every line in its source is `pub mod` or `pub use`. The math, types, and traits all live in the underlying crates. The facade exists to give downstream projects a single dependency and a unified namespace.

15.3. The Facade Pattern

A *facade* is a structural pattern: a single object (here, a crate) that provides a simplified, unified interface to a larger body of code. The facade does not implement new behaviour; it delegates.

In Rust, the facade crate is structurally tiny:

```
1 // crates/quant-lib/src/lib.rs (excerpt)  
2 pub mod prelude;  
3  
4 #[cfg(feature = "core")]  
5 pub mod core {  
6     pub use quant_core::*;
```



```

7 }
8
9 #[cfg(feature = "options")]
10 pub mod options {
11     pub use quant_options::*;
12 }
13
14 #[cfg(feature = "backtest")]
15 pub mod backtest {
16     pub use quant_backtest::*;
17 }
18
19 // ... other modules ...

```

Each module is a thin re-export. Downstream code can write either `use quant_core::mean` or `use quant_lib::core::mean`— both resolve to the same function. The facade adds a layer of indirection that costs nothing at runtime (everything is monomorphised away) but gives the user a single dependency to track.

15.4. Module Map

`quant-lib` organises the ecosystem into ten modules, one per underlying crate plus a dedicated traits module for the pluggable component trait catalogue.

Module	Source crate	Contents
<code>core()</code>	<code>quant-core</code>	moments, rolling windows, RNG, GBM
<code>traits()</code>	<code>quant-core</code>	Pluggable component traits
<code>risk()</code>	<code>quant-core</code>	PSR, DSR, Calmar, Omega, Ulcer
<code>timeseries()</code>	<code>quant-timeseries</code>	OLS, ACF, ADF, frac-diff, CUSUM
<code>vol()</code>	<code>quant-vol</code>	EWMA, ARCH, GARCH
<code>stochastic()</code>	<code>quant-stochastic</code>	Brownian, GBM, Monte Carlo
<code>options()</code>	<code>quant-options</code>	Black-Scholes, Greeks, IV
<code>portfolio()</code>	<code>quant-portfolio</code>	Markowitz, tangency, CAPM, VaR
<code>factors()</code>	<code>quant-factors</code>	PCA, Fama-French, attribution
<code>microstructure()</code>	<code>quant-microstructure</code>	LOB, OFI, impact, VWAP
<code>backtest()</code>	<code>quant-backtest</code>	triple-barrier, purged CV, Kelly

15.5. The Prelude

The prelude pulls in the most commonly used items from every module so that user code can start with a single import:

```

1 use quant_lib::prelude::*;

```

The prelude is intentionally curated. It includes:

- ▶ Core utilities: `mean()`, `std_dev()`, `variance()`, `RollingWindow()`, `XorShift64()`, `PriceSeries()`.
- ▶ Pluggable component traits: `CrossValidator()`, `Labeler()`, `BetSizer()`, `StochasticProcess()`, `OptionPricer()`, `Greeks()`, `FactorModel()`, `ImpactModel()`, `OrderBookOps()`, `SampleWeighter()`, `StructuralBreakDetector()`.
- ▶ Risk metrics: `probabilistic_sharpe_ratio()`, `deflated_sharpe_ratio()`, `calmar_ratio()`, `omega_ratio()`, `information_ratio()`, `ulcer_index()`.
- ▶ Time series: `ols()`, `adf_test()`, `fracDiff()`, `OlsFit()`.
- ▶ Volatility: `ewma_vol()`, `ArchModel()`, `GarchModel()`.

- Stochastic: `gbm()`, `brownian_motion()`, `bs_call()`, `bs_put()`, `mc_call()`, `mc_put()`.
- Options: `delta()`, `gamma()`, `vega()`, `theta()`, `implied_vol()`.
- Portfolio: `Portfolio()`, `efficient_frontier_point()`, `min_variance_portfolio()`, `tangency_portfolio()`, `sharpe_ratio()`.
- Factors: `pca()`, `ff3_regression()`, `risk_attribution()`, `PcaResult()`, `FF3Exposure()`.
- Microstructure: `OrderBook()`, `Side()`, `vwap()`, `sqrt_impact()`, `execution_cost()`.
- Backtest: `GenericBacktest()`, `BacktestBuilder()`, `TripleBarrierLabel()`, `TripleBarrierConfig()`, `FixedHorizonLabeler()`, `KellyBetSizer()`, `WalkForward()`, `WalkForwardConfig()`, `PurgedKFoldConfig()`, `triple_barrier_label()`, `purged_kfold_splits()`, `sample_weights()`, `kelly_fraction()`, `walk_forward_efficiency()`, `afml_backtest()`.

Items not in the prelude are still reachable through the module path, e.g., `quant-lib::backtest::EqualBetSizer`. The prelude is a convenience, not a gate.

15.6. Feature Flags for Module Gating

The `quant-lib` `Cargo.toml` exposes each module behind a feature flag so users pay only for what they compile:

```
1 [features]
2 default = ["all"]
3 all = ["core", "timeseries", "vol", "stochastic",
4       "options", "portfolio", "factors",
5       "microstructure", "backtest"]
```

A user who needs only option pricing can disable defaults and pick two modules:

```
1 [dependencies]
2 quant-lib = { version = "0.1", default-features = false, features = ["core", "options"] }
```

The rest of the crate is not compiled. In a CI pipeline that builds only the pricing surface, this shaves minutes off the build.

15.7. Root-Level Convenience Re-exports

Beyond the module re-exports, `quant-lib` also exposes a small set of root-level convenience functions so that the most common operations do not require a module prefix:

```
1 // These three are equivalent:
2 quant-lib::core::mean(&data);
3 quant-lib::mean(&data);           // root re-export
4 quant_core::mean(&data);          // direct from source
5
6 // Same for stochastic, options, portfolio, backtest:
7 quant-lib::bs_call(s, k, r, sigma, t);
8 quant-lib::delta(s, k, r, sigma, t);
9 quant-lib::tangency_portfolio(&cov, rf);
10 quant-lib::kelly_fraction(p, b, a);
```

The root re-exports exist because these functions account for the overwhelming majority of calls in examples and tests. Putting them at the root saves typing without obscuring the module structure.

15.8. Composable Backtest Architecture

The headline feature of the trait architecture is the `GenericBacktest()` engine, generic over three pluggable components:

```
1 pub struct GenericBacktest<L, CV, BS>
2 where
3     L: Labeler,
4     CV: CrossValidator,
5     BS: BetSizer,
6 {
7     labeler: L,
8     validator: CV,
9     sizer: BS,
10    entry_step: usize,
11 }
```

The bound where `L: Labeler`, `CV: CrossValidator`, `BS: BetSizer` is the entire contract. Adding a new labeler means writing an `impl Labeler` for `MyLabeler`—no engine changes, no new features, no feature flags.

15.8.1. The Builder API

The `BacktestBuilder()` gives a fluent API for composition:

```
1 let bt = BacktestBuilder::new()
2     .labeler(FixedHorizonLabeler::new(5, 0.01))
3     .cv(WalkForward::new(WalkForwardConfig::rolling(100, 25, 25)))
4     .sizer(KellyBetSizer::new(0.5)) // half Kelly
5     .entry_step(5)
6     .build();
7
8 let results = bt.run(&prices, &returns?);
```

Swapping one component is a one-line change. Example 16 in the companion code runs the same strategy with four different bet sizers (full Kelly, half Kelly, fixed 10%, equal 100%) and two different validators (rolling vs. anchored walk-forward) by changing one argument in the builder call.

15.8.2. What the Builder Checks

The builder rejects incomplete configurations at *build time*, returning `BacktestError()`:

- ▶ Missing labeler: `BacktestError::MissingLabeler`.
- ▶ Missing cross-validator: `BacktestError::MissingCrossValidator`.
- ▶ Missing bet sizer: `BacktestError::MissingBetSizer`.
- ▶ Entry step out of range: `BacktestError::InvalidEntryStep`.

This means misconfigurations are caught before any data is loaded, not in the middle of a 252-day backtest.

15.9. The End-to-End Pipeline

Example 15 in the companion code ties together five crates in a single pipeline:

```

1 use quant_lib::prelude::*;
2 use quant_lib::backtest::{compute_position_size, kelly_from_returns};
3 use quant_lib::core::simple_returns;
4
5 // 1. Simulate a GBM price path (quant-stochastic).
6 let mut rng = XorShift64::new(123);
7 let prices = gbm(100.0, 0.08, 0.20, 1.0, 252, &mut rng);
8
9 // 2. Price an ATM call with the trait-based BlackScholes pricer.
10 let pricer = quant_options::BlackScholes::new(0.05, 0.20);
11 let call = pricer.price(100.0, 100.0, 1.0, OptionType::Call?);
12
13 // 3. Analytical Greeks via the Greeks trait.
14 let d = pricer.delta(100.0, 100.0, 1.0, OptionType::Call?);
15 let g = pricer.gamma(100.0, 100.0, 1.0?);
16 let v = pricer.vega(100.0, 100.0, 1.0?);
17 let th = pricer.theta(100.0, 100.0, 1.0, OptionType::Call?);
18
19 // 4. Kelly fraction from synthetic trade returns.
20 let rets = simple_returns(&prices);
21 let f_kelly = kelly_from_returns(&rets);
22
23 // 5. Walk-forward cross-validation on the simulated path.
24 let wf = WalkForward::new(WalkForwardConfig::rolling(120, 30, 30));
25 let splits = wf.splits(0, prices.len());
26
27 // 6. Run the generic composable backtest.
28 let bt = GenericBacktest::new(
29     FixedHorizonLabeler::new(5, 0.01),
30     WalkForward::new(WalkForwardConfig::rolling(100, 25, 25)),
31     KellyBetSizer::new(0.5),
32     5,
33 );
34 let results = bt.run(&prices, &bt_returns?);

```

Companion Code:

quant-lib See `crates/quant-lib/examples/15_full_pipeline.rs` and `crates/quant-lib/examples/16_trait_compose.rs`. Run with `cargo run -p quant-lib -example 15_full_pipeline`.

This is the “hello world” of the facade: every step uses a different sub-crate, all reached through `quant-lib`. The pipeline demonstrates that the trait-based architecture composes cleanly—the `BlackScholes()` pricer implements `OptionPricer()` and `Greeks()`; the `FixedHorizonLabeler()` implements `Labeler()`; the `WalkForward()` validator implements `CrossValidator()`; the `KellyBetSizer()` implements `BetSizer()`. The `GenericBacktest()` engine stitches them together without knowing any of their concrete types.

15.10. The 16-Example Tour

`quant-lib` ships 16 numbered examples that progress from single-crate usage to multi-crate composition:

#	Example	Crates exercised
01	mean_variance	core
02	returns_basics	core
03	rolling_window	core
04	random_walk	core, stochastic
05	ols_regression	core, timeseries
06	stationarity	timeseries
07	volatility	vol
08	options_greeks	options
09	portfolio_frontier	portfolio
10	pca_factors	factors
11	microstructure_lob	microstructure
12	triple_barrier	backtest
13	purged_cv	backtest
14	kelly_sizing	backtest
15	full_pipeline	stochastic, options, backtest
16	trait_compose	backtest (builder pattern)

Examples 1–14 each exercise a single sub-crate. Example 15 is the multi-crate pipeline. Example 16 is the composable backtest with four bet sizers and two validators. The progression is deliberate: each example introduces one new concept, so a reader working through them in order never encounters a dependency they have not already seen.

15.11. Non-Destructive by Design

`quant-lib` is a strict facade. The source crates continue to work without `quant-lib` in their dependency tree:

```
1 # quant-backtest still builds without quant-lib:
2 cargo test -p quant-backtest
```

The 20-test TDD contract for `quant-lib` (Appendix B lists the full suite) includes a test that verifies this property: the source crates must pass independently. This guarantees that the facade remains a convenience, not a coupling mechanism. A downstream project that depends on `quant-backtest` directly, without `quant-lib`, is not broken when `quant-lib` adds a new re-export.

Key Insight

The facade is a one-way dependency: `quant-lib` depends on the source crates, never the reverse. If you find yourself writing `use quant_lib` inside a source crate, stop—you are creating a cycle. The source crates must remain oblivious to the facade’s existence.

15.12. Versioning Strategy

When a source crate bumps its version (e.g., `quant-options` from 0.1 to 0.2), `quant-lib` bumps its own version and updates the `Cargo.toml` dependency. Downstream code that uses `quant-lib` sees a single version bump, not nine. This is the main practical benefit of the facade for consumers: the dependency graph has one node, not nine.

15.13. When to Use `quant-lib` vs. Direct Crates

Use	When
<code>quant-lib</code>	You want the whole ecosystem, one dependency, and unified imports.
<code>quant-core</code>	You need only moments, rolling windows, or the trait catalogue.
<code>quant-options</code>	You are building a pricing tool and do not need back-testing.
<code>quant-backtest</code>	You are building a research tool and do not need pricing.
individual crates	You want minimum compile time and a small dependency tree.

There is no wrong answer. The facade and the source crates are both first-class supported surfaces. The choice is about ergonomics versus minimality.

15.14. Exercises

1. **Module discovery.** List the public items re-exported by `quant-lib::portfolio`. How many of them are types, how many are functions, and how many are traits?
2. **Prelude contents.** The prelude does not include `EqualBetSizer()` or `FixedBetSizer()`. Why? Add them to your own local prelude and rebuild—do any examples break?
3. **Feature flags.** Build `quant-lib` with `default-features = false`, `features = ["core", "options"]` and run example 08. Which modules are unavailable? What is the compile-time difference?
4. **Builder swap.** Modify example 16 to use a `TripleBarrierLabeler()` instead of `FixedHorizonLabeler()`. What single line do you change? Does the rest of the pipeline still compile?
5. **Pipeline extension.** Extend example 15 to compute the implied volatility from the simulated option price using `implied_vol()`. How many lines do you add? Do you need to change any existing line?
6. **Root re-exports.** Identify three functions in the root re-export list that you would remove if you were designing the facade for a minimal API. Justify each choice.
7. **Non-destructive check.** Run `cargo test -p quant-backtest` without `quant-lib` in the workspace. Confirm that all tests still pass. This verifies the non-destructive property.
8. **Cycle detection.** Search the source crates for any `use quant-lib` statement. If you find one, it is a bug. Report the file and line.

15.15. Summary

`quant-lib` is the unified library: a single crate that re-exports nine specialised crates under a unified namespace. It adds no new logic, depends on every source crate, and exposes a curated prelude for ergonomic imports. The `GenericBacktest()` engine, built on the pluggable trait

catalogue, makes the library composable: swapping a labeler, validator, or bet sizer is a one-line change via the `BacktestBuilder()` fluent API. The 16 examples progress from single-crate usage to multi-crate pipelines, giving the reader a guided tour of the ecosystem. The facade is non-destructive: every source crate continues to work without it, and the 20-test contract includes a test that verifies this property.

APPENDICES

Mathematical Notation

A.

This appendix consolidates the notation used throughout the book. Each chapter introduces symbols locally; here we gather them in one place for quick reference. Where a symbol has multiple meanings depending on context, the chapter in which each meaning is established is cited.

A.1. General Conventions

- Scalars are set in italics: x, y, r, σ .
- Vectors are bold lowercase: $\mathbf{w}, \mathbf{x}, \boldsymbol{\mu}$.
- Matrices are bold uppercase: $\boldsymbol{\Sigma}, \mathbf{X}, \mathbf{V}$.
- Random variables are uppercase: X, R, S ; their realisations lowercase: x, r, s .
- Time is indexed by t ; a series of length T runs $t = 1, \dots, T$.
- Lags are written with subscripts: r_{t-k} is the value k periods before t .
- Estimators wear a hat: $\hat{\mu}, \hat{\sigma}, \hat{\beta}$.
- The indicator function $\mathbb{1}[A]$ is 1 if condition A holds, 0 otherwise.

A.2. Probability and Statistics

Symbol	Meaning
$\mathbb{E}[X]$	Expectation of X
$\text{Var}(X)$	Variance of X
$\text{Cov}(X, Y)$	Covariance of X and Y
$\text{Corr}(X, Y)$	Pearson correlation
$X \sim \mathcal{N}(\mu, \sigma^2)$	X is Gaussian with mean μ , variance σ^2
$\text{argmin}_x f(x)$	The x that minimises f
$\text{argmax}_x f(x)$	The x that maximises f

A.2.1. Moments

The first four central moments of a sample $\{x_1, \dots, x_n\}$ with sample mean $\bar{x}$ are:

$$\hat{\mu}_k = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^k \quad (\text{A.1})$$

From these we obtain the dimensionless shape measures:

$$\text{Skew} = \hat{\mu}_3 / \hat{\mu}_2^{3/2} \quad (\text{A.2})$$

$$\text{Kurt} = \hat{\mu}_4 / \hat{\mu}_2^2 \quad (\text{A.3})$$

$$\text{ExcessKurt} = \text{Kurt} - 3 \quad (\text{A.4})$$

The normal distribution has $\text{Skew} = 0$ and $\text{ExcessKurt} = 0$; financial returns typically show $\text{ExcessKurt} > 0$ (fat tails, Ch. 6).

A.3. Returns

Let P_t be the price at time t .

Symbol	Definition	Use
r_t (simple)	$\frac{P_t - P_{t-1}}{P_{t-1}}$	P&L attribution
z_t (log)	$\ln\left(\frac{P_t}{P_{t-1}}\right)$	Statistical modelling
μ	$\mathbb{E}[r_t]$ (annualised)	Expected return
σ	$\sqrt{\text{Var}(r_t)}$ (annualised)	Volatility

Annualisation

Daily to annual: multiply mean by 252, standard deviation by $\sqrt{252}$. Monthly to annual: multiply mean by 12, standard deviation by $\sqrt{12}$.

Log returns are preferred for statistical work because they are additive over time:

$$z_{t:n} = \ln \frac{P_t}{P_{t-n}} = \sum_{k=0}^{n-1} \ln \frac{P_{t-k}}{P_{t-k-1}} = \sum_{k=0}^{n-1} z_{t-k}. \quad (\text{A.5})$$

Simple returns are preferred for portfolio P&L because they aggregate across assets:

$$r_p = \sum_{i=1}^N w_i r_i. \quad (\text{A.6})$$

A.4. Risk-Adjusted Performance

Metric	Formula
Sharpe	$\frac{\mathbb{E}[r_p] - r_f}{\sigma_p}$
Sortino	$\frac{\mathbb{E}[r_p] - r_f}{\sigma_{\text{downside}}}$
MaxDD	$\max_t (\max_{s \leq t} V_s - V_t)$
Calmar	$\frac{\text{annual return}}{\text{MaxDD}}$

Here V_t is the equity curve, r_f the risk-free rate, and σ_{downside} the standard deviation of returns below a target (typically zero or r_f).

A.5. Time Series

Symbol	Meaning
$\rho_k = \text{Corr}(r_t, r_{t-k})$	Autocorrelation at lag k (ACF)
π_k	Partial autocorrelation at lag k (PACF)
ADF	Augmented Dickey-Fuller test statistic
d	Differencing order (fractional allowed, Ch. 7)

A series is *stationary* if its mean and autocovariance do not depend on t . The ADF test rejects the unit-root null when the statistic is more negative than a critical value (-2.86 at 5%).

A.6. Volatility Models

For ARCH(p) and GARCH(p, q) (Ch. 8):

$$\sigma_t^2 = \omega + \sum_{i=1}^p \alpha_i r_{t-i}^2 \quad (\text{ARCH}) \quad (\text{A.7})$$

$$\sigma_t^2 = \omega + \sum_{i=1}^p \alpha_i r_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2 \quad (\text{GARCH}) \quad (\text{A.8})$$

- **Persistence:** $\sum \alpha_i + \sum \beta_j$; values < 1 give stationarity.
- **EWMA (RiskMetrics):** $\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$, with $\lambda \approx 0.94$.
- **Long-run variance:** $\sigma_\infty^2 = \omega / (1 - \sum \alpha_i - \sum \beta_j)$.

A.7. Stochastic Processes

For geometric Brownian motion (Ch. 9):

$$\frac{dS_t}{S_t} = \mu dt + \sigma dW_t, \quad S_t = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right). \quad (\text{A.9})$$

Here W_t is a standard Wiener process with $W_0 = 0$, $\text{Cov}(W_s, W_t) = \min(s, t)$.

A.8. Options (Black-Scholes)

Symbol	Meaning
S	Spot price
K	Strike price
r	Risk-free rate (continuously compounded)
σ	Volatility (annualised)
T	Time to maturity (years)
d_1, d_2	$\frac{\ln(S/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$
$N(\cdot)$	Standard normal CDF

A.8.1. Pricing

$$\text{Call} = S N(d_1) - K e^{-rT} N(d_2) \quad (\text{A.10})$$

$$\text{Put} = K e^{-rT} N(-d_2) - S N(-d_1) \quad (\text{A.11})$$

A.8.2. Greeks

Greek	Call	Put
Δ	$N(d_1)$	$N(d_1) - 1$
Γ	$N'(d_1)/(S\sigma\sqrt{T})$	same
v (vega)	$SN'(d_1)\sqrt{T}$	same
Θ	$-\frac{SN'(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)$	$-\frac{SN'(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)$
ρ	$KT e^{-rT}N(d_2)$	$-KT e^{-rT}N(-d_2)$

A.9. Portfolio Theory

- Weights $\mathbf{w} = (w_1, \dots, w_N)^\top$ with $\sum w_i = 1$.
- Expected returns $\boldsymbol{\mu} \in \mathbb{R}^N$.
- Covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{N \times N}$.
- Portfolio return $\mu_p = \mathbf{w}^\top \boldsymbol{\mu}$.
- Portfolio volatility $\sigma_p = \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}$.

A.9.1. Frontier

Minimum variance $\operatorname{argmin}_{\mathbf{w}} \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ subject to $\mathbf{1}^\top \mathbf{w} = 1$.

Efficient frontier Add constraint $\mathbf{w}^\top \boldsymbol{\mu} = \mu_{\text{target}}$.

Tangency (max Sharpe) $\operatorname{argmax}_{\mathbf{w}} \frac{\mathbf{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}}$.

CML $\mu_p = r_f + \text{Sharpe}_{\text{tan}} \cdot \sigma_p$.

Two-fund separation $y = \sigma_p / \sigma_{\text{tan}}$ in tangency, $1 - y$ in risk-free.

A.10. Factor Models

For a factor model with K factors (Ch. 12):

$$r_i = \alpha_i + \sum_{k=1}^K \beta_{i,k} f_k + \varepsilon_i, \quad \operatorname{Cov}(f_k, \varepsilon_i) = 0. \quad (\text{A.12})$$

- **CAPM** ($K = 1$): single market factor $f = r_m - r_f$.
- **Fama-French 3-factor** ($K = 3$): market, size (SMB), value (HML).
- **PCA**: factors are eigenvectors of the covariance matrix; the k -th factor explains $\lambda_k / \sum_j \lambda_j$ of total variance.

A.11. Market Microstructure

Symbol	Meaning
P_b, P_a	Best bid, best ask
spread	$P_a - P_b$
mid	$(P_a + P_b)/2$
OFI_t	Order flow imbalance at time t (Ch. 13)
VWAP	Volume-weighted average price

A.11.1. Market Impact (Almgren-Chriss)

$$\text{Impact} = \sigma \cdot \sqrt{\frac{Q}{\text{ADV}}} \quad (\text{A.13})$$

where Q is the order size, σ the daily volatility, and ADV the average daily volume. The execution cost decomposes as $\text{total} = \text{spread} + \text{impact}$, all in absolute return units (multiply by 10^4 for basis points).

A.12. AFML Backtesting

- ▶ **Triple barrier** (Ch. 14): take-profit ϕ , stop-loss ψ , time barrier τ . Label is $+1, -1, 0$ by which barrier hits first.
- ▶ **Purged k -fold**: each test fold is purged of samples whose label window overlaps the training fold; an embargo period ρ prevents leakage near fold boundaries.
- ▶ **Kelly**: $f^* = \frac{\mu}{\sigma^2}$ for a single asset; fractional Kelly uses $f = c f^*$ with $c \in (0, 1]$.
- ▶ **WFE** (Walk-Forward Efficiency): $\text{Sharpe}_{\text{oos}} / \text{Sharpe}_{\text{is}}$.

A.13. Common Subscripts

Subscript	Meaning
t	Time index
i, j	Asset index
k	Lag or factor index
p	Portfolio
f	Risk-free
m	Market
∞	Long-run / steady state

This appendix documents the Rust patterns that recur across the `quant - lab` crates. Each pattern is illustrated with a minimal example drawn from the actual codebase. The goal is to give the reader a vocabulary for the design decisions made in the companion code and a mental model for extending it.

B.1. The Trait as Extension Point

The single most important pattern in this book is the *trait as extension point*. The library defines a trait; the user implements it for their specific model; the generic engine drives the computation. This is how every pluggable component in `quant - lab` works.

```
1 // quant-core/src/traits/labeled.rs
2 pub trait Labeled {
3     type Input;
4     type Label;
5
6     fn label(&self, input: &Self::Input) -> Vec<Self::Label>;
7 }
```

A concrete implementation—the triple-barrier labeler from `quant - backtest`—supplies the method body:

```
1 pub struct TripleBarrier {
2     pub take_profit: f64,
3     pub stop_loss: f64,
4     pub time_barrier: usize,
5 }
6
7 impl Labeled for TripleBarrier {
8     type Input = (Vec<f64>, usize); // (prices, entry_index)
9     type Label = i8;               // +1, -1, or 0
10
11     fn label(&self, input: &Self::Input) -> Vec<Self::Label> {
12         // ... barrier-touch logic ...
13     }
14 }
```

Key Insight

The trait defines the contract; the struct implements it; the generic engine is oblivious to the concrete type. This is how Rust achieves polymorphism without dynamic dispatch when performance matters (monomorphisation) and with dynamic dispatch when flexibility matters (dyn Trait).

B.2. Pluggable Component Trait Catalogue

The `quant - core` traits module exposes the cross-cutting interfaces used by the higher-level crates. Each trait corresponds to a pluggable decision in a quantitative pipeline.

Trait	Role
CrossValidator()	Produces train/test index splits (k-fold, walk-forward, purged)
Labeler()	Maps a price series to a label series (triple-barrier, fixed-horizon)
BetSizer()	Converts a signal into a position size (Kelly, equal, fixed)
SampleWeighter()	Assigns per-sample weights for training (return-based, time-decay)
StructuralBreakDetector()	Locates regime-change indices in a series (CUSUM, Z-score)
StochasticProcess()	Simulates a path ($S_0, \dots, S_T$) (GBM, Brownian)
OptionPricer()	Prices a vanilla option (Black-Scholes, Monte Carlo)
Greeks()	Computes risk sensitivities ($\Delta, \Gamma, \nu, \Theta, \rho$)
FactorModel()	Maps returns to factor exposures (PCA, Fama-French)
ImpactModel()	Estimates market impact of an order (square-root, linear)
OrderBookOps()	Order book operations (add, cancel, match, depth)

Every trait in this table is used by at least one concrete struct in the companion crate, and every concrete struct is used in at least one example in `quant-lib`.

B.3. Generic Engines with Static Dispatch

A backtest engine is generic over its labeler, cross-validator, and bet sizer. The user composes these at construction time and the compiler generates a specialised binary—no vtable overhead on the hot path.

```

1 // quant-backtest/src/generic.rs
2 pub struct GenericBacktest<L, CV, BS>
3 where
4     L: Labeler,
5     CV: CrossValidator,
6     BS: BetSizer,
7 {
8     labeler: L,
9     validator: CV,
10    sizer: BS,
11    // ...
12 }
13
14 impl<L, CV, BS> GenericBacktest<L, CV, BS>
15 where
16     L: Labeler,
17     CV: CrossValidator,
18     BS: BetSizer,
19 {
20     pub fn run(&self, prices: &[f64]) -> BacktestResult {
21         let labels = self.labeler.label(&(prices.to_vec(), 0));
22         let splits = self.validator.splits(0, prices.len());
23         // ... drive the loop, calling self.sizer.size(...) per bar ...
24     }
25 }

```

The bound `where L: Labeler, CV: CrossValidator, BS: BetSizer` is the entire contract. Adding a new labeler means writing an `impl Labeler` for `MyLabeler`—no engine changes.

B.4. The Builder Pattern for Composable Backtests

When a struct has many optional parameters, a builder gives a fluent API while keeping the internal representation compact.

```

1 // quant-backtest/src/generic.rs
2 pub struct BacktestBuilder {
3     labeler: Option<Box<dyn Labeler<Input = (Vec<f64>, usize), Label = i8>>>,
4     sizer: Option<Box<dyn BetSizer>>,
5     // ... other pluggable components ...
6 }
7
8 impl BacktestBuilder {
9     pub fn new() -> Self { /* all None */ }
10
11     pub fn with_labeler<L: Labeler + 'static>(mut self, l: L) -> Self {
12         self.labeler = Some(Box::new(l));
13         self
14     }
15
16     pub fn with_sizer<BS: BetSizer + 'static>(mut self, bs: BS) -> Self {
17         self.sizer = Some(Box::new(bs));
18         self
19     }
20
21     pub fn build(self) -> Result<GenericBacktest<...>, BacktestError> {
22         // validate required fields, return Err if any missing
23     }
24 }

```

Usage reads as a pipeline:

```

1 let bt = BacktestBuilder::new()
2     .with_labeler(TripleBarrier::new(0.02, 0.01, 5))
3     .with_sizer(KellySizer::new(0.5))
4     .build()?;

```

Static vs. dynamic

The engine uses static dispatch (generics) for the hot path, but the builder uses `Box<dyn Trait>` for storage convenience. The trade-off is a single vtable call per bar, negligible against the per-bar computation.

B.5. Newtype Wrappers for Domain Safety

A bare `f64` can be a price, a return, a volatility, or a probability—the compiler cannot tell them apart. A newtype wrapper adds zero-cost type safety.

```

1 pub struct Price(f64);
2 pub struct LogReturn(f64);
3 pub struct Volatility(f64);
4
5 impl Price {
6     pub fn log_return(&self, prev: &Price) -> LogReturn {
7         LogReturn((self.0 / prev.0).ln())
8     }
9 }

```

The compiler now refuses to add a `Price` to a `Volatility`, even though both wrap an `f64`. The `quant-lab` crates use newtypes sparingly—mostly in public APIs—to avoid ceremony in internal computation.

B.6. Error Handling with `Result`

Library code never panics. Every fallible operation returns `Result<T, E>`, with the error type implementing `std::error::Error`. The crate-level error is an enum:


```

1 // quant-core/src/error.rs
2 #[derive(Debug, thiserror::Error)]
3 pub enum QuantError {
4     #[error("insufficient data: required {required}, got {actual}")]
5     InsufficientData { required: usize, actual: usize },
6
7     #[error("singular matrix: {0}")]
8     SingularMatrix(String),
9
10    #[error("optimisation failed to converge")]
11    NonConvergence,
12
13    #[error(transparent)]
14    Io(#[from] std::io::Error),
15 }

```

The `InsufficientData` variant appears in the ADF test (too few observations for the lag order), `SingularMatrix` in OLS regression (collinear regressors), and `NonConvergence` in implied-volatility inversion. Callers use `?` to propagate:

```

1 fn sharpe(returns: &[f64], rf: f64) -> Result<f64, QuantError> {
2     let m = mean(returns);
3     let s = std_dev(returns).ok_or(QuantError::InsufficientData {
4         required: 2,
5         actual: returns.len(),
6     })?;
7     Ok((m - rf) / s * 252.0_f64.sqrt())
8 }

```

B.7. Iterators and Lazy Evaluation

Most transformations are expressed as iterator adaptors, avoiding intermediate allocations:

```

1 let mean: f64 = returns.iter().sum::<f64>() / returns.len() as f64;
2 let var: f64 = returns.iter()
3     .map(|r| (r - mean).powi(2))
4     .sum::<f64>() / (returns.len() - 1) as f64;

```

For rolling windows, the `RollingWindow()` trait returns an iterator of slices, letting the caller fold, map, or collect without the library dictating the output shape.

B.8. Feature Flags for Module Gating

The `quant-lib` facade crate exposes every submodule behind a feature flag, so users pay only for what they use:

```

1 [features]
2 default = ["all"]
3 all = ["core", "timeseries", "vol", "stochastic",
4       "options", "portfolio", "factors",
5       "microstructure", "backtest"]

```

In Rust source, each module is conditionally compiled:

```

1 #[cfg(feature = "portfolio")]
2 pub mod portfolio;

```

A user who needs only option pricing can depend on `quant-lib` with `default-features = false, features = ["options"]`, and the rest of the crate is not compiled.

B.9. The Facade Crate Pattern

`quant-lib` is a *facade*: it contains `pub mod` and `pub use` declarations, zero new logic. It re-exports the public APIs of `quant-core`, `quant-timeseries`, `quant-vol`, ... under a single, versioned namespace.

```

1 // quant-lib/src/lib.rs
2 pub use quant_core;
3 pub use quant_timeseries;
4 pub use quant_vol;
5 // ...
6
7 pub mod prelude {
8     pub use crate::core::*;
9     pub use crate::timeseries::*;
10    // ...
11 }
```

Users write `use quant_lib::prelude::*;` and get the entire public surface. Upgrading a single underlying crate (e.g., `quant-options` for a new pricing model) is a one-line change in `quant-lib/Cargo.toml`; downstream code is unaffected.

B.10. Deterministic Randomness

Every stochastic example in this book uses a seeded RNG to be reproducible:

```

1 use quant_lib::prelude::*;
2 let mut rng = XorShift64::new(42);
3 let path = gbm(100.0, 0.05, 0.20, 1.0, 252, &mut rng)?;
```

The `Rng()` trait abstracts the RNG so a Monte Carlo engine can swap `XorShift64` for a cryptographic generator without code changes. Tests use a fixed seed to make results bit-for-bit reproducible across machines and CI runs.

B.11. Testing Strategy

Every crate ships three kinds of tests:

- ▶ **Unit tests** (in `src/`): small, fast, test one function in isolation.
- ▶ **Integration tests** (in `tests/`): exercise the public API end-to-end.
- ▶ **Examples** (in `examples/`): runnable narratives; each chapter's companion code is an example.

Examples are the primary documentation: `cargo run -p quant-lib -example 08_options_greeks` reproduces the numbers printed in the chapter.

Key Insight

The trait-based architecture makes the library testable at the composition boundary: a test that passes with a `FixedHorizonLabeler` will also pass with a `TripleBarrierLabeler` because both satisfy the same `Labeler` contract. Swapping implementations in tests is a one-line change.

Dataset Sources

C.

This appendix documents every dataset bundled with the book, its format, its provenance, and how to obtain more data from public sources for extending the examples. All bundled files live in `crates/quant-lab/data/` and are loaded by the `quant-lib` examples via the helpers in `examples/common/mod.rs`.

C.1. Bundled Datasets

C.1.1. Synthetic Stock Prices (`stock_prices.csv`)

A 63-row synthetic OHLCV series for a single fictitious stock from 2 January 2024 to 28 March 2024. Used by the early examples (Ch. 3, 4, 5) where a clean, controlled series is preferable to real market data.

Field	Description
date	Trading date (ISO 8601)
open, high, low, close	Daily OHLC prices
volume	Daily traded volume
adj_close	Dividend/split-adjusted close (equals close here)

```
1 use quant_lib::prelude::*;
2 let path = common::stock_csv_path(env!("CARGO_MANIFEST_DIR"));
3 let bars = common::load_stock_csv(&path);
```

C.1.2. B3 Brazilian Stocks (8 JSON files)

Daily OHLCV data for eight liquid stocks traded on B3 (Brasil Bolsa Balcão), the São Paulo exchange. Each file contains 1,247 daily bars from 22 July 2021 to 26 July 2024—roughly three years of trading. Prices are in Brazilian Reais (BRL).

Symbol	Company	Sector
PETR4	Petróleo Brasileiro (Petrobras)	Energy
VALE3	Vale	Mining
ITSA4	Itaúsa (Itaú)	Financial
BBDC4	Bradesco	Financial
B3SA3	B3 (exchange operator)	Financial
ABEV3	Ambev	Consumer staples
GGBR4	Gerdau	Steel
WEGE3	WEG	Industrial

Why Brazilian stocks?

B3 daily data is freely available, covers liquid names, and gives the examples a consistent cross-sectional universe for portfolio and factor exercises. The same code works with any OHLCV source—US equities, ETFs, or crypto—by swapping the data loader.

JSON format. Each file is a JSON array of objects with keys `date`, `open`, `high`, `low`, `close`, `volume`. The bundled parser in `common/mod.rs` is a minimal splitter that relies on the fixed key order; for general JSON, use `serde_json`.

```
1 use quant_lib::prelude::*;
2 let path = common::b3_json_path(env!("CARGO_MANIFEST_DIR"), "PETR4");
3 let bars = common::load_json_ohlcv(&path);
```

C.1.3. Credit Card Fraud Sample (`creditcard_sample.csv`)

A 20-row anonymised sample from the Kaggle Credit Card Fraud Detection dataset. Each row is one transaction; columns V1–V28 are PCA-transformed features, Amount is the transaction value, and Class is 1 for fraud, 0 otherwise. Used in Ch. 1–2 for the fraud-detection and loan-default classifiers.

Field	Description
Time	Seconds elapsed since first transaction
V1–V28	PCA-transformed features (anonymised)
Amount	Transaction value (EUR)
Class	1 = fraud, 0 = legitimate

Warning

The bundled sample is too small for real modelling. It exists only to make the chapter examples runnable offline. For actual experiments, download the full 284,807-row dataset from Kaggle (see below).

C.2. Public Data Sources for Extending the Examples

The bundled data is enough to run every example in the book. To go further—longer histories, cross-sectional studies, alternative asset classes—the following sources are all free and downloadable without registration beyond an email.

C.2.1. Equity OHLCV

Source	URL	Format
Stooq	stooq.com/q/d/?s={symbol}&i=d	CSV OHLCV
Yahoo Finance	finance.yahoo.com/quote/{TICKER}/history	CSV OHLCV
Binance API	api.binance.com/api/v3/klines	JSON OHLCV
Alpha Vantage	alphavantage.co (free tier)	JSON/CSV

Stooq is the easiest for bulk download: it serves daily and intraday bars for US, European, and Brazilian tickers with no API key. Yahoo Finance covers the widest universe. Binance is the primary source for crypto OHLCV via a clean REST API.

C.2.2. Factor and Macro Data

Source	URL	Content
Ken French Data Library	mba.tuck.dartmouth.edu/pages/faculty/ken.french	Fama-French factors, momentum, i
FRED (St. Louis Fed)	fred.stlouisfed.org	Macro series (rates, CPI, vix)
AQR Data Sets	aqr.com/Insights/Datasets	Factor returns, Betting-Against-Beta
World Bank	data.worldbank.org	Country-level macro indicators

The Ken French library is the canonical source for the factor data used in Ch. 12. FRED is the standard for US risk-free rates (DGS3M0, DGS10) and the VIX (VIXCLS).

C.2.3. Options Data

Source	URL	Content
OptionMetrics	(institutional, paid)	Historical option chains
CBOE	cboe.com	Free end-of-day quotes, VIX history
Yahoo Finance	finance.yahoo.com	Basic option chains

The book's option examples (Ch. 10) are self-contained: they price from the Black-Scholes formula rather than reading market option prices. For implied-volatility surface studies, CBOE's free VIX history is the practical starting point.

C.2.4. Microstructure and LOB Data

Source	URL	Content
LOBSTER	lobsterdata.com	NASDAQ ITCH, 10 levels
Databento	databento.com	Paid, multi-venue L2/L3
SEC EDGAR	sec.gov/edgar	Corporate filings (not LOB)

LOBSTER is the academic standard for limit-order-book research, with free access for academics. The book's microstructure examples (Ch. 13) use a simulated book to keep the companion code self-contained.

C.3. Loading Data in Practice

The `quant-lib` examples all use the helpers in `examples/common/mod.rs`, which resolve paths relative to `CARGO_MANIFEST_DIR` so they work regardless of where the binary is invoked from:

```

1 #[path = "common/mod.rs"]
2 mod common;
3
4 fn main() {
5     // Synthetic single-stock CSV
6     let bars = common::load_stock_csv(
7         &common::stock_csv_path(env!("CARGO_MANIFEST_DIR"))
8     );
9
10    // Real B3 stock JSON
11    let bars = common::load_json_ohlcv(
12        &common::b3_json_path(env!("CARGO_MANIFEST_DIR"), "PETR4")

```

```

13     );
14 }

```

To use your own data, either drop a file in `crates/quant-lab/data/` (and it will be found by the helpers), or write a loader that returns `Vec<OhlcvBar>` and pass it to the example code. The `OhlcvBar` struct is:

```

1 pub struct OhlcvBar {
2     pub date: String,
3     pub open: f64,
4     pub high: f64,
5     pub low: f64,
6     pub close: f64,
7     pub volume: f64,
8 }

```

C.4. Reproducibility and Licensing

- ▶ The B3 JSON files were downloaded from a public data provider and are included here for educational use; verify the provider's terms before redistributing.
- ▶ The synthetic `stock_prices.csv` and `creditcard_sample.csv` are generated by the book and are public domain.
- ▶ Public-source data (Stooq, Yahoo, Ken French, FRED) is generally free for research and personal use; check each provider's terms for commercial use.

Key Insight

Every example in the book can be reproduced from the bundled data alone. The public sources listed here are for readers who want to extend the analysis to other tickers, longer windows, or alternative asset classes. The companion code's loader abstraction means switching data sources is a one-function change.

Exercise Solutions

D.

This appendix provides worked solutions to the exercises at the end of each chapter. The solutions give the approach, the key code or formula, and the expected result. Where an exercise asks for experimentation, we report the qualitative finding rather than a specific number—the number depends on the random seed and data window.

Key Insight

The exercises are designed to be reproducible from the bundled data and the companion crates. Every solution below can be verified by running the corresponding example in `quant-lib` with minor modifications. The goal is understanding, not memorisation.

Chapter 1: Credit Card Fraud

1. Threshold Sensitivity. Sweep the threshold over $\{2.0, 2.5, 3.0, 3.5, 4.0\}$, computing precision and recall at each. Lowering the threshold raises recall (more transactions flagged) but lowers precision (more false alarms). The F1-maximising threshold is the one where the precision–recall curve has its knee—typically around 3.0–3.5 for the bundled sample. The lesson: there is no “correct” threshold; the choice depends on the cost of a false negative versus a false positive.

2. Feature Analysis. Modify `predict()` to return the per-feature z-scores alongside the boolean. The most discriminative feature is the one whose z-score most consistently exceeds the threshold on fraud samples. In the anonymised PCA dataset (V1–V28), features V14 and V4 typically dominate—they carry the highest mean $|z|$ on the fraud rows.

3. Train/Test Split. Split chronologically because fraud patterns evolve: a random split leaks future information into training. The test-set performance is almost always worse than training-set: precision drops by 10–20% because the z-score statistics are calibrated on the earlier period. This is the simplest demonstration of why time-series data must never be shuffled before splitting.

4. Amount Feature. Add amount as a z-scored feature. Performance barely moves because the PCA features already absorb most of the signal; the raw amount is weakly correlated with fraud after conditioning on the anonymised features. The lesson: not every available feature adds value—check before including.

5. Trait Extraction. The `AnomalyDetector` trait should require:

```
1 pub trait AnomalyDetector {  
2     fn fit(&mut self, x: &[Vec<f64>]);  
3     fn predict(&self, x: &[f64]) -> AnomalyResult;  
4 }  
5  
6 pub struct AnomalyResult {
```

```

7   pub is_anomaly: bool,
8   pub score: f64,
9   pub triggering_features: Vec<usize>,
10 }

```

`fit()` learns the per-feature mean and std; `predict()` returns the z-score, the boolean, and which features exceeded the threshold. This is the same shape as `BinaryClassifier()` plus `Scorer()` from `qf-02-loan`, generalised to multivariate input.

Chapter 2: Loan Default

1. Weight Sensitivity. Sweep each weight $\pm 50\%$ while holding the others fixed. The AUC moves by at most a few points, indicating the linear scorer is robust to moderate weight mis-specification. The most sensitive feature is the one whose weight change moves AUC the most—typically `debt_to_income` or `credit_score`, depending on the dataset.

2. Threshold Selection. The sigmoid output $\sigma(\mathbf{w}^\top \mathbf{x})$ is a probability. Sweep the threshold from 0.01 to 0.99; the F1 maximum is typically near the base rate (the fraction of defaults in the data), because that is where precision and recall balance for a balanced classifier.

3. Feature Importance. Train a single-feature scorer for each feature and compute its AUC. Rank by AUC. The ranking usually matches the weight magnitude from the multivariate scorer, but not always: correlated features can split importance, so a feature that is weak alone can be strong in combination.

4. Cross-Validation. Split into 5 chronological folds. The AUC standard deviation across folds is typically 0.02–0.05. A large std (relative to the mean) indicates the model is sensitive to the training window—a sign of instability, not robustness.

5. Calibration Plot. Bin predictions into deciles and plot the mean predicted probability against the empirical default rate in each bin. A well-calibrated model lies on the diagonal; a miscalibrated one is systematically above (overconfident) or below (underconfident). The linear scorer is usually underconfident at high probabilities because the sigmoid saturates.

Chapter 3: Market Data

1. Candlestick Patterns. Doji: $|O - C| < 0.1\%$ of the range $H - L$. Hammer: small body ($|O - C| < 0.3(H - L)$) and long lower shadow ($\min(O, C) - L > 2|O - C|$). Both are one-line tests inside the `Ohlcv()` struct's accessor methods.

2. Volume Analysis. `volume_ma()` is a simple `sma()` applied to the volume field. A day with volume $> 2 \times \text{MA}_N$ is flagged as high-volume. On the bundled `stock_prices.csv`, roughly 5–8% of days exceed 2x; these often coincide with earnings or macro releases.

3. Bollinger Bands. Upper = $SMA_N + 2\sigma_N$, lower = $SMA_N - 2\sigma_N$, where σ_N is the rolling standard deviation (use `rolling_std_dev()` from `quant-core`). Prices touch the upper band on $\sim 5\%$ of days under a normal approximation, confirming the bands are a 2-sigma envelope.

4. MACD. $MACD = EMA_{12} - EMA_{26}$, $signal = EMA_9(MACD)$. A buy signal fires when MACD crosses above its signal line. The indicator is a momentum oscillator; it lags the price turn because EMAs are backward-looking.

5. Backtest. Buy at the close on a golden cross, sell at the close on a death cross. On the synthetic series, the strategy is profitable only if the trend persists longer than the lag; in choppy markets it whipsaws and loses. The lesson: SMA crossover is a trend filter, not a predictor—it works in regimes and fails in transitions.

Chapter 4: Returns and Volatility

1. Return Conversion. $r^{\log} = \ln(1 + r^{\text{simple}})$ and $r^{\text{simple}} = e^{r^{\log}} - 1$. A property test generates random simple returns, converts to log and back, and asserts the round-trip error is below 10^{-15} . The identity is exact in real arithmetic; floating-point round-off is the only source of error.

2. Annualisation Sanity Check. Resample daily returns to weekly and monthly. Annualised volatility is $\sigma_{\text{daily}}\sqrt{252} \approx \sigma_{\text{weekly}}\sqrt{52} \approx \sigma_{\text{monthly}}\sqrt{12}$. The three estimates agree to within sampling noise; the discrepancy is $O(1/\sqrt{T})$ where T is the number of observations at each frequency.

3. Calmar Ratio. $\text{Calmar} = \text{annualised return} / |\text{max drawdown}|$. On a synthetic strategy with 12% annual return and 15% max drawdown, $\text{Calmar} = 0.8$. Sharpe penalises all volatility; Calmar penalises only the worst drawdown. A strategy with high Sharpe but deep drawdown has a low Calmar—common in trend-following.

4. Rolling Sharpe. Windowed Sharpe = $\frac{\bar{r}_N - r_f}{\sigma_N} \sqrt{m}$. It dips during volatile periods (e.g., a 2-sigma shock raises σ_N immediately but the mean responds slowly). The rolling chart visualises regime changes that the full-sample Sharpe hides.

5. Drawdown with Recovery. Average drawdown = $\frac{1}{T} \sum_t \min(0, DD_t)$. Proportion in drawdown = $\frac{1}{T} \sum_t \mathbb{1}[DD_t < 0]$. A buy-and-hold equity curve spends 60–80% of its time in drawdown—the “tyranny of drawdowns” that motivates risk-managed strategies.

Chapter 5: Backtesting

1. Cost Sensitivity. For $\tau \in \{0, 0.001, 0.0025, 0.005, 0.01\}$, total return declines roughly linearly in τ because turnover is roughly constant. The strategy stops being profitable when $\tau \times \text{turnover} > \text{gross edge}$. For the 10/30 crossover on the synthetic series, this happens near $\tau \approx 0.005$ (50 bps round-trip).

2. Parameter Sensitivity. The (S, L) that maximises Sharpe is not the one that maximises total return: a faster crossover (small S, L) has higher gross return but higher turnover, so net return (after costs) is lower. The Sharpe-maximising configuration is typically (10, 50) or (20, 100)—slow enough to filter noise, fast enough to catch trends.

3. EMA Crossover. EMA crossover has lower turnover than SMA because EMAs decay smoothly rather than dropping observations. On the same data, the EMA strategy typically trades 20–30% less and has a comparable or slightly higher Sharpe. The trade-off: EMAs react faster but are more sensitive to outliers.

4. Position Sizing. With fraction $f = 0.5$, the equity curve is flatter (lower return, lower volatility, shallower drawdowns). The trade-off is return-for-risk: $f = 0.5$ roughly halves both the annual return and the max drawdown. Full-Kelly ($f = f^*$) maximises log growth; fractional Kelly trades growth for safety.

5. Walk-Forward. Tune (S, L) on the first 180 bars; freeze; evaluate on the last 72. The out-of-sample Sharpe is typically 30–60% of the in-sample Sharpe—a WFE of 0.3–0.6. A WFE below 0.3 signals overfitting; the in-sample optimum exploited noise.

Chapter 6: Statistical Foundations

1. Sample vs Population Variance. For $\{1, 2, 3, 4, 5\}$: population variance = 2.0 (denominator $n = 5$), sample variance = 2.5 (denominator $n - 1 = 4$). The bias is $(n - 1)/n = 0.8$, a 20% underestimate. It matters for small samples and for any statistic that scales with variance (Sharpe, t-stat). Always use $n - 1$ for sample-based estimators.

2. Skewness Sign. Negative skew: mix a normal with a few large negative draws (e.g., $x = N(0, 1)$ with probability 0.9, $x = -5$ with probability 0.1). Positive skew: the mirror. `skewness()` returns the correct sign. Equity returns typically have *negative* skew—large drops are more frequent than large jumps.

3. RNG Period. A test asserts the first 1000 outputs from seed 42 are distinct. “Distinct” is necessary (a PRNG that repeats early is broken) but not sufficient (a PRNG can be distinct yet correlated—e.g., a counter). The full statistical battery (TestU01, DieHarder) checks uniformity, independence, and higher-order structure.

4. GBM Convergence. With $S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$, $T = 1$, the analytical $\mathbb{E}[S_T] = 100 \cdot e^{0.05} = 105.13$. A 10 000-path Monte Carlo gives $\hat{\mathbb{E}}[S_T] \approx 105.1 \pm 0.2$ (the SE scales as $\sigma\sqrt{T}/\sqrt{N}$). The estimate is within one SE of the truth—the small discrepancy is sampling noise, not bias.

5. Jump-Diffusion. Add $J \sim N(0, 0.05^2)$ with intensity λ . As λ grows from 0 to 1 (one jump per year on average), the excess kurtosis of terminal prices rises from 0 to ~ 3 –5. Jumps inject fat tails that GBM alone cannot reproduce—the basis of the Merton model.

Chapter 7: Time Series

1. Partial Autorrelation. The PACF via Yule-Walker solves the linear system $\mathbf{R}\phi = \mathbf{r}$ where $R_{ij} = \rho_{|i-j|}$ and $r_i = \rho_i$. For AR(2) with $\phi_1 = 0.5$, $\phi_2 = -0.3$, PACF is ~ 0.5 at lag 1, ~ -0.3 at lag 2, and ~ 0 beyond—the cutoff at lag $p = 2$ identifies the order.

2. KPSS Test. KPSS tests H_0 : stationary (opposite of ADF). On white noise, both ADF and KPSS agree (stationary). On a random walk, ADF fails to reject (non-stationary) and KPSS rejects (non-stationary). On fractionally differenced series with $d < d^*$, the two can disagree: ADF may reject (stationary by unit-root criterion) while KPSS rejects (non-stationary by trend criterion). The disagreement signals a long-memory process—neither pure stationary nor pure unit root.

3. Real Returns. For $\sigma \in \{0.1, 0.2, 0.4\}$, the minimum d^* for stationarity is approximately 0.35–0.45 and is largely insensitive to σ . The reason: d^* depends on the memory structure (how slowly the ACF decays), not the scale. Higher σ adds noise but does not change the differencing order needed to remove the unit root.

4. Expanding-Window Frac-Diff. The expanding window uses all available history for early points, truncating the weight sequence at the available length rather than dropping the first $K - 1$ points. The trade-off: more data for early points (no warm-up loss) but the early values are biased because they use fewer terms of the infinite series. Fixed-width is consistent (every point uses the same number of terms); expanding is efficient (no data loss) but heterogeneous.

5. Lag Selection for ADF. $\text{AIC} = n \ln(\hat{\sigma}^2) + 2k$ where k is the number of lags. Sweep k from 0 to 10, pick the AIC-minimising k , then run ADF. The stationarity conclusion rarely changes for well-behaved series; it can change for borderline series where the lag structure affects the residual autocorrelation. AIC-based selection is more principled than a fixed lag.

Chapter 8: Volatility Models

1. EGARCH. $\log \sigma_t^2 = \omega + \alpha g(z_{t-1}) + \beta \log \sigma_{t-1}^2$ where $g(z) = \theta z + \gamma(|z| - \mathbb{E}|z|)$. The θ term captures the leverage effect: negative returns ($z < 0$) raise $\log \sigma^2$ more than positive ones. On equity returns, EGARCH's log-likelihood typically exceeds GARCH(1,1) by 5–15 points, with $\theta < 0$ significantly.

2. GJR-GARCH. $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \gamma \mathbb{1}_{r_{t-1} < 0} r_{t-1}^2 + \beta \sigma_{t-1}^2$. On equity index returns, γ is typically 0.03–0.10 and significant, confirming the leverage effect. The effective coefficient on negative shocks is $\alpha + \gamma$, larger than α alone.

3. Forecast Evaluation. Fit on 80%, forecast the conditional variance on the remaining 20%. The GARCH forecast converges to the long-run variance $\sigma_\infty^2 = \omega / (1 - \alpha - \beta)$ as the horizon grows. The mean squared error of the variance forecast is typically 20–40% lower than the EWMA forecast, because GARCH adapts to the estimated persistence while EWMA uses a fixed $\lambda = 0.94$.

4. Student- t Innovations. Replace the Gaussian density with the Student- t density with ν degrees of freedom. On heavy-tailed returns, the fitted ν is typically 4–7, and the t -GARCH log-likelihood exceeds the Gaussian GARCH by 10–30 points. The improvement is largest in the tails, which the Gaussian model underweights.

5. Long-Memory FIGARCH. FIGARCH(1, d , 1) replaces the geometric decay with a hyperbolic $(1 - L)^d$ filter on the variance recursion. The half-life of shocks grows from ~ 10 days (GARCH(1,1)) to ~ 30 –50 days (FIGARCH), reflecting the slow decay of volatility autocorrelation observed in high-frequency data.

Chapter 9: Stochastic Processes

1. Control Variates. $\hat{C}_{cv} = \hat{C} - \beta(\hat{M} - \mathbb{E}[S_T])$ with $\beta = \text{Cov}(\text{payoff}, S_T) / \text{Var}(S_T)$. For a call option, $\beta \approx e^{-rT} N(d_1)$ (the delta). The standard error drops by a factor of $\sqrt{1 - \rho^2}$ where ρ is the correlation between payoff and S_T —typically 70–90% for vanilla calls, giving a 2–3x SE reduction.

2. Asian Option. Time-step the GBM over $n = 252$ daily closes, compute $\bar{S} = \frac{1}{n} \sum S_{t_k}$, payoff $\max(\bar{S} - K, 0)$. The Asian price is below the European price (averaging reduces variance, so the option is cheaper). The SE is comparable to the European for the same N , because the variance reduction from averaging is in the payoff, not the estimator.

3. Euler vs Exact. Euler discretisation $S_{t+\Delta t} = S_t(1 + \mu\Delta t + \sigma\sqrt{\Delta t}Z)$ has bias $O(\Delta t)$ relative to the exact $S_T = S_0 \exp((\mu - \frac{1}{2}\sigma^2)T + \sigma W_T)$. At $\Delta t = 1/252$, the bias in the mean is $\sim 0.01\%$ —negligible for pricing but visible in high-precision convergence studies.

4. Confidence Intervals. For $N = 10\,000$, the 95% CI is $\hat{C} \pm 1.96 \cdot \text{SE}$. Repeating 1000 times, the fraction of CIs covering the BS price is $\sim 94\text{--}96\%$, consistent with the nominal 95%. Coverage below 93% signals a bug (e.g., wrong discounting); coverage above 97% suggests the SE is overestimated.

Chapter 10: Options Pricing

1. Black-Scholes with Dividends. Replace S_0 with $S_0 e^{-qT}$. The call becomes $C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$ with $d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$. Delta = $e^{-qT} N(d_1)$ (the dividend yield discounts the delta). The forward price $F = S_0 e^{(r-q)T}$ replaces $S_0 e^{rT}$ in put-call parity.

2. Local Volatility. $\sigma(S, t) = \sigma_0 + \alpha(S - S_0)^2$ creates a smile: ATM has vol σ_0 , OTM and ITM have higher vol. Explicit Euler on a (S, t) grid with $\Delta S = 1$, $\Delta t = 1/252$ prices the option. The IV surface recovered by inverting BS on the grid prices is convex in moneyness, matching the local-vol shape.

3. Heston Model. $dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}dW_t^{(v)}$ with $\text{Cov}(dW, dW^{(v)}) = \rho dt$. Monte Carlo with full truncation ($v_t = \max(v_{t-1}, 0)$) generates call prices across a strip of strikes. The IV smile is skewed: negative ρ produces a downward-sloping skew (puts have higher IV), matching equity markets.

4. Vega-Clipped Bisection. For $S_0 = 1000$, $K = 10$, the call price equals the intrinsic $990 \cdot e^{-rT}$ to machine precision. Vega is essentially zero (dividing by $d_2 \rightarrow \infty$), so Newton's method diverges. The fix: detect $\text{Vega} < \epsilon$ and return $\text{ok}(\text{sigma_min})$ with a warning flag, because the IV is genuinely unrecoverable—any vol gives the same price.

5. Smile Fitting. Fit $\sigma(K) = \beta_0 + \beta_1 \ln(K/S_0) + \beta_2 (\ln(K/S_0))^2$ by OLS. On SPX option data, the residual is typically 0.1–0.5 vol points, β_1 is negative (the skew), and β_2 is positive (the convexity). The skew coefficient β_1 quantifies how much the market prices downside risk.

6. Newton Convergence. Instrument `implied_vol()` to print $|C(\sigma_n) - C_{\text{mkt}}|$ at each step. For ATM options, the error roughly squares each iteration (quadratic convergence) until it hits machine precision in 4–6 steps. For deep ITM/OTM, Newton slows to linear (the derivative is tiny), and the bisection fallback takes over, converging in $\sim 10\text{--}20$ iterations.

Chapter 11: Portfolio Optimization

1. Correlated Assets. For $\rho = 0.5$, the frontier bends left (diversification helps but less than independence). For $\rho = -0.5$, it bends further left. For $\rho = -1$, the frontier touches the $\sigma = 0$ axis: the zero-variance portfolio has weight $w_A = \sigma_B/(\sigma_A + \sigma_B)$, giving $\sigma_p = 0$ exactly. This is the only case where risk can be fully eliminated.

2. Three-Asset Frontier. With $\mu_C = 0.08, \sigma_C = 0.15$, the global minimum-variance portfolio mixes all three assets to minimise $\mathbf{w}^\top \Sigma \mathbf{w}$. The tangency Sharpe is higher than the two-asset case because the third asset adds a diversification opportunity. The gain is largest when the new asset has low correlation with the existing two.

3. Long-Only Constraint. Projected gradient descent: take a gradient step, then project onto the probability simplex ($w_i \geq 0, \sum w_i = 1$) by sorting and thresholding. For a high target $\mu_\star = 0.15$, the unconstrained solution shorts the low-return asset; the constrained solution allocates 100% to the high-return asset, with higher variance. The constraint widens the frontier.

4. Black-Litterman. Prior $\mu \propto \Sigma \mathbf{w}_{\text{mkt}}$ (reverse-optimised from market weights). Posterior $\mu_{\text{BL}} = [(\tau \Sigma)^{-1} + \Omega^{-1}]^{-1}[(\tau \Sigma)^{-1} \mu + \Omega^{-1} \mathbf{q}]$ where $\mathbf{q}$ is the view and Ω the view confidence. The posterior tangency tilts toward the view; the prior tangency is the market portfolio. The shrinkage parameter τ controls how much the view moves the estimate.

5. Rolling CAPM. Rolling 60-day beta = $\text{Cov}(r_i, r_m) / \text{Var}(r_m)$ computed over a sliding window. Betas are time-varying: a defensive stock's beta drops in bear markets (investors flock to safety), a cyclical stock's beta rises. The plot visualises the regime change—a constant beta assumption is a first-order approximation.

6. CVaR Optimisation. $\text{CVaR}_{95\%} = \mathbb{E}[-r_p \mid -r_p > \text{VaR}_{95\%}]$. Minimising CVaR over the two-asset simplex by grid search gives a portfolio that is more concentrated in the lower-variance asset than the Markowitz minimum-variance portfolio. The two differ when the return distribution is asymmetric—CVaR penalises tail shape, variance penalises spread.

Chapter 12: Factor Models

1. Four- and Five-Factor Models. Add momentum (UMD) and quality (e.g., gross profitability). The R^2 typically rises by 2–8 percentage points over the 3-factor model, with momentum contributing more for individual stocks and quality more for portfolios. The incremental gain diminishes as more factors are added—diminishing returns to factor zoo.

2. Jacobi Eigenvalue. The Jacobi algorithm rotates pairs of rows/columns to zero out off-diagonal entries. For a 5×5 symmetric matrix, it converges in 5–15 sweeps, each $O(n^2)$ rotations. Compared to the power method, Jacobi computes all eigenvalues simultaneously (no deflation error) but is slower for large n . For $n = 5$ the two agree to machine precision.

3. PCA on Real Equity Returns. Run PCA on 10 stocks, 252 days. The first PC is typically a “market factor” (all-positive loadings, ~ 30 –50% of variance). The second and third PCs capture sector tilts (e.g., growth vs value, financials vs tech). The economic interpretation is not automatic—it requires labelling the loadings by sector.

4. Rolling 60-Day Beta. $\beta_t = \text{Cov}_{60}(r_i, r_m) / \text{Var}_{60}(r_m)$, plotted over time. Betas of defensive stocks (utilities) are < 1 and stable; betas of cyclical stocks (tech, finance) are > 1 and volatile. The rolling chart shows when a stock's sensitivity to the market changes—useful for regime-aware risk management.

5. Market-Neutral Long-Short. Long the high-SMB quintile, short the low-SMB quintile, scaled so $\beta_{\text{Mkt}} = 0$. The portfolio's variance decomposes into systematic (SMB-driven) and idiosyncratic (stock-specific) parts. Diversification (many stocks per leg) eliminates the idiosyncratic part, leaving the systematic SMB exposure—a “pure size factor” portfolio.

Chapter 13: Market Microstructure

1. Order Book Builder.

```

1 fn build_book(orders: Vec<(Side, i64, u64, u64)>) -> OrderBook {
2     let mut book = OrderBook::new(1);
3     for (side, price, qty, ts) in orders {
4         book.add_order(Order { id: ts, side, price, quantity: qty, timestamp: ts }).
            unwrap();
5     }
6     book
7 }

```

Assert `best_bid()` and `best_ask()` return the expected prices and `total_volume()` sums the quantities.

2. Liquidity Sweep. Build 5 ask levels totalling 1000 shares. A market buy of 750 sweeps the first ~ 3 –4 levels. The VWAP of the fills is $\leq$ the volume-weighted ask price of all swept levels (equality when the order exactly exhausts a level). The test confirms price-time priority: earlier orders fill first.

3. OFI and Returns. Generate snapshots where bid quantity rises with “buy pressure” and ask quantity falls. $\text{OFI} = \Delta q^{\text{bid}} - \Delta q^{\text{ask}}$ is positive when buy pressure rises. Mid-price returns are also positive. The correlation $\rho(\text{OFI}, r_{\text{mid}})$ is typically 0.3–0.6, confirming OFI is a signed predictor of short-term returns.

4. Impact Scaling. $\text{sqrt_impact}(\sigma, Q, V) = \sigma \sqrt{Q/V}$. Plotting impact vs Q on a log-log scale gives a line of slope $1/2$, confirming the square-root law. Doubling Q multiplies impact by $\sqrt{2} \approx 1.41$, not by 2—the concavity is the empirical regularity that motivates the Almgren-Chriss model.

5. Net Frontier. Per-rebalance cost = `execution_cost()` ($Q = 0.5V_{\text{daily}}, V_{\text{daily}}, \sigma, \text{spread}$). At 50% turnover, the cost is ~ 5 –15 bps. Subtract from the expected return: the net efficient frontier shifts down by the cost times turnover. The tangency Sharpe drops by ~ 0.05 –0.15, depending on the spread and impact. The lesson: gross frontier optimisation overstates net performance.

Chapter 14: AFML Backtesting

1. Custom Barriers. Set the upper barrier to $\phi_t = 2\sigma_t$ and the lower to $\psi_t = -2\sigma_t$ where σ_t is the rolling 20-day volatility. On a noisy series, the barriers widen, fewer events hit the profit-taking label (more time-outs); on a calm series, the barriers narrow, more events hit the barriers. The label distribution shifts from $\sim 20\%$ up to $\sim 10\%$ up as volatility rises—volatility-scaled barriers adapt to the regime.

2. Embargo Calibration. Compute the ACF of FFD returns. The embargo ρ is the first lag where $|\rho_k| < 1.96/\sqrt{T}$ (the 5% significance band). For daily FFD returns, ρ is typically 1–5 days. Without embargo, the out-of-sample Sharpe is inflated by $\sim 5\text{--}15\%$ due to leakage; with embargo, the Sharpe drops to its honest value.

3. Volatility-Scaled Kelly. $f_t = f^* \cdot \bar{\sigma}/\sigma_t$ where $\bar{\sigma}$ is the long-run volatility. In high-vol periods, f_t shrinks; in low-vol periods, it grows. The drawdown distribution tightens because the largest losses (which occur in high-vol periods) are scaled down. The trade-off: the strategy grows more slowly in calm periods, but survives turbulent ones.

4. Meta-Labeling. Primary: sign of SMA crossover. Label with triple barrier. Train a logistic regression on the labelled events to predict correctness (meta-model). The meta-filtered strategy takes the primary signal only when the meta-model predicts “correct” (probability > 0.5). The meta-filtered Sharpe is typically 20–50% higher than the primary alone, with fewer trades—the meta-model filters out the primary’s low-confidence signals.

Mathematical Derivations

E.

This appendix provides full derivations for several results that are stated without proof in the main text. The derivations are collected here so that the chapters can stay focused on implementation while the mathematically inclined reader can still verify every claim.

Key Insight

Every result in this appendix is reproduced by a test in the companion crates. The derivations are not optional theory—they are the reason the code works. Where a derivation ends in a formula, the corresponding Rust function is named in the margin.

E.1. E.1: Kelly Criterion from Expected Log Utility

E.1.1. Setup

An investor allocates fraction f of wealth to a risky asset with binary return: with probability p the asset gains fraction b (upside), and with probability $q = 1 - p$ it loses fraction a (downside). The remaining fraction $1 - f$ earns the risk-free rate, which we absorb into b and a for notational clarity (i.e., b and a are excess returns). The gross return per unit invested in the risky asset is $1 + bf$ on an up move and $1 - af$ on a down move.

Wealth after one period is

$$W_1 = W_0 \cdot \begin{cases} 1 + bf & \text{with prob. } p, \\ 1 - af & \text{with prob. } q. \end{cases}$$

The Kelly objective is to maximise the expected log of terminal wealth:

$$g(f) = \mathbb{E}[\ln W_1] = p \ln(1 + bf) + q \ln(1 - af).$$

E.1.2. First-Order Condition

Differentiate with respect to f :

$$g'(f) = \frac{pb}{1 + bf} - \frac{qa}{1 - af}.$$

Set $g'(f^*) = 0$:

$$\frac{pb}{1 + bf^*} = \frac{qa}{1 - af^*}.$$

Cross-multiply:

$$pb(1 - af^*) = qa(1 + bf^*).$$

Expand:

$$pb - pba f^* = qa + qab f^*.$$

Collect f^* terms on the right:

$$pb - qa = pba f^* + qab f^* = ab f^* (p + q) = ab f^*.$$

Since $p + q = 1$:

$$f^* = \frac{pb - qa}{ab} = \frac{p}{a} - \frac{q}{b}.$$

E.1.3. Interpretation

The Kelly fraction is the edge $pb - qa$ divided by the odds product ab . When $a = b = 1$ (a fair coin-like payoff where you win or lose your stake), $f^* = p - q = 2p - 1$, the classic Kelly formula for an even-money bet.

E.1.4. Second-Order Condition

The second derivative is

$$g''(f) = -\frac{pb^2}{(1 + bf)^2} - \frac{qa^2}{(1 - af)^2} < 0,$$

so g is strictly concave in f and f^* is the unique global maximum. The growth rate at the optimum is

$$g(f^*) = p \ln(1 + bf^*) + q \ln(1 - af^*),$$

which is positive whenever $pb > qa$ (i.e., the bet has positive expected return).

E.1.5. Continuous-Time Limit

In continuous time the result generalises. If the risky asset follows geometric Brownian motion with drift μ and volatility σ , and the risk-free rate is r , the Kelly fraction is

$$f^* = \frac{\mu - r}{\sigma^2}.$$

This is obtained by maximising

$$\mathbb{E}[\ln(1 + f \cdot (\mu - r) \cdot dt - \frac{1}{2} f^2 \sigma^2 dt)] = (f(\mu - r) - \frac{1}{2} f^2 \sigma^2) dt$$

over f , yielding $f^* = (\mu - r)/\sigma^2$.

Rust: `kelly_fraction()` in `quant-backtest` implements the discrete formula. The continuous case is Exercise 14.3.

E.1.6. Fractional Kelly

Full Kelly is notoriously aggressive: the drawdowns are proportional to $\sqrt{f}$. Fractional Kelly uses $f = c \cdot f^*$ with $c \in (0, 1]$. The trade-off is that growth rate decreases as c^2 while volatility decreases as c :

$$g(cf^*) = c g(f^*) - \frac{1}{2} c(1 - c) \sigma_f^2,$$

where σ_f^2 is the variance of the log return at full Kelly. Half-Kelly ($c = 0.5$) gives 75% of the growth rate at 50% of the volatility—a common practitioner choice.

E.2. E.2: Triple-Barrier Return Distribution

E.2.1. Setup

Let S_t follow geometric Brownian motion

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

with S_0 the entry price. The three barriers are:

- **Take profit** (horizontal upper): $S = S_0(1 + \phi)$.
- **Stop loss** (horizontal lower): $S = S_0(1 - \psi)$.
- **Time barrier** (vertical): $t = T$.

The labelled return y is +1 if the price hits the upper barrier first, −1 if it hits the lower barrier first, and 0 if it hits the time barrier without touching either horizontal barrier.

E.2.2. Log-Price Transformation

Define $X_t = \ln(S_t/S_0)$. By Itô's lemma,

$$dX_t = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t = \nu dt + \sigma dW_t,$$

where $\nu = \mu - \frac{1}{2}\sigma^2$ is the log-drift. The barriers in X -space are

$$\alpha = \ln(1 + \phi), \quad \beta = \ln(1 - \psi), \quad T,$$

with $\alpha > 0$ and $\beta < 0$.

E.2.3. Hitting Probabilities

Let $\tau_\alpha = \inf\{t : X_t = \alpha\}$ and $\tau_\beta = \inf\{t : X_t = \beta\}$ be the first-passage times. The probability that the upper barrier is hit before the lower, and before T , is

$$P_+ = \Pr(\tau_\alpha < \tau_\beta \wedge T).$$

By the reflection principle and the method of images (see Karlin & Taylor, *A Second Course in Stochastic Processes*, Ch. 7), the unconditional hitting probabilities are:

$$\Pr(\tau_\alpha < \tau_\beta) = \frac{e^{-2\nu\beta/\sigma^2} - 1}{e^{-2\nu\beta/\sigma^2} - e^{-2\nu\alpha/\sigma^2}},$$

$$\Pr(\tau_\beta < \tau_\alpha) = \frac{1 - e^{-2\nu\alpha/\sigma^2}}{e^{-2\nu\beta/\sigma^2} - e^{-2\nu\alpha/\sigma^2}}.$$

When $\nu = 0$ (zero drift), these reduce to

$$\Pr(\tau_\alpha < \tau_\beta) = \frac{|\beta|}{\alpha + |\beta|}, \quad \Pr(\tau_\beta < \tau_\alpha) = \frac{\alpha}{\alpha + |\beta|},$$

which is the gambler's ruin formula on the log-price lattice.

E.2.4. Time Barrier Contribution

The event $\tau_\alpha \wedge \tau_\beta > T$ means the price stays within the band $[\beta, \alpha]$ for the entire horizon. The distribution of X_T is $\mathcal{N}(\nu T, \sigma^2 T)$, but we need the probability that X_t never leaves the band, not just the terminal distribution. Using the reflection principle and the CDF of the running maximum (Harrison, *Brownian Motion and Stochastic Flow Systems*):

$$P_0 = \Pr(\tau_\alpha \wedge \tau_\beta > T) = \sum_{k=-\infty}^{\infty} \left[\Phi\left(\frac{\alpha - \nu T + 2k(\alpha - \beta)}{\sigma\sqrt{T}}\right) - \Phi\left(\frac{\beta - \nu T + 2k(\alpha - \beta)}{\sigma\sqrt{T}}\right) \right],$$

where Φ is the standard normal CDF. In practice, the series is truncated to $|k| \leq 5$ because the terms decay geometrically.

E.2.5. Final Label Distribution

The triple-barrier label distribution is:

$$\Pr(y = +1) = P_+, \quad \Pr(y = -1) = P_-, \quad \Pr(y = 0) = P_0,$$

with $P_+ = \Pr(\tau_\alpha < \tau_\beta \wedge T)$ and similarly for P_- . Note $P_+ + P_- + P_0 = 1$ by the law of total probability.

Rust: `triple_barrier_label()`
in `quant-backtest`
uses Monte Carlo to estimate P_+ , P_- , P_0 . The closed form above is the benchmark for the test suite.

E.2.6. Zero-Drift Simplification

When $\nu \approx 0$ (the practitioner assumption for short holding periods), P_0 reduces to

$$P_0 \approx \sum_{k=-\infty}^{\infty} (-1)^k \left[\Phi\left(\frac{\alpha + 2k(\alpha - \beta)}{\sigma\sqrt{T}}\right) - \Phi\left(\frac{\beta + 2k(\alpha - \beta)}{\sigma\sqrt{T}}\right) \right].$$

For $\sigma\sqrt{T} \ll \alpha + |\beta|$ (tight barriers), $P_0 \approx 0$ and the label is almost always ± 1 . For $\sigma\sqrt{T} \gg \alpha + |\beta|$ (loose barriers), $P_0 \approx 1$ because the price stays in the band with high probability.

E.3. Hierarchical Risk Parity Recursive Bisection

E.3.1. Setup

HRP (López de Prado, *Machine Learning for Asset Managers*, 2019) builds a portfolio in three stages:

1. **Stage 1 — Tree clustering:** From the correlation matrix, compute a distance matrix, hierarchically cluster assets, and get a linkage matrix.
2. **Stage 2 — Quasi-diagonalisation:** Reorder the covariance matrix so that similar assets sit adjacent on the diagonal.
3. **Stage 3 — Recursive bisection:** Starting from the full allocation (inverse variance), repeatedly split each cluster in two and redistribute weight using the inverse variance of each sub-cluster.

We prove here that Stage 3 produces a weight vector w that (a) sums to one, (b) is invariant to the scale of the covariance matrix, and (c) reduces to inverse-variance weighting when the hierarchy is the trivial (single-cluster) tree.

E.3.2. Recursive Bisection Algorithm

Let the quasi-diagonal covariance matrix be Σ , with assets ordered to match the dendrogram leaves. Define the inverse-variance weight of a subset S of assets as

$$\tilde{w}_S = \frac{\Sigma_S^{-1}\mathbf{1}}{\mathbf{1}^\top \Sigma_S^{-1}\mathbf{1}}, \quad \text{where } \Sigma_S \text{ is the submatrix on } S.$$

The scalar variance of the inverse-variance portfolio on S is

$$\tilde{\sigma}_S^2 = \tilde{w}_S^\top \Sigma_S \tilde{w}_S = \frac{\mathbf{1}^\top \Sigma_S^{-1} \Sigma_S \Sigma_S^{-1} \mathbf{1}}{(\mathbf{1}^\top \Sigma_S^{-1} \mathbf{1})^2} = \frac{\mathbf{1}^\top \Sigma_S^{-1} \mathbf{1}}{(\mathbf{1}^\top \Sigma_S^{-1} \mathbf{1})^2} = \frac{1}{\mathbf{1}^\top \Sigma_S^{-1} \mathbf{1}}.$$

The bisection splits a cluster S into children S_1, S_2 and updates:

$$\alpha = 1 - \frac{\tilde{\sigma}_{S_1}^2}{\tilde{\sigma}_{S_1}^2 + \tilde{\sigma}_{S_2}^2}, \quad \text{weight on } S_1 = \alpha, \quad \text{weight on } S_2 = (1 - \alpha).$$

E.3.3. Property 1: Weights Sum to One

Claim: After all bisections, $\sum_i w_i = 1$.

Proof by induction on the number of bisections k .

Base case ($k = 0$): The initial allocation is $\tilde{w}_S = \Sigma^{-1}\mathbf{1}/(\mathbf{1}^\top \Sigma^{-1}\mathbf{1})$, which sums to one by construction: $\mathbf{1}^\top \tilde{w}_S = (\mathbf{1}^\top \Sigma^{-1}\mathbf{1})/(\mathbf{1}^\top \Sigma^{-1}\mathbf{1}) = 1$.

Inductive step: Assume after k bisections the weights in each leaf cluster sum to its current allocation. At step $k + 1$, cluster S with current weight w_S is split into S_1, S_2 . We update $w_{S_1} = \alpha$ and $w_{S_2} = 1 - \alpha$, so $w_{S_1} + w_{S_2} = \alpha w_S + (1 - \alpha)w_S = w_S$. The total weight is unchanged, hence still sums to one. $\square$

E.3.4. Property 2: Scale Invariance

Claim: If we rescale $\Sigma \rightarrow c\Sigma$ for $c > 0$, the weights w are unchanged.

Proof: The inverse-variance weight scales as

$$\tilde{w}_S(c\Sigma) = \frac{(c\Sigma_S)^{-1}\mathbf{1}}{\mathbf{1}^\top (c\Sigma_S)^{-1}\mathbf{1}} = \frac{c^{-1}\Sigma_S^{-1}\mathbf{1}}{c^{-1}\mathbf{1}^\top \Sigma_S^{-1}\mathbf{1}} = \tilde{w}_S(\Sigma),$$

so $\tilde{w}_S$ is scale-invariant. The sub-cluster variance scales as

$$\tilde{\sigma}_S^2(c\Sigma) = \frac{1}{\mathbf{1}^\top (c\Sigma_S)^{-1}\mathbf{1}} = c \cdot \tilde{\sigma}_S^2(\Sigma),$$

but the ratio in α cancels:

$$\alpha(c\Sigma) = 1 - \frac{c \tilde{\sigma}_{S_1}^2}{c \tilde{\sigma}_{S_1}^2 + c \tilde{\sigma}_{S_2}^2} = \alpha(\Sigma).$$

Since both $\tilde{w}_S$ and α are scale-invariant, so is the final w . $\square$

Rust: `hrp_weights()`
in `quant-portfolio`
returns the weight vector
produced by this algorithm.
Tests verify Properties 1–3.

E.3.5. Property 3: Reduction to Inverse Variance

Claim: If the hierarchy has a single root cluster (i.e., no bisections are performed), HRP reduces to inverse-variance weighting.

Proof: With zero bisections, $k = 0$, the algorithm returns the initial allocation $\tilde{w} = \Sigma^{-1}\mathbf{1}/(\mathbf{1}^\top \Sigma^{-1}\mathbf{1})$, which is the standard inverse-variance portfolio. $\square$

E.3.6. Why HRP Outperforms Markowitz in Singular Cases

When Σ is singular (collinear assets), Σ^{-1} does not exist and Markowitz weights are undefined. HRP never requires a full matrix inverse: $\tilde{w}_S$ uses Σ_S^{-1} on small submatrices, which are typically well-conditioned because the quasi-diagonalisation groups correlated assets together. Even if a submatrix is singular, one can replace Σ_S^{-1} with a ridge-regularised inverse $\Sigma_S^{-1} = (\Sigma_S + \lambda I)^{-1}$ without breaking Properties 1–3 (the scaling arguments still hold).

E.4. E.4: Combinatorial Purged Cross-Validation

E.4.1. Setup

Combinatorial Purged Cross-Validation (CPCV, López de Prado 2018) generates N backtest paths from a dataset of T observations. The construction is parameterised by N (number of paths) and k (purging factor, i.e., test group size as a fraction of a path segment).

E.4.2. Construction

Partition the dataset into $N \cdot k$ contiguous groups $G_1, G_2, \dots, G_{Nk}$, each of size $|G| = T/(Nk)$. A backtest path is built by choosing k groups out of the Nk to be the test set, subject to two constraints:

1. The k test groups are separated by at least one training group (so that no observation in the train set leaks across a test boundary).
2. No two paths share the same test-set combination (paths are distinct).

E.4.3. Path Count

The number of distinct paths N_{paths} is the number of ways to choose k non-adjacent groups from Nk groups. This is a classical combinatorial identity.

Lemma: The number of ways to choose k non-adjacent items from n items arranged in a circle is

$$C_{\text{circle}}(n, k) = \frac{n}{n-k} \binom{n-k}{k},$$

and in a line (not a circle) is

$$C_{\text{line}}(n, k) = \binom{n-k+1}{k}.$$

Proof sketch: Use the inclusion–exclusion or the “stars-and-bars” argument. For the line version, represent the k chosen items as k stars with at least one unchosen bar between each pair of adjacent stars. After placing $k-1$ mandatory bars, there are $n-k-(k-1) = n-2k+1$ free bars to distribute among $k+1$ slots (before, between, and after the stars). The count is $\binom{n-2k+1+k}{k} = \binom{n-k+1}{k}$.

Theorem (CPCV path count): CPCV produces

$$N_{\text{paths}} = \binom{N \cdot k - k}{k} = \binom{k(N-1)}{k}$$

paths, where $n = Nk$ is the total number of groups.

Derivation: CPCV enforces that test groups in a single path are non-adjacent in the line of Nk groups. By the lemma with $n = Nk$ and choosing k groups:

$$N_{\text{paths}} = \binom{Nk - k + 1}{k}.$$

The $+1$ is sometimes dropped in the practitioner formulation because of the circular wrapping used to handle the boundary. The two versions differ only at the edges and agree up to $O(1)$.

E.4.4. Worked Example

For $N = 6$ paths and $k = 2$:

$$N_{\text{paths}} = \binom{6 \cdot 2 - 2}{2} = \binom{10}{2} = 45.$$

For $N = 10$ and $k = 5$:

$$N_{\text{paths}} = \binom{50 - 5}{5} = \binom{45}{5} = 1,221,759.$$

This is the combinatorial explosion that makes CPCV a powerful backtest engine: it produces many paths without leaking information across the train/test boundary.

E.4.5. Purging

Even after non-adjacency, labels computed at time t may depend on returns over a window $[t, t + h]$ (e.g., triple-barrier labels with holding period h). If t is in the train set but $t + h$ falls in the test set, the label leaks. Purging removes the h observations around each train/test boundary from the train set. Embargo further removes observations immediately after the test set to prevent autocorrelation-driven leakage.

Rust: `cpcv_splits()` in `quant-backtest` returns the train/test index pairs. The path count is verified in the test `t06_cpcv_count`.

E.4.6. Property: Every Observation Appears in Test Exactly Once

Claim: Each observation appears in exactly $\binom{Nk-k-1}{k-1}$ of the N_{paths} paths' test sets.

Proof: Fix an observation in group G_i . The number of paths for which G_i is a test group equals the number of ways to choose the remaining $k - 1$ test groups from the $Nk - k$ groups that are not G_i and not adjacent to G_i (there are $k - 1$ such groups to choose and $Nk - 1 - 2$ available groups). By the lemma:

$$\text{appearances} = \binom{Nk - 1 - (k - 1)}{k - 1} = \binom{Nk - k}{k - 1}.$$

By symmetry over all Nk groups:

$$\sum_{i=1}^{Nk} \text{appearances} = Nk \cdot \binom{Nk - k}{k - 1} = k \cdot \binom{Nk - k + 1}{k} = k \cdot N_{\text{paths}},$$

using the identity $k \binom{n}{k} = n \binom{n-1}{k-1}$. Each path has k test groups, so the total number of test-set appearances is $k \cdot N_{\text{paths}}$, which matches. $\square$

This confirms that every observation is tested exactly the right number of times: each observation appears in test $\binom{Nk-k}{k-1}$ times, and across all paths and groups the counts are uniform.

E.5. E.5: Backpropagation Chain Rule

E.5.1. Setup

Consider a feedforward neural network with layers $l = 1, \dots, L$. For layer l , the pre-activation is

$$z^l = W^l a^{l-1} + b^l,$$

and the activation is

$$a^l = \sigma^l(z^l),$$

where σ^l is the elementwise activation function. The output layer uses a loss $\mathcal{L}(a^L, y)$.

E.5.2. The Chain Rule

Backpropagation computes $\partial\mathcal{L}/\partial W^l$ and $\partial\mathcal{L}/\partial b^l$ for each layer. Define the error signal at layer l as

$$\delta^l = \frac{\partial\mathcal{L}}{\partial z^l}.$$

Output layer ($l = L$):

$$\delta^L = \frac{\partial\mathcal{L}}{\partial a^L} \odot \sigma'^L(z^L),$$

where $\odot$ is elementwise product. This is the chain rule applied to $\mathcal{L} \circ \sigma^L$.

Hidden layer ($l < L$): By the chain rule,

$$\delta^l = \left((W^{l+1})^\top \delta^{l+1} \right) \odot \sigma'^l(z^l).$$

Derivation: The loss depends on z^l only through $a^l = \sigma^l(z^l)$, and each a_i^l feeds into every z_j^{l+1} through W_{ji}^{l+1} . By the multivariate chain rule:

$$\frac{\partial\mathcal{L}}{\partial z_i^l} = \sum_j \frac{\partial\mathcal{L}}{\partial z_j^{l+1}} \frac{\partial z_j^{l+1}}{\partial z_i^l} = \sum_j \delta_j^{l+1} W_{ji}^{l+1} \sigma'^l(z_i^l) = \left[(W^{l+1})^\top \delta^{l+1} \right]_i \sigma'^l(z_i^l).$$

Weight gradient: The loss depends on W^l only through $z^l = W^l a^{l-1} + b^l$. Each W_{ij}^l affects only z_i^l :

$$\frac{\partial\mathcal{L}}{\partial W_{ij}^l} = \delta_i^l \frac{\partial z_i^l}{\partial W_{ij}^l} = \delta_i^l a_j^{l-1},$$

so in matrix form:

$$\frac{\partial\mathcal{L}}{\partial W^l} = \delta^l (a^{l-1})^\top.$$

Bias gradient: Similarly, b_i^l affects only z_i^l :

$$\frac{\partial\mathcal{L}}{\partial b_i^l} = \delta_i^l, \quad \text{i.e., } \frac{\partial\mathcal{L}}{\partial b^l} = \delta^l.$$

E.5.3. Complexity

The forward pass costs $O(\sum_l n_l n_{l-1})$ (matrix–vector products). The backward pass costs the same: each layer requires one matrix–vector product $(W^{l+1})^\top \delta^{l+1}$ and one outer product $\delta^l (a^{l-1})^\top$. The total cost of backpropagation is thus $O(\text{forward cost})$, which is optimal—any algorithm must at least read the weights to compute their gradient.

Rust: `backward()` in `quant-ml` implements the recursion above. The gradient-check test `t07_backprop` verifies it against a finite-difference estimate.

E.5.4. Numerical Gradient Check

A standard test for backpropagation correctness is the finite-difference check. For a small ϵ :

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^l} \approx \frac{\mathcal{L}(W_{ij}^l + \epsilon) - \mathcal{L}(W_{ij}^l - \epsilon)}{2\epsilon}.$$

The relative error between the analytical and numerical gradient

$$\text{rel-err} = \frac{\|\nabla_{\text{analytic}} - \nabla_{\text{numeric}}\|}{\|\nabla_{\text{analytic}}\| + \|\nabla_{\text{numeric}}\|}$$

should be below 10^{-7} for a correct implementation. Values above 10^{-3} indicate a bug in the chain rule. This is the basis of the `t07_backprop_gradient_check` test in `quant-ml`.

E.6. E.6: Adam Optimiser Derivation

E.6.1. Setup

Adam (Adaptive Moment Estimation, Kingma & Ba 2015) adapts the learning rate per parameter using estimates of the first and second moments of the gradient. Let θ_t be the parameter vector at step t , $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$ the gradient, α the learning rate, $\beta_1, \beta_2 \in [0, 1)$ the moment decay rates, and ϵ a numerical stability constant.

E.6.2. Bias-Corrected Moment Estimates

The raw moment estimates are exponential moving averages:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

where g_t^2 is elementwise. Initialising $m_0 = v_0 = 0$ introduces a bias toward zero, especially in early steps. We correct this bias.

First moment: Unrolling the recurrence,

$$m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i.$$

Taking expectations over the gradient distribution (treating g_t as a random variable):

$$\mathbb{E}[m_t] = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} \mathbb{E}[g_i].$$

If we assume $\mathbb{E}[g_i] = \mathbb{E}[g_t] = g$ (approximately stationary):

$$\mathbb{E}[m_t] = (1 - \beta_1) g \sum_{i=1}^t \beta_1^{t-i} = (1 - \beta_1) g \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t) g.$$

Hence $\mathbb{E}[m_t] = (1 - \beta_1^t)g$, so the bias is $1 - \beta_1^t$. The bias-corrected estimate is

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

Second moment: By the same argument with β_2 in place of β_1 and g_t^2 in place of g_t :

$$\mathbb{E}[v_t] = (1 - \beta_2^t)\mathbb{E}[g^2], \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

E.6.3. Update Rule

The Adam update is

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Substituting the bias-corrected estimates:

$$\theta_{t+1} = \theta_t - \alpha \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \epsilon}.$$

E.6.4. Approximate Invariance to Gradient Scale

Consider scaling the gradient by a constant c : $g_t \rightarrow c g_t$. Then $m_t \rightarrow c m_t$ and $v_t \rightarrow c^2 v_t$. The update becomes

$$\theta_{t+1} = \theta_t - \alpha \frac{c m_t / (1 - \beta_1^t)}{\sqrt{c^2 v_t / (1 - \beta_2^t)} + \epsilon} = \theta_t - \alpha \frac{c m_t / (1 - \beta_1^t)}{c \sqrt{v_t / (1 - \beta_2^t)} + \epsilon}.$$

For $c\sqrt{\hat{v}_t} \gg \epsilon$ (typical), the c cancels:

$$\theta_{t+1} \approx \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}},$$

which is scale-invariant. This is the key property of Adam: the effective learning rate per parameter is $\alpha / \sqrt{\hat{v}_t}$, which adapts to the curvature of the loss surface.

E.6.5. Connection to RMSProp and Momentum

Adam combines two earlier methods:

- **Momentum** (Polyak 1964): $\theta_{t+1} = \theta_t - \alpha m_t$. Uses only the first moment.
- **RMSProp**: $\theta_{t+1} = \theta_t - \alpha g_t / \sqrt{v_t}$. Uses only the second moment.
- **Adam**: Uses both, with bias correction.

Setting $\beta_1 = 0$ recovers RMSProp (with bias correction on the second moment). Setting $\beta_2 = 0$ recovers momentum (with bias correction on the first moment). The default values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across a wide range of problems.

Rust: `adam_step()`
in `quant-ml`
implements this update.
Convergence is verified
by `t08_sgd_convergence`
on the XOR problem.

E.6.6. Convergence Sketch

Under the assumptions of convex $\mathcal{L}$, bounded gradients $\|g_t\| \leq G$, and bounded domain $\|\theta - \theta^*\| \leq D$, Reddi et al. (2018) show that Adam achieves regret

$$R(T) = \sum_{t=1}^T (\mathcal{L}(\theta_t) - \mathcal{L}(\theta^*)) \leq \frac{D^2 \sqrt{\beta_2}}{2\alpha(1-\beta_1)} + \frac{\alpha\epsilon^2}{1-\beta_2} \ln(1+T) + \frac{D^2}{2(1-\beta_1)} \sum_{t=1}^T \frac{(1-\beta_1)^2}{(1-\beta_1^t)^2} \cdot \text{const.}$$

The regret is $O(\sqrt{T})$ when α is chosen as $\alpha_t = \alpha/\sqrt{t}$, giving $R(T)/T \rightarrow 0$ and hence convergence of the averaged iterate. (The original AMSGrad paper by Reddi et al. identifies a non-convergence mode for Adam with constant α in certain convex settings; AMSGrad fixes this by retaining the max of v_t over time.)

E.6.7. Practical Default Settings

Hyperparameter	Symbol	Default
Learning rate	α	0.001
First moment decay	β_1	0.9
Second moment decay	β_2	0.999
Numerical stabiliser	ϵ	10^{-8}

The learning rate is the only hyperparameter that usually needs tuning. A grid search over $\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$ covers most practical cases. The moment decays are robust and rarely need adjustment.

E.7. Summary

Derivation	Key takeaway
E.1 Kelly	$f^* = (pb - qa)/(ab)$; max log-wealth growth
E.2 Triple-barrier	Hitting probabilities via reflection principle
E.3 HRP	Weights sum to 1, scale-invariant, reduce to inverse variance
E.4 CPCV	$\binom{k(N-1)}{k}$ paths, each obs. tested equally
E.5 Backprop	Chain rule in $O(\text{forward cost})$
E.6 Adam	Bias-corrected moments, scale-invariant update

Each derivation ends in a formula that is implemented and tested in the companion Rust code. The math is not optional decoration—it is the contract between the equations in the chapters and the functions in the library.